

## Contents

|                                                                   |           |                                                                 |           |
|-------------------------------------------------------------------|-----------|-----------------------------------------------------------------|-----------|
| <b>1 Exam Reference and Proof Toolkit</b>                         | <b>2</b>  | <b>6.10 Minimum-cut structure and tie-breaking</b>              | <b>15</b> |
| 1.1 Choose a technique                                            | 2         | <b>A Targeted Practice Questions</b>                            | <b>15</b> |
| 1.2 What every solution must state                                | 2         | A.1 Questions                                                   | 15        |
| 1.3 Guarantee conventions and relaxation directions               | 2         | A.2 Quick true/false drill                                      | 16        |
| 1.4 Probability and inequalities used in the course               | 2         | A.3 Answer sketches                                             | 17        |
| 1.5 Small counterexamples and quantifiers to remember             | 2         | A.4 True/false answers                                          | 18        |
| <b>2 Greedy Approximation and Scaling</b>                         | <b>2</b>  | <b>B Pseudo-Polynomial / Exponential DP</b>                     | <b>18</b> |
| 2.1 Greedy algorithms for 0/1 knapsack                            | 2         | B.1 0/1 Knapsack — $O(nW)$                                      | 18        |
| 2.2 The best-of-candidates argument                               | 3         | B.2 $d$ -Dimensional Knapsack — $O(n \prod_j W_j)$              | 18        |
| 2.3 Subset sum and group admission                                | 3         | B.3 Subset Sum — $O(nB)$                                        | 18        |
| 2.4 Knapsack FPTAS: value-indexed dynamic programming             | 3         | B.4 Partition — $O(nS)$                                         | 19        |
| 2.5 PTAS and asymptotic PTAS                                      | 4         | B.5 TSP: Held-Karp — $O(n^{2^{2n}})$                            | 19        |
| 2.5.1 One-dimensional bin packing: asymptotic PTAS                | 4         | B.6 Minimum Vertex Cover — $O^*(2^n)$ Memoized DP               | 19        |
| 2.5.2 Fixed-dimensional knapsack: profit guessing and LP rounding | 4         | B.7 Maximum Independent Set — $O^*(2^n)$                        | 19        |
| 2.6 Weighted set cover: greedy charging                           | 4         | B.8 Shortest Superstring — $O(n^2 2^n)$                         | 19        |
| 2.7 Vertex cover from a maximal matching                          | 5         | B.9 Set Cover — $O(m 2^n)$                                      | 19        |
| 2.8 Triangle deletion and disjoint-obstruction packing            | 5         | B.10 Weighted Set Cover — $O(m 2^n)$                            | 19        |
| <b>3 Metric TSP and Eulerian Structure</b>                        | <b>5</b>  | B.11 Minimum Graph Coloring — $O^*(3^n)$ Simple Subset DP       | 19        |
| 3.1 General TSP, metric TSP, and the shortcut rule                | 5         | B.12 Bin Packing — $O(n 2^n)$                                   | 20        |
| 3.2 Euler circuits, open trails, and orientation                  | 6         | B.13 Hamiltonian Path — $O(n^2 2^n)$                            | 20        |
| 3.3 Double-tree: a 2-approximation                                | 6         | B.14 Steiner Tree — $O(3^k n + 2^k n^2)$                        | 20        |
| 3.4 Christofides: a 3/2-approximation                             | 6         | <b>C Background and Prerequisites</b>                           | <b>20</b> |
| 3.5 Held-Karp exact dynamic programming                           | 6         | C.1 The optimization viewpoint                                  | 20        |
| <b>4 Randomization and Derandomization</b>                        | <b>7</b>  | C.2 Asymptotics, input size, and cost models                    | 20        |
| 4.1 The indicator-variable proof template                         | 7         | C.3 Proof obligations for algorithms                            | 20        |
| 4.2 Weighted MAX-CUT, MAX- $k$ -CUT, and MAX-DICUT                | 7         | C.4 Algorithm-design tools retained from CS3230                 | 21        |
| 4.3 Conditional expectation: how to remove randomness             | 7         | C.5 Probability toolkit for randomized optimization             | 21        |
| 4.4 Weighted MAX-SAT by fair and biased assignment                | 7         | C.6 Logic and constraint modelling from CS4269                  | 21        |
| <b>5 Linear Programming and Rounding</b>                          | <b>7</b>  | C.7 From Boolean constraints to binary optimization             | 21        |
| 5.1 Modelling, relaxation, and the direction of the bound         | 7         | C.8 Complexity classes and reductions                           | 21        |
| 5.2 FOL feasibility and ILP formulation reference                 | 8         | C.9 A checklist for the rest of CS4234                          | 22        |
| 5.2.1 Metric TSP                                                  | 8         | <b>D Extended Probability and Mathematical Reference</b>        | <b>22</b> |
| 5.2.2 Single-machine scheduling                                   | 8         | D.1 Basic Probability Identities                                | 22        |
| 5.2.3 Parallel-machine scheduling                                 | 9         | D.2 Indicator Trick & Linearity                                 | 22        |
| 5.2.4 Bin packing                                                 | 9         | D.3 Expectation/Variance Identities                             | 22        |
| 5.2.5 Set packing                                                 | 9         | D.4 Inequalities (Concentration)                                | 22        |
| 5.2.6 Prize-collecting Steiner tree                               | 9         | D.5 Common Inequalities for Mathematical Proofs                 | 22        |
| 5.2.7 Balanced cut                                                | 9         | D.6 Common Taylor Series Expansions                             | 23        |
| 5.2.8 $k$ -minimum spanning tree                                  | 9         | D.7 Common Distributions (means/vars)                           | 23        |
| 5.2.9 Capacitated minimum spanning tree                           | 9         | D.8 Asymptotic Approximations                                   | 23        |
| 5.2.10 Feedback vertex set                                        | 10        | D.9 Classic “Balls-into-Bins” Facts                             | 23        |
| 5.2.11 Shortest superstring                                       | 10        | D.10 Coupon Collector                                           | 23        |
| 5.2.12 3-dimensional matching                                     | 10        | D.11 Conditioning & Decomposition                               | 24        |
| 5.2.13 Fair matching with group quotas                            | 10        | D.12 Jensen & Convexity                                         | 24        |
| 5.2.14 Capacitated facility location                              | 10        | D.13 “With High Probability” (w.h.p.)                           | 24        |
| 5.2.15 Maximum independent set                                    | 10        | <b>E Discrete Mathematics Reference</b>                         | <b>24</b> |
| 5.2.16 Capacitated knapsack                                       | 10        | E.1 Divisibility, GCD, and Bézout                               | 24        |
| 5.2.17 Higher-dimensional knapsack                                | 10        | E.2 Modular Arithmetic & Inverses                               | 24        |
| 5.3 Weighted vertex cover: threshold rounding                     | 11        | E.3 Chinese Remainder Theorem (CRT)                             | 24        |
| 5.4 Set cover: bounded frequency                                  | 11        | E.4 Prime Factorization Facts                                   | 24        |
| 5.5 Weighted MAX-SAT: LP rounding                                 | 11        | E.5 Primitive Roots & Orders                                    | 24        |
| 5.6 MAX-DICUT: two 1/2 rounding proofs                            | 11        | E.6 Combinatorics Mod Prime                                     | 24        |
| 5.7 Randomized set cover: repeat until feasible                   | 12        | E.7 Base Representation & Digit Properties                      | 24        |
| <b>6 Network Flow</b>                                             | <b>12</b> | E.8 Sieve & Multiplicative Precomputation                       | 24        |
| 6.1 Integer network flow and residual graph                       | 12        | E.9 Miscellaneous Handy Facts                                   | 24        |
| 6.2 $s$ - $t$ cuts and max-flow/min-cut                           | 12        | E.10 Planar Graph: Definition: Kuratowski’s theorem             | 24        |
| 6.3 Ford-Fulkerson and integral flows                             | 13        | <b>F Boolean-to-ILP Encoding Reference</b>                      | <b>25</b> |
| 6.4 Capacity scaling: $O(m^2(1 + \log C))$                        | 13        | F.1 Binary encodings of logical constraints                     | 25        |
| 6.5 Shortest augmenting paths: $O(nm^2)$                          | 13        | <b>G Flow Extensions: Lower Bounds and Demands</b>              | <b>25</b> |
| 6.6 Flow decomposition and reduction templates                    | 13        | <b>H Additional Algorithm Examples</b>                          | <b>25</b> |
| 6.7 Bipartite matching: Hall’s and König’s theorems               | 14        | H.1 An infinite Ford-Fulkerson augmentation sequence            | 25        |
| 6.8 Vertex-disjoint paths and Menger’s theorem                    | 15        | H.2 Random-endpoint vertex cover: an optional expectation proof | 26        |
| 6.9 Layered scheduling with two quota levels                      | 15        |                                                                 |           |

# 1 Exam Reference and Proof Toolkit

## 1.1 Choose a technique

| Problem cue                              | Method                           | Comparison with OPT                                                           |
|------------------------------------------|----------------------------------|-------------------------------------------------------------------------------|
| Knapsack, one blocked large item         | Density prefix or best singleton | Fractional optimum $\leq G + v_{\max}$                                        |
| Several nonnegative objectives           | Best of component optimizers     | $\text{OPT} \leq \sum_j f_j(x_j) \leq k \text{ ALG}$                          |
| Cost per newly covered element           | Greedy weighted set cover        | Element prices sum to ALG; each optimal set pays at most $H_n$ times its cost |
| Destroy fixed-size obstructions          | Maximal disjoint packing         | Each packed obstruction forces a distinct OPT action                          |
| At most $k$ covering choices per element | LP threshold $1/k$               | Feasible by a covering constraint; cost $\leq kLP^* \leq k \text{ OPT}$       |
| Metric travel and parity                 | MST, Euler tour, matching        | MST costs $\leq \text{OPT}$ ; odd-set matching costs $\leq \text{OPT}/2$      |
| Nonnegative local rewards                | Random labels and indicators     | $\text{OPT} \leq \text{total available reward}$                               |
| Deterministic guarantee required         | Conditional expectation          | Choose a branch whose expectation does not decrease                           |
| Fractional decisions                     | LP rounding                      | Per-object bound, weighted sum, relaxation bound                              |
| Compatibility and quotas                 | Layered network flow             | Integral flow at a specified target value                                     |
| Vertex throughput or disjointness        | Split each vertex                | Capacity on $v_{\text{in}} \rightarrow v_{\text{out}}$                        |
| Minimum cut, secondary objective         | Integer capacity perturbation    | $(m+1)c_e + 1$ preserves capacity priority                                    |
| Large numeric DP coordinate              | Scale values                     | Total rounding loss $\leq nK$                                                 |

## 1.2 What every solution must state

1. **Model and assumptions:** feasible objects, objective direction, nonnegative weights, metric properties, integrality, and any special graph structure.
2. **Algorithm:** explicit choices, stopping rule, and the actual output object.
3. **Feasibility:** why all constraints hold. For a reduction, prove both directions and explain how to decode the solution.
4. **Guarantee:** exhibit a lower bound on OPT for minimization, or an upper bound for maximization, then write the complete inequality chain.
5. **Runtime:** construction plus iterations times work per iteration. Numeric capacity  $C$  has  $O(\log(C+1))$  bits; polynomial in  $C$  need not mean polynomial in input size.

## 1.3 Guarantee conventions and relaxation directions

For  $\rho \geq 1$ , minimization means  $\text{OPT} \leq \text{ALG} \leq \rho \text{OPT}$  and maximization means  $\text{OPT}/\rho \leq \text{ALG} \leq \text{OPT}$ . Equivalently, a maximization algorithm retains a fraction  $\alpha = 1/\rho \leq 1$ . State the inequality explicitly when different sources use different ratio conventions. For randomized algorithms replace ALG by  $\mathbb{E}[\text{ALG}]$  in the promised bound; an expectation guarantee is not a guarantee for every run.

$$\begin{aligned} \text{Minimization relaxation: } & LP^* \leq \text{OPT}, \\ \text{Maximization relaxation: } & LP^* \geq \text{OPT}. \end{aligned}$$

These compare *optimal* relaxation values. An arbitrary fractional feasible objective does not automatically bound the integral optimum in the required direction. When  $\text{OPT} = 0$ , use inequalities instead of dividing by it.

## 1.4 Probability and inequalities used in the course

$$\begin{aligned} \mathbb{E}[\mathbf{1}_A] &= \Pr[A], \\ \mathbb{E}\left[\sum_i w_i X_i\right] &= \sum_i w_i \mathbb{E}[X_i], \\ \Pr\left[\bigcup_i A_i\right] &\leq \sum_i \Pr[A_i], \\ \Pr[A \mid B] &= \frac{\Pr[A \cap B]}{\Pr[B]} \quad (\Pr[B] > 0). \end{aligned}$$

Linearity and the union bound require no independence. Products of probabilities need the corresponding independence assumption. For a nonnegative random cost  $Z$  and a success event  $A$  of probability  $q > 0$ ,

$$\mathbb{E}[Z \mid A] = \frac{\mathbb{E}[Z\mathbf{1}_A]}{q} \leq \frac{\mathbb{E}[Z]}{q}.$$

Independent trials with success probability  $q$  have expected trial count  $1/q$  and failure probability  $(1-q)^t$  after  $t$  trials. Cost and success within one trial may be correlated.

$$\begin{aligned} 1 - x &\leq e^{-x}, \\ \prod_{i=1}^k a_i &\leq \left(\frac{1}{k} \sum_{i=1}^k a_i\right)^k \quad (a_i \geq 0), \\ \left(\sum_i a_i b_i\right)^2 &\leq \left(\sum_i a_i^2\right) \left(\sum_i b_i^2\right), \\ H_n &= \sum_{j=1}^n \frac{1}{j} \leq 1 + \ln n. \end{aligned}$$

If  $g$  is concave on  $[0, 1]$ , then  $g(y) \geq (1-y)g(0) + yg(1)$ . In particular,  $g(0) = 0$  implies  $g(y) \geq yg(1)$ . For candidate values,  $\max_i A_i \geq \frac{1}{k} \sum_i A_i$ ; this holds pointwise, so it also holds after taking expectations, without independence between candidates.

## 1.5 Small counterexamples and quantifiers to remember

- **Exact equivalence need not preserve approximation:**  $q$  disjoint edges plus an isolated vertex give a maximal-matching cover whose complement has size 1, while maximum independent set has size  $q+1$ .
- **Maximizing satisfied versus minimizing unsatisfied clauses:** a satisfiable 2-literal clause has optimum unsatisfied count 0, but fair assignment leaves it unsatisfied with probability  $1/4$ . A MAX-SAT guarantee does not transfer by subtraction.
- **Integral-flow theorem:** an integral maximum flow *exists*; fractional maximum flows may also exist. Two parallel branches merging before a unit bottleneck can each carry  $1/2$ .
- **Vertex-cover LP gap:** on  $K_n$ ,  $\text{OPT} = n-1$  and  $LP^* = n/2$ , so the worst-case gap is at least  $2 - 2/n$ . This is not a statement about every  $n$ -vertex graph.
- **Maximal versus maximum:** maximality is enough for the matching-based cover. Even a maximum matching's endpoints need not form a minimum cover (one edge suffices).
- **Numeric versus bit complexity:**  $O(mC)$  may be exponential in  $\log C$ . Capacity-scaling phases must include threshold 1 for integer capacities.

## 2 Greedy Approximation and Scaling

For each approximation below, feasibility and comparison with an optimal solution are separate proof obligations. Assume nonnegative values and costs, and positive item weights where density is used.

### 2.1 Greedy algorithms for 0/1 knapsack

We can relax 0/1 knapsack to fractional knapsack, for which prioritizing items by highest value density is optimal. Applying the same ordering to whole items is a natural first greedy approach, but it can be arbitrarily bad for 0/1 knapsack.

#### Value-density-first greedy

**Algorithm 1:** Value-density-first greedy for 0/1 knapsack

**Input:** Values  $v_1, \dots, v_n$ , weights  $w_1, \dots, w_n$ , capacity  $W$   
**Output:** A feasible item set  $S_D$  and its total value  $V_D$   
 $F \leftarrow \{i \in \{1, \dots, n\} : w_i \leq W\}$ ;  
Sort the items in  $F$  by non-increasing density  $v_i/w_i$ ;  
 $S_D \leftarrow \emptyset, W_D \leftarrow 0, V_D \leftarrow 0$ ;  
**foreach**  $i \in F$  **in density order do**  
    **if**  $W_D + w_i > W$  **then**  
        **break**;  
         $S_D \leftarrow S_D \cup \{i\}$ ;  
         $W_D \leftarrow W_D + w_i$ ;  
         $V_D \leftarrow V_D + v_i$ ;  
**return**  $(S_D, V_D)$ ;

#### Counterexample

For an integer capacity  $W > 2$ , consider

$$\begin{aligned} v &= \{2, W\} \\ w &= \{1, W\} \end{aligned}$$

The value-density-first algorithm takes only the first item and returns value 2, while the optimal solution takes the second item and returns  $W$ .

$$\frac{OPT}{ALG} = \frac{W}{2}$$

There is no constant approximation factor, meaning this algorithm can give an arbitrarily bad result.

### Identify the gap

From the counter example, we can see that a gap is caused by high-value low density (consequently high weight) items. This suggests another greedy approach: prioritize items by value rather than by density.

---

#### Algorithm 2: Value-first greedy for 0/1 knapsack

---

**Input:** Values  $v_1, \dots, v_n$ , weights  $w_1, \dots, w_n$ , capacity  $W$

**Output:** A feasible item set  $S_V$  and its total value  $V_V$

$F \leftarrow \{i \in \{1, \dots, n\} : w_i \leq W\}$ ;

Sort the items in  $F$  by non-increasing value  $v_i$ ;

$S_V \leftarrow \emptyset$ ,  $W_V \leftarrow 0$ ,  $V_V \leftarrow 0$ ;

**foreach**  $i \in F$  *in value order* **do**

**if**  $W_V + w_i \leq W$  **then**

$S_V \leftarrow S_V \cup \{i\}$ ;

$W_V \leftarrow W_V + w_i$ ;

$V_V \leftarrow V_V + v_i$ ;

**return**  $(S_V, V_V)$ ;

---

This solves the problem of overlooking a high-value item, but it can still be arbitrarily bad.

### Counterexample for value-first greedy

Let the capacity be an integer  $W > 2$ , with one heavy item and  $W$  unit items:

$$v = \{2, \underbrace{1, 1, \dots, 1}_{W \text{ items}},\}$$

$$w = \{W, \underbrace{1, 1, \dots, 1}_{W \text{ items}}\}.$$

Value-first greedy takes the heavy item, fills the knapsack, and returns value 2. The optimal solution takes all  $W$  unit items and has value  $W$ . Therefore,

$$\frac{OPT}{ALG} = \frac{W}{2} \rightarrow \infty.$$

Thus neither density-first nor value-first greedy has a constant approximation factor by itself.

### Modified greedy: keep the better candidate

The fractional solution is a whole-item density-greedy prefix plus a fraction of the first item that does not fit. This identifies the two candidates needed for the repair: the density-greedy prefix and the best feasible singleton.

---

#### Algorithm 3: Modified two-candidate greedy for 0/1 knapsack

---

**Input:** Values  $v_1, \dots, v_n$ , weights  $w_1, \dots, w_n$ , capacity  $W$

**Output:** A feasible item set  $S$  and its total value

$F \leftarrow \{i \in \{1, \dots, n\} : w_i \leq W\}$ ;

**if**  $F = \emptyset$  **then**

**return**  $(\emptyset, 0)$ ;

Sort the items in  $F$  by non-increasing density  $v_i/w_i$ ;

$S_G \leftarrow \emptyset$ ,  $W_G \leftarrow 0$ ,  $V_G \leftarrow 0$ ;

**foreach**  $i \in F$  *in density order* **do**

**if**  $W_G + w_i > W$  **then**

**break**;

$S_G \leftarrow S_G \cup \{i\}$ ;

$W_G \leftarrow W_G + w_i$ ;

$V_G \leftarrow V_G + v_i$ ;

$i_{\max} \leftarrow \arg \max_{i \in F} v_i$ ;

$S_M \leftarrow \{i_{\max}\}$ ,  $V_M \leftarrow v_{i_{\max}}$ ;

**if**  $V_G \geq V_M$  **then**

**return**  $(S_G, V_G)$ ;

**else**

**return**  $(S_M, V_M)$ ;

---

The value-first greedy solution always contains the best feasible singleton, so it could replace the singleton candidate and would preserve the guarantee. The singleton is used because it is the minimal repair required by the proof.

## 2.2 The best-of-candidates argument

Let  $g(x) = \sum_{i=1}^k f_i(x)$  on a common feasible domain, with  $f_i(x) \geq 0$ . Suppose  $x_i$  is a *global* maximizer of  $f_i$ . Return the candidate  $x_j$  with largest  $g(x_j)$ . For an optimum  $x^*$ ,

$$\begin{aligned} \text{OPT} &= \sum_i f_i(x^*) \leq \sum_i f_i(x_i) \\ &\leq \sum_i g(x_i) \leq k \max_i g(x_i) = k \text{ALG}. \end{aligned}$$

Nonnegativity justifies the middle inequality. Feasibility follows because every candidate lies in the same domain. Evaluate all  $k$  candidates in addition to the cost of finding them.

For modified knapsack, the fractional optimum consists of a whole-item density prefix of value  $G$  and at most one fractional item  $\theta v_j$ ,  $0 \leq \theta \leq 1$ . All overweight items were discarded, so the best feasible singleton has value at least  $v_j$ . Therefore

$$\begin{aligned} \text{OPT} &\leq \text{OPT}_{\text{frac}} = G + \theta v_j \leq G + v_{\max} \\ &\leq 2 \max\{G, v_{\max}\} = 2 \text{ALG}. \end{aligned}$$

If all items fit, the density prefix itself is optimal. Sorting dominates the running time,  $O(n \log n)$ .

## 2.3 Subset sum and group admission

Given positive item sizes  $a_i$  and capacity  $B$ , maximize the admitted total subject to the capacity. Groups must be accepted in full. Ignore items larger than  $B$ .

---

#### Algorithm 4: Largest-first admission

---

**Input:** Positive sizes  $a_1, \dots, a_n$ ; fixed capacity  $B$

**Output:** A feasible index set  $S$

Sort indices in nonincreasing order of  $a_i$ ;

$S \leftarrow \emptyset$ ;  $A \leftarrow 0$ ;

**foreach** *index*  $i$  *in that order* **do**

**if**  $A + a_i \leq B$  **then**

$S \leftarrow S \cup \{i\}$ ;  $A \leftarrow A + a_i$ ;

**return**  $S$ ;

---

*Proof.* The admission test preserves  $A \leq B$ . If the final total  $A \geq B/2$ , then  $A \geq \text{OPT}/2$ . Otherwise the final remaining capacity exceeds  $B/2$ . There cannot have been a feasible item larger than  $B/2$ : the first such item in sorted order would have fitted into the initially empty solution, contradicting  $A < B/2$ . Every feasible item is therefore at most  $B/2$  and fits whenever considered, because remaining capacity never falls below its final value. All feasible items were admitted, so  $A = \text{OPT}$ . Sorting gives  $O(n \log n)$  time.  $\square$

**Tight family:** capacity  $2k$ , sizes  $k+1, k, k$ . Greedy returns  $k+1$ , whereas  $\text{OPT} = 2k$ ; the retained fraction tends to  $1/2$ .

**Read-only array and constant extra space.** Scan once for a largest individually feasible item, of size  $M$ . If none exists, return the empty solution. If  $M \geq B/2$ , return that item. Otherwise scan again in the original order, admitting each item when it fits. If the result is below  $B/2$ , every feasible item (all smaller than  $B/2$ ) fitted, so it is optimal. Otherwise it is at least  $\text{OPT}/2$ . Time is  $O(n)$  and extra space is  $O(1)$  when admissions can be output as a stream.

## 2.4 Knapsack FPTAS: value-indexed dynamic programming

An FPTAS returns a feasible value at least  $(1 - \epsilon)\text{OPT}$  for  $0 < \epsilon < 1$ , with time polynomial in input length and  $1/\epsilon$ . Assume positive integer weights and nonnegative integer values. Discard items with  $w_i > W$ , redefine  $n$  to count the remaining items, and let  $V = \max_i v_i$ . If no items remain or  $V = 0$ , return the empty set.

**Exact pseudo-polynomial DP.** For  $0 \leq i \leq n$  and  $0 \leq p \leq nV$ , define  $D[i, p]$  as the minimum weight using items  $1, \dots, i$  with total value exactly  $p$ . Set  $D[0, 0] = 0$  and other impossible states to  $+\infty$ . The recurrence is

$$D[i, p] = \begin{cases} \min\{D[i-1, p], w_i + D[i-1, p - v_i]\}, & p \geq v_i, \\ D[i-1, p], & p < v_i. \end{cases}$$

Compute rows in increasing  $i$ . Return the largest  $p$  with  $D[n, p] \leq W$ , and backtrack through minimizing choices to recover its subset. Every optimum either omits item  $i$  or contains it, leaving value  $p - v_i$  among earlier items; conversely both recurrence alternatives describe valid subsets. This proves the state invariant by induction. There are  $O(n^2V)$  states and constant work per state, giving  $O(n^2V)$  time and space. Two rows give  $O(nV)$  space when only the optimum value is needed.

**Scale the value coordinate, not the weights.** Set  $K = \epsilon V/n$  and  $\hat{v}_i = \lfloor v_i/K \rfloor$ . Run the value-indexed DP using  $\hat{v}_i$  and the unchanged weights. Since  $\hat{v}_i \leq n/\epsilon$ , the total scaled value is at most  $n^2/\epsilon$  and the runtime is  $O(n^3/\epsilon)$  arithmetic operations. Weights and arithmetic are represented with polynomially many bits; rational  $\epsilon$  can be handled by integer arithmetic in  $\lfloor nv_i/(\epsilon V) \rfloor$ .

*Approximation guarantee.* Let  $S$  maximize scaled value and let  $S^*$  be an original optimum. For each item,  $K\hat{v}_i \leq v_i < K(\hat{v}_i + 1)$ . Feasibility is unchanged because weights were not rounded. Writing  $v(S) = \sum_{i \in S} v_i$  and similarly for  $\hat{v}(S)$ ,

$$\begin{aligned} v(S) &\geq K\hat{v}(S) \geq K\hat{v}(S^*) \\ &\geq v(S^*) - K|S^*| \geq \text{OPT} - nK \\ &= \text{OPT} - \epsilon V \geq (1 - \epsilon) \text{OPT}. \end{aligned}$$

The last step uses  $V \leq \text{OPT}$ : the largest remaining item fits by itself. This is why overweight items must be discarded *before* defining  $V$ .  $\square$

Scaling values does not reduce the state space of a weight-indexed  $O(nW)$  DP. Neither  $V$  nor  $W$  is a constant; the improvement comes from bounding the scaled coordinate.

## 2.5 PTAS and asymptotic PTAS

For each fixed  $\epsilon > 0$ , a PTAS runs in polynomial time and returns at least  $(1 - \epsilon) \text{OPT}$  for maximization or at most  $(1 + \epsilon) \text{OPT}$  for minimization. Its exponent may depend on  $1/\epsilon$ ; an FPTAS is also polynomial in  $1/\epsilon$ . Both schemes below assume the items are explicitly listed and their numerical data have polynomial bit length.

### 2.5.1 One-dimensional bin packing: asymptotic PTAS

For sizes  $s_i \in (0, 1]$  and unit bins (model in §5.2.4), the guarantee is *asymptotic*: for every fixed  $\epsilon \in (0, 1)$ , use  $\delta = \epsilon/3$  to obtain at most  $(1 + \epsilon) \text{OPT} + 1$  bins. An absolute PTAS is impossible unless  $P = NP$ : a ratio below  $3/2$  would decide whether a bin-packing instance needs two or at least three bins.

1. Split items into large ( $s_i > \delta$ ) and small ( $s_i \leq \delta$ ). Sort the  $N$  large items by decreasing size. If  $N = 0$ , skip to step 4.
2. Put the sorted large items into consecutive groups of size at most  $g = \lceil N/K \rceil$ , where  $K = \lceil \delta^{-2} \rceil$ . Set aside the first (largest) group. In every other group, round each size *up* to the largest size in its group.
3. Pack the rounded items optimally by enumerating bin configurations and their multiplicities; then restore their original sizes and pack each set-aside item alone. There are at most  $K$  rounded sizes and at most  $\lceil 1/\delta \rceil$  large items per bin, so the number of configurations depends only on  $\epsilon$ . Enumerating at most  $N$  copies of each takes  $n^{O_\epsilon(1)}$  time.
4. Insert small items by first fit into all existing bins, opening a new bin only when none has room.

**Why it works.** A rounded item in group  $j \geq 2$  fits in a slot occupied by an original item of group  $j - 1$ . Shifting all groups this way shows that the rounded items need at most  $\text{OPT}$  bins. The first group adds at most  $g \leq \delta \text{OPT} + 1$  bins, since  $N\delta < \sum_{\text{large}} s_i \leq \text{OPT}$ . If no new bin is opened for small items, the count is at most  $(1 + \delta) \text{OPT} + 1$ . Otherwise every bin except possibly the last ends more than  $1 - \delta$  full, so the count is at most  $\text{OPT}/(1 - \delta) + 1 \leq (1 + \epsilon) \text{OPT} + 1$ .

### 2.5.2 Fixed-dimensional knapsack: profit guessing and LP rounding

Use the  $d$ -resource 0/1 model in §5.2.17, with nonnegative profits and *fixed*  $d$ . Let  $k = \lceil d/\epsilon \rceil$ . Enumerate every feasible subset of at most  $k$  items as a candidate solution. For each feasible  $k$ -item set  $T$ , let  $q = \min_{i \in T} v_i$ . Force  $T$  into the solution, discard other items with  $v_i > q$ , subtract  $T$ 's resource use from each capacity, and solve the residual LP with  $0 \leq x_i \leq 1$ . Choose an optimal *basic* LP solution and round every fractional  $x_i$  down to zero. Return the best solution over all guesses.

**Why it works.** If an optimum has fewer than  $k$  items, it is an enumerated candidate. Otherwise one guess  $T$  consists of its  $k$  highest-profit items, so its remaining items all survive the profit cutoff. The residual LP therefore upper-bounds the optimum's remaining profit. A basic solution has at most  $d$  fractional variables: with more, some fractional coordinates could move while preserving the at most  $d$  resource constraints. Rounding down preserves feasibility and loses at most  $dq \leq (d/k) \text{OPT} \leq \epsilon \text{OPT}$ , because  $\text{OPT} \geq kq$ . Thus the result has value at least  $(1 - \epsilon) \text{OPT}$  in  $n^{O(d/\epsilon)}$  poly(input bits) time. The fixed- $d$  condition matters: this bound is not a PTAS when  $d$  is part of the input.

## 2.6 Weighted set cover: greedy charging

Given a universe  $U$  and a family of candidate sets  $\mathcal{S}_U$ , each  $S \in \mathcal{S}_U$  has a non-negative cost  $\text{cost}(S)$ . The weighted set cover problem asks for a subfamily

$S' \subseteq \mathcal{S}_U$  such that

$$\bigcup_{S \in S'} S = U,$$

while minimizing  $\sum_{S \in S'} \text{cost}(S)$ .

### Greedy score

Let  $R$  be the set of elements that remain uncovered. For every set that can still cover at least one new element, define

$$\text{score}(S) = \frac{\text{cost}(S)}{|S \cap R|}.$$

This is the average cost per newly covered element. Greedy repeatedly chooses a set of minimum score.

---

**Input:** Universe  $U$ , candidate family  $\mathcal{S}_U$  with costs

**Output:** A set cover  $S' \subseteq \mathcal{S}_U$

$R \leftarrow U$  and  $S' \leftarrow \emptyset$ ;

**while**  $R \neq \emptyset$  **do**

$A_i \leftarrow \arg \min_{\substack{S \in \mathcal{S}_U \\ S \cap R \neq \emptyset}} \frac{\text{cost}(S)}{|S \cap R|}$ ;

$S' \leftarrow S' \cup \{A_i\}$ ;

$R \leftarrow R \setminus A_i$ ;

**return**  $S'$ ;

---

We prove

$$\text{ALG} \leq H_n \text{OPT} \leq (1 + \ln n) \text{OPT}, \quad H_n = \sum_{j=1}^n \frac{1}{j}, \quad n = |U|.$$

*Proof.* Assume the instance is feasible, so  $\bigcup_{S \in \mathcal{S}_U} S = U$ . The invariant is

$$U \setminus R = \bigcup_{A \in S'} A:$$

the elements removed from  $R$  are exactly those covered by the sets already selected. If  $R \neq \emptyset$ , feasibility guarantees a candidate set with  $S \cap R \neq \emptyset$ , so every iteration removes at least one element from  $R$ . Hence the algorithm terminates with  $R = \emptyset$  and  $S'$  is a cover.

**Blue:** express the greedy cost exactly. **Red:** bound prices using **OPT**. **Purple:** combine the two.

### 1. Rewrite the greedy cost as element prices

Let  $R_i$  be the set of uncovered elements at the beginning of iteration  $i$ , and let  $A_i$  be the set selected in that iteration. The elements newly covered in this iteration are exactly  $A_i \cap R_i$ . For each  $e \in A_i \cap R_i$ , define

$$p(e) = \frac{\text{cost}(A_i)}{|A_i \cap R_i|}.$$

This is an accounting price, not an intrinsic cost of  $e$ . Already-covered elements of  $A_i$  are not charged again. The prices assigned in iteration  $i$  recover exactly the cost of  $A_i$ :

$$\sum_{e \in A_i \cap R_i} p(e) = |A_i \cap R_i| \frac{\text{cost}(A_i)}{|A_i \cap R_i|} = \text{cost}(A_i).$$

Every element is newly covered exactly once, so summing over all iterations gives the exact identity

$$\text{ALG} = \sum_{A \in S'} \text{cost}(A) = \sum_{e \in U} p(e).$$

## 2. Use one optimal set to bound its elements' prices

Let  $\mathcal{O} \subseteq \mathcal{S}_U$  be a fixed optimal cover and define

$$\text{OPT} = \sum_{S \in \mathcal{O}} \text{cost}(S).$$

Fix any one set  $S^* \in \mathcal{O}$  and let  $m = |S^*|$ . The set  $S^*$  may or may not also be selected by greedy; the proof only needs it to be an available candidate for comparison.

Order its elements by when greedy first covers them:

$$S^* = \{e_1, e_2, \dots, e_m\}.$$

If several elements are first covered in the same iteration, break the tie arbitrarily. Let  $j(t)$  be the iteration in which  $e_t$  is first covered and define

$$r_t = |S^* \cap R_{j(t)}|.$$

Immediately before that iteration, at least  $e_t, e_{t+1}, \dots, e_m$  are still uncovered. Therefore

$$r_t \geq m - t + 1.$$

At iteration  $j(t)$ , greedy chooses  $A_{j(t)}$  with minimum score. Comparing its score with the available candidate  $S^*$  gives

$$\begin{aligned} p(e_t) &= \frac{\text{cost}(A_{j(t)})}{|A_{j(t)} \cap R_{j(t)}|} \\ &\leq \frac{\text{cost}(S^*)}{|S^* \cap R_{j(t)}|} = \frac{\text{cost}(S^*)}{r_t} \\ &\leq \frac{\text{cost}(S^*)}{m - t + 1}. \end{aligned}$$

The same inequality handles simultaneous coverage: tied elements are all still in  $R_{j(t)}$  before that iteration, making  $r_t$  larger and the bound only stronger. Summing over the elements of this one optimal set,

$$\begin{aligned} \sum_{e \in S^*} p(e) &= \sum_{t=1}^m p(e_t) \\ &\leq \text{cost}(S^*) \sum_{t=1}^m \frac{1}{m - t + 1} \\ &= \text{cost}(S^*) H_m \leq \text{cost}(S^*) H_n. \end{aligned}$$

This is a bound on the *accounting prices of elements in  $S^*$* ; it is not a separate cost paid by greedy to cover only those elements.

## 3. Bridge the optimal-set bounds back to ALG

Every  $e \in U$  belongs to at least one set in the optimal cover  $\mathcal{O}$ . Thus the middle sum below counts every  $p(e)$  at least once (and may count it more than once when optimal sets overlap):

$$\begin{aligned} \text{ALG} &= \sum_{e \in U} p(e) \\ &\leq \sum_{S^* \in \mathcal{O}} \sum_{e \in S^*} p(e) \\ &\leq \sum_{S^* \in \mathcal{O}} H_n \text{cost}(S^*) \\ &= H_n \text{OPT}. \end{aligned}$$

Therefore greedy weighted set cover is an  $H_n$ -approximation, and since  $H_n \leq 1 + \ln n$ , it is also a  $(1 + \ln n)$ -approximation.  $\square$

**Running time.** Each iteration covers at least one new element, so there are at most  $n = |U|$  iterations. If  $m$  candidate sets are represented by their incidence matrix, recomputing all marginal scores costs  $O(mn)$  per iteration, for  $O(mn^2)$  time. Faster updates are possible, but this already establishes polynomial time. If  $U$  is empty, return the empty cover.

## 2.7 Vertex cover from a maximal matching

View each edge as an element and each vertex as the set of its incident edges. Every element then belongs to exactly two sets. For the *unweighted* problem, greedily construct any maximal matching  $M$  and return the set  $C$  of all endpoints of  $M$ .

---

**Algorithm 5:** Maximal-matching vertex cover

---

**Input:** Undirected graph  $G = (V, E)$   
**Output:** A vertex cover  $C \subseteq V$   
 $M \leftarrow \emptyset$ ; mark all vertices unmatched;  
**foreach**  $\{u, v\} \in E$  **do**  
    **if** both  $u$  and  $v$  are unmatched **then**  
         $M \leftarrow M \cup \{\{u, v\}\}$ ; mark  $u, v$  matched;  
**return**  $C = \bigcup_{e \in M} e$ ;

---

*Proof.* An uncovered edge would have two unmatched endpoints and could be added to  $M$ , contrary to maximality. Any vertex cover must take a distinct endpoint from each pairwise disjoint matching edge, so  $|M| \leq \text{OPT}$ . Hence  $|C| = 2|M| \leq 2\text{OPT}$ . The scan takes  $O(|V| + |E|)$  time. Maximum matching is unnecessary; maximality proves coverage, while disjointness proves the lower bound.  $\square$

This unweighted cardinality proof does not give a weighted-vertex-cover guarantee by simply summing endpoint weights; use LP rounding for that version.

**Why complements do not preserve approximation.** The complement of any vertex cover is an independent set, and  $\text{OPT}_{IS} = |V| - \text{OPT}_{VC}$ . But  $|C| \leq 2\text{OPT}_{VC}$  only implies  $|V \setminus C| \geq 2\text{OPT}_{IS} - |V|$ , which can be useless. On  $q$  disjoint edges plus one isolated vertex, the matching algorithm selects all  $2q$  nonisolated vertices. Its complement has size 1, while  $\text{OPT}_{IS} = q + 1$ . Thus exact optimization equivalence does not imply approximation preservation.

## 2.8 Triangle deletion and disjoint-obstruction packing

To delete as few edges as possible so that an undirected graph becomes triangle-free, repeatedly find a remaining triangle and delete *all three* of its edges. Return the union  $F$  of deleted edges.

*Proof.* The process stops with no triangle. The selected triangles are pairwise edge-disjoint, because earlier selected edges were deleted. If  $k$  triangles were selected, every feasible deletion set must remove at least one distinct edge from each of these triangles, so  $\text{OPT} \geq k$ . Our output has  $|F| = 3k \leq 3\text{OPT}$ . One implementation scans all vertex triples once with an adjacency matrix: select a triangle only if none of its edges was selected earlier. At the end, any surviving triangle would have been eligible when its triple was scanned, a contradiction. Time is  $O(|V|^3 + |E|)$  with constant-time edge tests.  $\square$

**Counterexample to deleting an arbitrary single edge per triangle:** take a shared edge  $ab$  and vertices  $v_1, \dots, v_q$ , each joined to both  $a$  and  $b$ . Deleting  $ab$  solves the instance, while repeatedly deleting a spoke can require  $q$  edges.

**Reusable generalization.** For fixed forbidden subgraphs whose occurrences cannot be created by deleting edges, suppose every forbidden configuration has  $r$  edges for a fixed constant  $r$ . Select a maximal collection of edge-disjoint configurations and delete all their edges. Maximality gives feasibility, disjointness gives  $\text{OPT} \geq k$ , and  $\text{ALG} = rk \leq r\text{OPT}$ . Detection must be polynomial; the fact that  $r$  is fixed is relevant. Induced patterns need separate care: edge deletion can create a new induced pattern. Weighted deletions need a different cost argument.

**Greedy by number of triangles removed.** This is weighted set cover on the universe of original triangles, with each graph edge covering the triangles that contain it. Recompute marginal coverage after each deletion. The generic proof gives an  $H_N$  bound, where  $N$  is the number of original triangles; it does not by itself establish the factor 3 above.

## 3 Metric TSP and Eulerian Structure

### 3.1 General TSP, metric TSP, and the shortcut rule

The traveling salesperson problem asks for a minimum-cost Hamiltonian cycle: visit every vertex exactly once and return to the start. Throughout the approximation algorithms below, the input is a complete undirected graph with nonnegative symmetric costs satisfying the triangle inequality

$$c(u, w) \leq c(u, v) + c(v, w).$$

These are *metric* costs. Completeness supplies each shortcut edge; triangle inequality ensures that shortcutting a walk never increases its cost. Repeatedly bypass already visited vertices, retaining the final return to the start, to turn a spanning closed walk into a Hamiltonian cycle.

**Why the metric assumption matters.** General TSP has no polynomial-time approximation with any fixed finite factor  $\rho \geq 1$ , unless  $P = NP$ . Reduce Hamiltonian Cycle on an unweighted graph  $G$  with  $n$  vertices: complete the graph, give original edges cost 1 and nonedges cost an integer  $B > \rho n$ . A YES instance has  $\text{OPT} = n$ , so a  $\rho$ -approximation returns cost at most  $\rho n$ . In a NO instance every tour uses a nonedge, hence has cost at least  $B > \rho n$ . The algorithm would distinguish the two cases in polynomial time. For fixed  $\rho$ ,  $B$  has polynomial encoding length. These constructed costs need not satisfy triangle inequality; a cost- $B$  shortcut can replace a two-edge detour of cost 2. Metric TSP remains NP-hard: assign costs 1 to edges of a Hamiltonian Cycle instance and 2 to nonedges. Every triangle satisfies the metric inequality, and a tour of cost  $n$  exists exactly when the original graph is Hamiltonian. This proves exact hardness, not the absence of constant-factor approximation.

### 3.2 Euler circuits, open trails, and orientation

An Euler trail uses every *edge occurrence* exactly once; vertices may repeat. Parallel edges are distinct occurrences. An Euler circuit is a closed Euler trail. A Hamiltonian cycle instead visits every vertex once.

**Theorem 3.1** (Euler’s criterion). An undirected multigraph with at least one edge has an Euler circuit iff all its nonisolated vertices are connected and every vertex has even degree. It has an Euler trail between distinct vertices  $s, t$  iff all nonisolated vertices are connected and precisely  $s, t$  have odd degree.

*Proof.* In a closed traversal, each arrival at a vertex is paired with a departure, so every degree is even. For an open traversal only the start and end have one unpaired incident edge. Conversely, start at any nonisolated vertex and keep following unused edges. An even-degree vertex other than the start cannot be the first place where this trail gets stuck: after arriving, an unused departure remains. Thus the trail closes. If edges remain, connectivity gives a vertex of the current circuit incident to unused edges. The unused-edge degrees are still even, so construct another closed trail there and splice it into the first. Repeat until every edge is used (Hierholzer’s algorithm). For the open case add one extra  $\{s, t\}$  edge, find an Euler circuit, rotate it to place that edge last, and delete the extra edge.  $\square$

Using adjacency lists and marking each edge occurrence, Hierholzer’s algorithm runs in  $O(|V| + |E|)$  time. The handshaking identity  $\sum_v \deg(v) = 2|E|$  also shows that the number of odd-degree vertices is even. Adding one edge toggles the parity of its two endpoints.

**Orienting an even-degree connected graph.** Find an Euler circuit and orient each edge in the direction in which the circuit traverses it. The resulting directed graph is strongly connected: from any vertex  $u$ , follow the directed circuit cyclically until any desired vertex  $v$  is reached. Also  $\deg^+(v) = \deg^-(v)$  at every vertex. This proof concerns the nonisolated vertices; a disconnected even-degree graph does not become strongly connected by orienting its components.

### 3.3 Double-tree: a 2-approximation

1. Compute a minimum spanning tree  $T$ .
2. Duplicate every edge of  $T$ , retaining both copies in a multigraph.
3. Find an Euler circuit in this connected even-degree multigraph.
4. Shortcut repeated vertex visits to obtain the output tour  $H$ .

**Feasibility.** Doubling makes every degree even; the spanning tree ensures connectivity. The Euler circuit visits all vertices, and shortcutting yields a Hamiltonian cycle in the complete graph.

**Lower bound and guarantee.** Deleting any edge of an optimal tour leaves a spanning tree, so  $c(T) \leq \text{OPT}$ . Therefore

$$c(H) \leq 2c(T) \leq 2\text{OPT}.$$

The first inequality uses triangle inequality; the second uses the MST lower bound. Dense Prim takes  $O(n^2)$  time on the complete graph; doubling, Euler traversal, and shortcutting take  $O(n)$  further time.

**Four-vertex metric counterexample to optimality.** Use the shortest-path metric of the unit-cost cycle  $A, B, D, C, A$ :

| $c$ | $A$ | $B$ | $C$ | $D$ |
|-----|-----|-----|-----|-----|
| $A$ | 0   | 1   | 1   | 2   |
| $B$ | 1   | 0   | 2   | 1   |
| $C$ | 1   | 2   | 0   | 1   |
| $D$ | 2   | 1   | 1   | 0   |

The tree  $\{AB, AC, CD\}$  is an MST of cost 3. The doubled tree admits Euler circuit  $A, B, A, C, D, C, A$ . First-visit shortcutting gives  $A, B, C, D, A$ , of cost  $1 + 2 + 1 + 2 = 6$ . The original cycle has cost 4 and is optimal because every tour has four edges, each of cost at least 1. Thus a valid double-tree execution need not be optimal, even on a metric.

### 3.4 Christofides: a 3/2-approximation

1. Compute an MST  $T$ , and let  $O = \{v : \deg_T(v) \text{ is odd}\}$ .
2. Find a *minimum-weight perfect matching*  $M$  on the complete graph induced by  $O$ , using the original metric costs.
3. Form the multigraph  $T \cup M$ , find an Euler circuit, and shortcut repeated vertices to produce a tour  $H$ .

**Feasibility.** The set  $O$  has even cardinality. Every vertex in  $O$  gains exactly one incident matching edge and becomes even; all other degrees stay even. The tree ensures connectivity. Keep parallel copies if a matching edge is also in  $T$ .

**Lemma 3.2** (Matching budget). The matching chosen above has cost  $c(M) \leq \text{OPT}/2$ .

*Proof.* Take an optimal tour and shortcut it to retain only the vertices of  $O$ . The resulting even-length cyclic walk has cost at most  $\text{OPT}$ . Its alternating edges split into two perfect matchings  $M_1, M_2$  on  $O$ . Therefore, by minimum weight of  $M$ ,

$$c(M) \leq \min\{c(M_1), c(M_2)\} \leq \frac{c(M_1) + c(M_2)}{2} \leq \frac{\text{OPT}}{2}.$$

If  $O$  has two vertices, the restricted cyclic walk traverses their edge twice; its two edge occurrences give the same one-edge matching. If  $O = \emptyset$ , use the empty matching.  $\square$

Combining the spanning and matching budgets with metric shortcutting gives

$$c(H) \leq c(T) + c(M) \leq \text{OPT} + \frac{1}{2} \text{OPT} = \frac{3}{2} \text{OPT}.$$

All steps are polynomial time; minimum-weight perfect matching is the main additional subroutine beyond double-tree. A maximal matching is insufficient: it may leave odd vertices unmatched, and an arbitrary perfect matching has no  $\text{OPT}/2$  cost guarantee.

### 3.5 Held–Karp exact dynamic programming

This exact algorithm works for arbitrary tour costs; triangle inequality is unnecessary. Fix a root  $r$ . For  $r \in S \subseteq V$  and  $v \in S$ , let  $D[S, v]$  be the minimum cost of a path starting at  $r$ , ending at  $v$ , and visiting exactly the vertices in  $S$ , once each.

**Bases.**  $D[\{r\}, r] = 0$ . Set all impossible states to  $\infty$ , including  $D[S, r]$  for  $|S| > 1$  and states with  $r \notin S$  or  $v \notin S$ . For  $v \neq r$ ,

$$D[S, v] = \min_{u \in S \setminus \{v\}} (D[S \setminus \{v\}, u] + c(u, v)).$$

**Why the recurrence is exact.** The penultimate vertex  $u$  partitions all feasible paths. Removing their last edge leaves precisely a feasible path for the smaller state; conversely, appending  $(u, v)$  to any such path gives a feasible path for  $(S, v)$ .

Evaluate states in increasing  $|S|$ . For  $n \geq 2$ , close the path by returning to  $r$ :

$$\text{OPT} = \min_{v \neq r} (D[V, v] + c(v, r)).$$

Store a minimizing predecessor with each state; choose the minimizing final vertex, backtrack while removing the last vertex from  $S$ , reverse the resulting sequence, and append  $r$ . There are  $O(n2^n)$  states with  $O(n)$  transitions each, giving  $O(n^2 2^n)$  time and  $O(n2^n)$  space, including predecessors. This is exponential, although faster than enumerating tours.

## 4 Randomization and Derandomization

### 4.1 The indicator-variable proof template

For a maximization objective with nonnegative local rewards, write

$$\text{ALG} = \sum_i w_i I_i, \quad \mathbb{E}[\text{ALG}] = \sum_i w_i \Pr[I_i = 1].$$

Linearity of expectation does *not* require the indicators to be independent. Independence is needed only when a probability calculation multiplies probabilities of separate random choices. If every reward is collected with probability at least  $\alpha$  and  $W = \sum_i w_i \geq \text{OPT}$ , then  $\mathbb{E}[\text{ALG}] \geq \alpha \text{OPT}$ . This guarantees the existence of a good outcome, not that every run is good. An expected approximation bound alone is not a high-probability bound.

### 4.2 Weighted MAX-CUT, MAX- $k$ -CUT, and MAX-DICUT

Assume nonnegative edge weights and remove self-loops, which can never cross a cut. For undirected MAX- $k$ -CUT, independently label every vertex uniformly from  $\{1, \dots, k\}$ , for  $k \geq 2$ . Count edges whose endpoint labels differ. For every edge  $e = \{u, v\}$ ,

$$\Pr[I_e = 1] = 1 - \sum_{a=1}^k \Pr[u = a, v = a] = 1 - \frac{1}{k}.$$

Consequently

$$\mathbb{E}[\text{ALG}] = (1 - 1/k)W \geq (1 - 1/k) \text{OPT}.$$

The ordinary MAX-CUT case is  $k = 2$ , giving  $\text{OPT}/2$ ; MAX-3-CUT gives  $2\text{OPT}/3$ . A run takes  $O(n + m)$  time including evaluation. For directed MAX-DICUT, select each vertex into  $S$  independently with probability  $1/2$ , and count only arcs from  $S$  to  $V \setminus S$ . Each arc  $(u, v)$  is counted with probability  $1/4$ , so  $\mathbb{E}[\text{ALG}] = W/4 \geq \text{OPT}/4$ . Do not count arcs in both directions: doing so changes the objective. LP rounding improves the retained fraction to  $1/2$  (Section 5.6).

### 4.3 Conditional expectation: how to remove randomness

Let  $F$  be the expected final objective conditional on the decisions already fixed. If the next random variable has outcomes  $a$  with probabilities  $p_a$ , then

$$F = \sum_a p_a F_a, \quad F_a = \mathbb{E}[\text{ALG} \mid \text{fixed decisions}, X = a].$$

At least one positive-probability branch has  $F_a \geq F$ . Fix such a branch and repeat. The invariant is that the conditional expectation never decreases; after the last decision it equals the actual objective. Thus the deterministic output is at least the initial expectation. To claim a polynomial-time algorithm, show how to compute each  $F_a$  in polynomial time.

#### MAX- $k$ -CUT: computable conditional contributions

Under a partial labelling, an edge with both endpoints fixed contributes its full weight if their labels differ and zero otherwise. An edge with at least one endpoint unfixed contributes  $(1 - 1/k)w_e$ . When assigning a vertex  $v$ , only edges to already fixed neighbors have label-dependent contributions. Choose a label  $a$  maximizing

$$\sum_{u: \{u, v\} \in E, u \text{ fixed}, \text{label}(u) \neq a} w_{uv},$$

equivalently minimizing the weight of fixed neighbors with label  $a$ . Process vertices in any order. If  $k \geq n$ , assign distinct labels to all vertices instead; this cuts every edge and is optimal. For  $k < n$ , the conditional-expectation procedure gives a deterministic  $(1 - 1/k)$  retained-value approximation in  $O(m + nk)$  time by accumulating neighbor weights in  $k$  buckets at each vertex.

**Worked MAX-CUT example.** For a unit-weight triangle the initial expectation is  $3/2$ . Fix the first vertex to side 0; the expectation remains  $3/2$ . Putting the second on side 1 makes their edge contribute 1 and the two unresolved edges contribute  $1/2$  each, giving conditional expectation 2. Either choice for the third vertex then yields cut value 2.

#### MAX-SAT: computable conditional contributions

First remove duplicate literals, discard empty clauses (always false), and separate tautologies (always true). Thus every remaining clause contains distinct

variables and no variable together with its negation. Suppose an unfixed variable  $x_j$  is independently true with probability  $p_j$ . For each clause  $C_i$ , its conditional expected contribution is

$$\begin{cases} w_i, & \text{a fixed literal is already true,} \\ 0, & \text{all literals are fixed and false,} \\ w_i \left(1 - \prod_{\ell \text{ unfixed in } C_i} q_\ell\right), & \text{otherwise,} \end{cases}$$

where  $q_{x_j} = 1 - p_j$  and  $q_{\neg x_j} = p_j$  are literal-failure probabilities. For fair variables and  $r$  unfixed literals, the last line is  $w_i(1 - 2^{-r})$ . Evaluate the total for setting the next variable to 0 and to 1, and keep the larger value. Scanning all  $L$  literal occurrences for each of  $n$  variables gives  $O(nL)$  arithmetic operations. This works for fair, biased, and LP-derived independent probabilities.

**Worked example.** For unit-weight clauses  $(x)$  and  $(\neg x \vee y)$  with fair variables,  $F = 1/2 + 3/4 = 5/4$ . Setting  $x = 1$  gives  $F_1 = 1 + 1/2 = 3/2$ ; setting  $x = 0$  gives  $F_0 = 0 + 1 = 1$ . Choose  $x = 1$ , then  $y = 1$ , obtaining value 2. For a directed cut with arbitrary independent membership probabilities, an arc's conditional contribution is  $w_{uv}p'_u(1 - p'_v)$ , where  $p'_v$  is 0 or 1 for a fixed vertex and its original probability otherwise. This also makes both MAX-DICUT rounding schemes below derandomizable.

### 4.4 Weighted MAX-SAT by fair and biased assignment

Use the clause preprocessing above; handle tautological clauses as a constant reward added to every solution. All weights are nonnegative. In this subsection and Section 5.5,  $W$ ,  $\text{OPT}$ , and  $LP^*$  refer to the remaining formula after removing constant clauses. If the removed true clauses have total weight  $T \geq 0$ , restore it at the end:  $\text{ALG}_{\text{total}} = T + \text{ALG}$  and  $\text{OPT}_{\text{total}} = T + \text{OPT}$ . Every retained-fraction guarantee  $\text{ALG} \geq \alpha \text{OPT}$  with  $0 \leq \alpha \leq 1$  (also in expectation) is preserved because  $T + \alpha \text{OPT} \geq \alpha(T + \text{OPT})$ . For a remaining clause  $C_i$  of length  $\ell_i \geq 1$ , fair independent assignment gives

$$\begin{aligned} \Pr[C_i \text{ satisfied}] &= 1 - 2^{-\ell_i}, \\ \mathbb{E}[\text{ALG}] &= \sum_i w_i(1 - 2^{-\ell_i}) \geq W/2 \geq \text{OPT}/2. \end{aligned}$$

The satisfaction probability *increases* with clause length. If every clause has at least  $k$  literals, the guarantee strengthens to  $(1 - 2^{-k}) \text{OPT}$ . An assignment and its value can be computed in  $O(n + L)$  time, where  $L$  is the number of literal occurrences.

#### Biased assignment and the unit-clause issue

Merge repeated unit clauses by adding their weights. For each variable, rename it consistently throughout the formula if needed so that the total positive unit-clause weight is at least the negative one. Let  $N$  be the sum of the negative unit-clause weights after this renaming. Every assignment loses at least the smaller weight in each opposing pair, so  $\text{OPT} \leq W - N$ . Set every renamed variable independently true with probability  $p \in [1/2, 1]$ . A clause with  $a$  positive and  $b$  negative literals is satisfied with probability

$$1 - (1 - p)^a p^b \geq 1 - p^{a+b}.$$

In a lower bound, ignore the negative unit clauses. Every remaining unit clause is satisfied with probability  $p$ , and every nonunit clause with probability at least  $1 - p^2$ . Therefore

$$\mathbb{E}[\text{ALG}] \geq \min\{p, 1 - p^2\}(W - N) \geq \min\{p, 1 - p^2\} \text{OPT}.$$

The maximum occurs when  $p = 1 - p^2$ , giving

$$p = \frac{\sqrt{5} - 1}{2}, \quad \mathbb{E}[\text{ALG}] \geq \frac{\sqrt{5} - 1}{2} \text{OPT}.$$

If no negative unit clauses remain, this simplifies to  $N = 0$ . Without the renaming/upper-bound argument, assuming all unit clauses are positive is not justified. Conditional expectation gives deterministic versions of these guarantees.

## 5 Linear Programming and Rounding

### 5.1 Modelling, relaxation, and the direction of the bound

An ILP uses integer variables; a 0–1 formulation uses binary decision variables. A relaxation replaces  $x \in \{0, 1\}$  by  $0 \leq x \leq 1$  and keeps the linear constraints. Its feasible set contains all integral feasible points. Write  $LP^*$  for the *optimal* relaxed objective:

|              | relaxation bound       | target rounding proof                                            |
|--------------|------------------------|------------------------------------------------------------------|
| minimization | $LP^* \leq \text{OPT}$ | $\text{ALG} \leq \rho LP^* \leq \rho \text{OPT}$                 |
| maximization | $LP^* \geq \text{OPT}$ | $\mathbb{E}[\text{ALG}] \geq \alpha LP^* \geq \alpha \text{OPT}$ |

An arbitrary feasible fractional objective is not necessarily a valid bound on OPT in the displayed direction. LP is solvable in polynomial time in its encoding length; general ILP is NP-hard. After rounding, prove *feasibility* separately from the objective bound.

**Proving a model is exact.** Show both (i) each original solution gives a feasible integral point of the same objective and (ii) an integral point decodes into a solution of at least its objective for maximization (at most for minimization). These establish equality of optimal values. An auxiliary reward variable often needs only upper bounds: maximizing a nonnegative weighted objective pushes it up when possible. Such constraints need not force its intended Boolean meaning at *every feasible point*. If its coefficient is zero, it need not be pushed up even at an optimum; the decoded solution and optimal value remain correct. Add equivalence constraints if the variable's exact meaning is required independently of the objective.

## 5.2 FOL feasibility and ILP formulation reference

Each entry fixes a problem variant, gives logical feasibility conditions, and then a linear objective and integer constraints. Costs, profits, and capacities are nonnegative unless stated otherwise. All graphs are finite; undirected graphs are simple. A binary variable is the indicator of its corresponding selection predicate. Quantifiers range over the indicated input sets. The objective is separate from the FOL feasibility formula. Separate displayed conditions are conjoined. Numerical comparisons use the input's fixed arithmetic structure; scheduling times are integer-valued.

| Formulation                       | Page |
|-----------------------------------|------|
| Metric TSP                        | 8    |
| Single-machine scheduling         | 8    |
| Parallel-machine scheduling       | 9    |
| Bin packing                       | 9    |
| Set packing                       | 9    |
| Prize-collecting Steiner tree     | 9    |
| Balanced cut                      | 9    |
| $k$ -minimum spanning tree        | 9    |
| Capacitated minimum spanning tree | 9    |
| Feedback vertex set               | 9    |
| Shortest superstring              | 10   |
| 3-dimensional matching            | 10   |
| Fair matching with group quotas   | 10   |
| Capacitated facility location     | 10   |
| Maximum independent set           | 10   |
| Capacitated knapsack              | 10   |
| Higher-dimensional knapsack       | 10   |

### How to read the logical notation

**Candidate interpretations.** Predicates such as  $P(i)$  (select item) and  $A(i, j)$  (assign  $i$  to  $j$ ) describe a candidate solution. The displayed FOL formula tests that interpretation. Searching over interpretations is an additional optimization step; existentially quantifying a predicate would be *second-order*, not ordinary FOL.

**Counting.**  $\exists^k z \in D : P(z)$  abbreviates “exactly  $k$ ”, using  $k$  distinct witnesses satisfying  $P$  and asserting that every  $P$ -element equals one of them. For  $k = 0$ , it means  $\forall z \in D : \neg P(z)$ . Likewise,  $\exists \leq k$  forbids  $k+1$  distinct witnesses, and  $\exists \geq k$  requires  $k$  distinct witnesses. All these are finite FOL expansions for a fixed input integer  $k$ .

**Capacity.** For nonnegative weights  $w_i$  and bound  $B$ , write

$$\text{Cap}_{D,w,B}(P) := \bigwedge_{A \subseteq D} \left( \bigvee_{i \in A} \neg P(i) \right) \quad \text{if } \sum_{i \in A} w_i > B$$

It forbids selecting every member of any overweight set, and is equivalent to  $\sum_{i \in D} w_i \mathbf{1}_{P(i)} \leq B$ . In  $\text{Cap}(A(\cdot, j))$ , the dot is the quantified object and  $j$  is fixed. The conjunction over subsets is a *finite input-indexed schema*, not a first-order variable ranging over arbitrary sets. Expanding it can be exponential even when the resulting ILP has just one capacity inequality.

**Connectivity and trees.** Let  $Z(v)$  select vertices and  $X(e)$  select edges. Write  $E(S)$  for edges internal to  $S$ ,  $\delta(S)$  for edges with exactly one endpoint in  $S$ , and  $\delta(v) = \delta(\{v\})$ . Require selected edges to have selected endpoints:

$$\text{Ends}(Z, X) := \bigwedge_{uv \in E} (X(uv) \Rightarrow Z(u) \wedge Z(v)).$$

For each fixed  $\emptyset \neq S \subsetneq V$ , connectivity requires

$$\begin{aligned} & (\exists u \in S : Z(u)) \wedge (\exists v \notin S : Z(v)) \\ & \Rightarrow \bigvee_{e \in \delta(S)} X(e). \end{aligned}$$

Call the conjunction of these formulas  $\text{Conn}(Z, X)$ . For the input graph's simple cycles  $\mathcal{C}(G)$ , define

$$\text{Acyc}(X) := \bigwedge_{C \in \mathcal{C}(G)} \bigvee_{e \in E(C)} \neg X(e).$$

Then  $\text{Tree}(Z, X)$  means Ends, Conn, Acyc, and  $\exists v : Z(v)$ . Here too, the cut/cycle lists are fixed finite schemas; these abbreviations do not claim that general reachability has a uniform ordinary-FOL definition.

**Reusable exact tree ILP.** With  $x_e, y_v \in \{0, 1\}$ , let  $\mathcal{T}(x, y)$  denote

$$\begin{aligned} & x_{uv} \leq y_u, \quad x_{uv} \leq y_v & (uv \in E), \\ & \sum_{e \in E(S)} x_e \leq |S| - 1 & (S \subseteq V, |S| \geq 2), \\ & \sum_{e \in E} x_e = \sum_{v \in V} y_v - 1, \quad \sum_{v \in V} y_v \geq 1. \end{aligned}$$

The subset inequalities make the selected graph a forest; a forest on  $q$  selected vertices has  $q - c$  edges for  $c$  components. The edge count therefore forces  $c = 1$ . This is exact, including singleton trees, but has exponentially many subset constraints. Merely imposing  $q - 1$  edges is insufficient: a cycle plus an isolated selected vertex can satisfy that count.

### 5.2.1 Metric TSP

**Variants.** A complete undirected graph on  $n \geq 3$  cities, with symmetric costs satisfying  $c_{uw} \leq c_{uv} + c_{vw}$ . Select a minimum-cost Hamiltonian cycle. Let  $X(uv)$  mean that the tour uses edge  $uv$ .

**FOL.** With  $Z(v)$  always true,

$$(\forall v : \exists^2 u \neq v : X(uv)) \wedge \text{Conn}(Z, X).$$

**ILP.** Binary  $x_e$ ; minimize  $\sum_e c_e x_e$ , subject to

$$\begin{aligned} & \sum_{e \in \delta(v)} x_e = 2 \quad (v \in V), \\ & \sum_{e \in \delta(S)} x_e \geq 2 \quad (\emptyset \neq S \subsetneq V). \end{aligned}$$

Degree 2 gives a collection of cycles; the cut constraints force a single tour. Triangle inequality supports the approximation algorithms, while the same integer formulation also models nonmetric TSP.

### 5.2.2 Single-machine scheduling

**Variants.** Jobs  $J$ , positive integer processing times  $p_i$ , weights  $w_i \geq 0$ , release time 0, no preemption or setup times. Minimize total weighted completion time  $\sum_i w_i C_i$ . Put  $H = \sum_i p_i$ .

**FOL.** A candidate gives integer start times  $S_i$  in the finite time domain  $\{0, \dots, H\}$ . Using the fixed arithmetic relations,

$$\begin{aligned} & \forall i : \quad 0 \leq S_i, \quad S_i + p_i \leq H, \\ & \forall i \neq j : \quad (S_i + p_i \leq S_j) \vee (S_j + p_j \leq S_i). \end{aligned}$$

**ILP.** For each  $i < j$ , binary  $z_{ij} = 1$  means  $i$  precedes  $j$ . Minimize  $\sum_i w_i (S_i + p_i)$ , subject to

$$\begin{aligned} & S_i + p_i \leq S_j + H(1 - z_{ij}) \quad (i < j), \\ & S_j + p_j \leq S_i + H z_{ij} \quad (i < j), \\ & 0 \leq S_i \leq H - p_i, \quad S_i \in \mathbb{Z} \quad (i \in J). \end{aligned}$$

The bound  $H$  makes the unused ordering inequality redundant. Positive processing times exclude cyclic precedences. An optimal schedule can have no idle time, so this horizon and integer start times lose no optimal solution. A pure makespan objective on this one-machine, all-jobs-available variant would always have optimum  $H$ .

### 5.2.3 Parallel-machine scheduling

**Variante.**  $m$  identical machines, all jobs available at time 0, positive integer  $p_i$ , no preemption, precedences, or setup times. Minimize makespan  $T$  and put  $H = \sum_i p_i$ . Let  $A(i, h)$  assign job  $i$  to machine  $h$ .

**FOL.** With integer  $0 \leq S_i$  and  $S_i + p_i \leq T \leq H$ ,

$$\begin{aligned} \forall i : \exists! h : A(i, h), \\ \forall i \neq j, h : A(i, h) \wedge A(j, h) \\ \Rightarrow (S_i + p_i \leq S_j) \vee (S_j + p_j \leq S_i). \end{aligned}$$

**ILP.** Binary  $x_{ih}$  and integer  $0 \leq T \leq H$ :

$$\begin{aligned} \min \quad & T, \\ \sum_{h=1}^m x_{ih} &= 1 \quad (i \in J), \\ \sum_{i \in J} p_i x_{ih} &\leq T \quad (h = 1, \dots, m). \end{aligned}$$

Start-time variables are unnecessary in this ILP: run each machine's assigned jobs consecutively in any order. Its completion time equals its load. For unrelated machines replace  $p_i$  by  $p_{ih}$  and choose a valid horizon; release times or precedences require additional scheduling constraints.

### 5.2.4 Bin packing

**Variante.** Pack  $n$  items of sizes  $0 \leq a_i \leq B$  into as few capacity- $B$  bins as possible; at most  $n$  bins are needed.  $A(i, b)$  means assignment and  $U(b)$  means bin  $b$  is used.

**FOL.**

$$\begin{aligned} \forall i : \exists! b : A(i, b), \\ \forall i, b : A(i, b) \Rightarrow U(b), \\ \forall b : \text{Cap}_{J, a, B}(A(\cdot, b)). \end{aligned}$$

**ILP.** Binary  $x_{ib}, y_b$ ; minimize  $\sum_{b=1}^n y_b$ :

$$\begin{aligned} \sum_{b=1}^n x_{ib} &= 1 \quad (i \in J), \\ x_{ib} &\leq y_b \quad (i \in J, b = 1, \dots, n), \\ \sum_i a_i x_{ib} &\leq B y_b \quad (b = 1, \dots, n). \end{aligned}$$

The linking constraint also handles zero-size items correctly. Optional symmetry breaking  $y_b \geq y_{b+1}$  orders the used bins without changing OPT.

### 5.2.5 Set packing

**Variante.** Given subsets  $S_j \subseteq U$  and profits  $w_j$ , select a pairwise disjoint subfamily of maximum total profit. Use  $P(j)$  for selection.

**FOL.**

$$\forall j \neq k : (P(j) \wedge P(k)) \Rightarrow S_j \cap S_k = \emptyset.$$

Here disjointness is a fixed input relation, equivalently  $\neg \exists u : (u \in S_j \wedge u \in S_k)$ .

**ILP.** Binary  $x_j$ :

$$\max \sum_j w_j x_j, \quad \sum_{j: u \in S_j} x_j \leq 1 \quad (u \in U).$$

Every element can belong to at most one chosen set. Taking  $w_j = 1$  gives maximum-cardinality set packing.

### 5.2.6 Prize-collecting Steiner tree

**Variante.** Given an undirected graph, edge costs  $c_e \geq 0$ , and penalties  $\pi_v \geq 0$ , select a nonempty tree; pay  $\pi_v$  for each omitted vertex. Nonterminals can have penalty 0. Use  $Z(v)$  and  $X(e)$  for selected vertices and edges.

**FOL.**  $\text{Tree}(Z, X)$ ; for a rooted variant also require  $Z(r)$ . Mandatory terminals satisfy  $Z(v)$  regardless of their penalty.

**ILP.** Binary  $x_e, y_v$ , with the full tree system  $\mathcal{T}(x, y)$  from page 8:

$$\min \sum_{e \in E} c_e x_e + \sum_{v \in V} \pi_v (1 - y_v).$$

Add  $y_r = 1$  only when a root is prescribed. Internal Steiner vertices count as selected vertices even when their penalty is zero. Connectivity is essential: otherwise isolated selected vertices avoid penalties for free. If the variant also requires collected prize at least  $Q$ , with prizes  $p_v$ , add  $\sum_v p_v y_v \geq Q$ ; prizes and omission penalties need not agree.

### 5.2.7 Balanced cut

**Variante.** Minimum-weight bisection of an undirected graph: choose one side of size  $q = \lfloor |V|/2 \rfloor$ . Let  $P(v)$  select that side and  $D(uv)$  indicate a crossing edge.

**FOL.**

$$\begin{aligned} \exists^{=q} v : P(v), \\ \forall uv \in E : D(uv) \Leftrightarrow (P(u) \oplus P(v)). \end{aligned}$$

**ILP.** Binary  $x_v, d_{uv}$ ; minimize  $\sum_{uv \in E} c_{uv} d_{uv}$ :

$$\begin{aligned} \sum_v x_v &= q, \\ d_{uv} &\geq x_u - x_v, \quad d_{uv} \geq x_v - x_u, \\ d_{uv} &\leq x_u + x_v, \quad d_{uv} \leq 2 - x_u - x_v \quad (uv \in E). \end{aligned}$$

The four inequalities enforce XOR at every feasible binary point. For a balance interval  $L \leq |S| \leq U$ , replace the equality by  $L \leq \sum_v x_v \leq U$  and use the corresponding counting bounds in FOL.

### 5.2.8 $k$ -minimum spanning tree

**Variante.** Minimum-cost tree spanning exactly  $k$  selected vertices of  $G$ , where  $1 \leq k \leq |V|$ . Vertices not in the tree are omitted.

**FOL.**

$$\text{Tree}(Z, X) \wedge (\exists^{=k} v : Z(v)).$$

**ILP.** Binary  $x_e, y_v$ :

$$\min \sum_e c_e x_e, \quad \mathcal{T}(x, y), \quad \sum_v y_v = k.$$

In particular  $\sum_e x_e = k - 1$ . A rooted variant adds  $y_r = 1$ . Check the convention: “ $k$  nonroot vertices” means  $k + 1$  total vertices. For nonnegative costs, an at-least- $k$  tree can be pruned to  $k$  vertices without increasing cost (preserving a required root), so the optimum values agree under those assumptions.

### 5.2.9 Capacitated minimum spanning tree

**Variante.** Root  $r$ , positive integer demands  $q_v$  for  $v \neq r$ ,  $q_r = 0$ , and branch capacity  $Q$ . Every component after deleting  $r$  from the spanning tree must have total demand at most  $Q$ .

**FOL.** With  $Z(v)$  always true, require  $\text{Tree}(Z, X)$  and

$$\forall b \in N(r) : X(rb) \Rightarrow \text{Cap}_{V \setminus \{r\}, q, Q}(\text{Branch}_b).$$

Here  $\text{Branch}_b(v)$  means a selected-edge path from  $b$  to  $v$  avoiding  $r$ ; expand it as the finite disjunction of path lengths  $0, \dots, |V| - 1$ , with first-order quantified intermediate vertices.

**ILP.** Replace edges by both arcs but omit arcs entering  $r$ . Binary  $x_{uv}$  selects an arc directed away from  $r$ ; integer  $f_{uv} \geq 0$  carries demand. Put  $D = \sum_{v \neq r} q_v$ . Minimize  $\sum_{uv} c_{uv} x_{uv}$ , subject to

$$\begin{aligned} \sum_{u:uv} x_{uv} &= 1 \quad (v \neq r), \\ \sum_{u:uv} f_{uv} - \sum_{w:vw} f_{vw} &= q_v \quad (v \neq r), \\ \sum_{w:rw} f_{rw} &= D, \\ 0 &\leq f_{uv} \leq Q x_{uv} \quad (uv \text{ an arc}). \end{aligned}$$

Positive demands exclude disconnected directed cycles; one parent per non-root vertex then gives a rooted tree. Each root arc carries exactly its branch demand. If zero demands are allowed, add connectivity/acyclicity constraints explicitly. Unit demands give the branch-size version.

### 5.2.10 Feedback vertex set

**Variante.** Delete a minimum-cost vertex set so the remaining undirected graph is acyclic. Let  $D(v)$  indicate deletion and  $w_v$  its cost.

**FOL.**

$$\bigwedge_{C \in \mathcal{C}(G)} \bigvee_{v \in V(C)} D(v).$$

Every input cycle must contain a deleted vertex.

**ILP.** Binary  $d_v$ :

$$\min \sum_v w_v d_v, \quad \sum_{v \in V(C)} d_v \geq 1 \quad (C \in \mathcal{C}(G)).$$

These cycle inequalities can be exponential in number. For the directed variant use directed cycles. Feedback vertex set differs from vertex cover: a tree has feedback vertex set size 0, even though its edges may need a nonempty vertex cover.

### 5.2.11 Shortest superstring

**Variante.** Given nonempty strings  $s_i$  over alphabet  $\Sigma$ , find a shortest word containing each as a contiguous substring. Let  $\ell_i = |s_i|$  and  $H = \sum_i \ell_i$ ; concatenation proves this length bound.

**FOL.** A candidate has length  $L \in \{0, \dots, H\}$  and a character function  $a$  on its positions. Require

$$\forall i \exists t : \quad [1 \leq t \wedge t + \ell_i - 1 \leq L \\ \wedge (\forall k \in \{1, \dots, \ell_i\} : a(t + k - 1) = s_i[k])].$$

Minimize  $L$ . The position and alphabet domains are finite; arithmetic and input characters are fixed relations/functions.

**ILP (direct position model).** Binary  $u_p$  activates position  $p$ ;  $z_{p,a}$  puts character  $a$  there;  $o_{it}$  chooses an occurrence of  $s_i$  starting at  $t$ . For  $p = 1, \dots, H$ , minimize  $\sum_p u_p$  with

$$\sum_{a \in \Sigma} z_{p,a} = u_p \quad (p = 1, \dots, H), \\ u_p \geq u_{p+1} \quad (p < H), \\ \sum_{t=1}^{H-\ell_i+1} o_{it} \geq 1 \quad (\text{each } i), \\ o_{it} \leq z_{t+k-1, s_i[k]} \quad (\text{each valid } i, t, k).$$

Here  $1 \leq t \leq H - \ell_i + 1$  and  $1 \leq k \leq \ell_i$  in the last line. The active positions form one prefix; exactly one character occupies each. Every chosen occurrence must match all its characters. Conversely, every superstring of length at most  $H$  gives these variables. Contained strings and duplicate strings are allowed by this model.

### 5.2.12 3-dimensional matching

**Variante.** Disjoint coordinate sets  $A, B, C$  and allowed triples  $\mathcal{R} \subseteq A \times B \times C$ . Choose a maximum-weight collection sharing no coordinate;  $P(t)$  selects triple  $t$ .

**FOL.**

$$\forall u \in A \cup B \cup C : \quad \exists \leq 1 t \in \mathcal{R} : (P(t) \wedge u \in t).$$

Membership  $u \in t$  means that  $u$  is one of the triple's coordinates.

**ILP.** Binary  $x_t$ :

$$\max \sum_{t \in \mathcal{R}} w_t x_t, \quad \sum_{t \in \mathcal{R} : u \in t} x_t \leq 1 \quad (u \in A \cup B \cup C).$$

Set  $w_t = 1$  for maximum cardinality. In the *perfect* version,  $|A| = |B| = |C|$  and every coordinate must be used: change  $\leq 1$  to  $= 1$  in both the counting formula and ILP and solve feasibility.

### 5.2.13 Fair matching with group quotas

**Variante.** Bipartite  $G = (L \cup R, E)$ , matching profits  $w_e$ , and fixed groups  $E_g \subseteq E$  with integer quotas  $0 \leq \ell_g \leq u_g$ . For a group of left vertices  $L_g$ , use  $E_g = \{uv \in E : u \in L_g\}$ ; then the group count equals the number of its matched vertices.

**FOL.** With  $M(e)$  denoting selection,

$$\forall v : \exists \leq 1 e \in \delta(v) : M(e), \\ \forall g : \quad (\exists \geq \ell_g e \in E_g : M(e)) \wedge (\exists \leq u_g e \in E_g : M(e)).$$

**ILP.** Binary  $x_e$ :

$$\max \sum_{e \in E} w_e x_e, \\ \sum_{e \in \delta(v)} x_e \leq 1 \quad (v \in L \cup R), \\ \ell_g \leq \sum_{e \in E_g} x_e \leq u_g \quad (\text{each group } g).$$

Groups may overlap; quotas may make the model infeasible. Set  $w_e = 1$  for maximum cardinality. Proportional quotas can instead use  $\alpha_g \sum_e x_e \leq \sum_{e \in E_g} x_e \leq \beta_g \sum_e x_e$ , which is still linear for fixed rational  $\alpha_g, \beta_g$ . Extra quotas do not automatically preserve the ordinary bipartite matching LP's integrality.

### 5.2.14 Capacitated facility location

**Variante.** Clients  $I$  have demands  $d_i > 0$ ; facilities  $J$  have capacities  $Q_j$  and opening costs  $f_j$ . Each client uses exactly one open facility. Let  $c_{ij}$  be the *total* cost of serving client  $i$  at  $j$ .

**FOL.** Assignment  $A(i, j)$  and opening  $O(j)$  satisfy

$$\forall i : \exists ! j : A(i, j), \\ \forall i, j : \quad A(i, j) \Rightarrow O(j), \\ \forall j : \quad \text{Cap}_{I, d, Q_j}(A(\cdot, j)).$$

**ILP (unsplittable demand).** Binary  $x_{ij}, y_j$ :

$$\min \sum_j f_j y_j + \sum_{i, j} c_{ij} x_{ij}, \\ \sum_j x_{ij} = 1 \quad (i \in I), \\ x_{ij} \leq y_j \quad (i \in I, j \in J), \\ \sum_i d_i x_{ij} \leq Q_j y_j \quad (j \in J).$$

If shipping costs are *per unit*, use  $d_i c_{ij}$  in the objective. For splittable demand, use shipment variables  $a_{ij} \geq 0$  with  $\sum_j a_{ij} = d_i$ ,  $\sum_i a_{ij} \leq Q_j y_j$ , and  $a_{ij} \leq d_i y_j$ ; continuous shipments give a MILP rather than a pure ILP.

### 5.2.15 Maximum independent set

**Variante.** Select a maximum-weight set of pairwise nonadjacent vertices; use weights 1 for maximum cardinality.

**FOL.**

$$\forall u, v : \quad uv \in E \Rightarrow \neg(P(u) \wedge P(v)).$$

**ILP.** Binary  $x_v$ :

$$\max \sum_v w_v x_v, \quad x_u + x_v \leq 1 \quad (uv \in E).$$

The complement is a vertex cover, but approximation factors do not transfer simply by complementing an approximate solution.

### 5.2.16 Capacitated knapsack

**Variante.** One knapsack of capacity  $B$ , indivisible items with sizes  $a_i \geq 0$  and profits  $v_i \geq 0$ , each available once.

**FOL.**  $\text{Cap}_{J, a, B}(P)$  for the selected-item predicate  $P(i)$ . Maximize the sum of selected profits.

**ILP.**

$$\max \sum_i v_i x_i, \quad \sum_i a_i x_i \leq B, \quad x_i \in \{0, 1\}.$$

For bounded multiplicities replace the domain by  $x_i \in \{0, 1, \dots, b_i\}$ ; the FOL version can select individual copies. With unlimited copies use  $x_i \in \mathbb{Z}_{\geq 0}$  and check for zero-size, positive-profit items, which make the objective unbounded.

### 5.2.17 Higher-dimensional knapsack

**Variante.** Each selected item simultaneously consumes  $a_{ri} \geq 0$  units of each resource  $r = 1, \dots, d$ , with capacity  $B_r$ . Each item is available once; its profit is  $v_i$ .

**FOL.**

$$\bigwedge_{r=1}^d \text{Cap}_{J, a_r, B_r}(P).$$

**ILP.**

$$\max \sum_i v_i x_i, \quad \sum_i a_{ri} x_i \leq B_r \quad (r = 1, \dots, d), \quad x_i \in \{0, 1\}.$$

There is one capacity constraint per resource and one shared selection variable per item. This models multiple resource budgets, not geometric placement of rectangles or boxes. For several separate knapsacks with assignment choices, introduce  $x_{ib}$  and require  $\sum_b x_{ib} \leq 1$ .

**Model-checking reminders.** Keep domains, quantifier ranges, objective direction, and the precise variant with each formulation. Cut/cycle schemas may be exponential. Connectivity, acyclicity, and capacity are separate obligations; degree or edge-count constraints alone rarely establish all three. Dropping all integrality restrictions gives a relaxation, not automatically an exact polynomial-time algorithm for the original problem.

**Further formulation references.** The formulations above are written out for the stated variants. Useful primary references are the SCIP modelling book's [TSP formulations](#), [bin-packing model](#), and [facility-location models](#); the DIMACS prize-collecting Steiner definitions; the CMST branch-capacity definition; and [proportionally fair matching](#).

### 5.3 Weighted vertex cover: threshold rounding

For nonnegative vertex costs  $c_v$ , let  $x_v$  indicate selection. The ILP is

$$\begin{aligned} \min \quad & \sum_{v \in V} c_v x_v \\ \text{subject to} \quad & x_u + x_v \geq 1 \quad (\{u, v\} \in E), \\ & x_v \in \{0, 1\}. \end{aligned}$$

Relax to  $[0, 1]$ , obtain an optimum  $x^*$ , and return  $C = \{v : x_v^* \geq 1/2\}$ . At least one endpoint of every edge reaches  $1/2$ , so  $C$  is feasible. Also

$$c(C) = \sum_{v \in C} c_v \leq \sum_{v \in C} 2c_v x_v^* \leq 2LP^* \leq 2\text{OPT}.$$

The algorithm is polynomial time (LP plus a linear scan). On the unweighted complete graph  $K_n$ ,  $\text{OPT} = n - 1$  and  $LP^* = n/2$ : the all- $1/2$  point is feasible, and summing all edge constraints gives  $(n - 1) \sum_v x_v \geq n(n - 1)/2$ . Hence the worst-case integrality gap is at least  $2 - 2/n$ , approaching 2. This is a family of gap examples, not a claim that every  $n$ -vertex graph has this gap.

### 5.4 Set cover: bounded frequency

Let  $U$  be a universe,  $S_1, \dots, S_m$  candidate sets with  $\bigcup_j S_j = U$ , and costs  $c_j \geq 0$ . One binary variable per set suffices:

$$\begin{aligned} \min \quad & \sum_{j=1}^m c_j x_j \\ \text{subject to} \quad & \sum_{j: e \in S_j} x_j \geq 1 \quad (e \in U), \\ & x_j \in \{0, 1\}. \end{aligned}$$

There is no need for an extra variable indicating coverage when every element must be covered. If each element belongs to at most  $k$  sets, relax and select every set with  $x_j^* \geq 1/k$ . Each covering constraint has at most  $k$  terms, so at least one reaches the threshold. The cost satisfies

$$\text{ALG} \leq k \sum_j c_j x_j^* = kLP^* \leq k\text{OPT}.$$

This generalizes vertex-cover rounding, where an edge has two endpoint choices. The hypothesis is bounded *frequency*, not bounded set size. For example, with one element and two identical cost-1 singleton sets,  $(1/2, 1/2)$  is LP-optimal. Each set has size 1, but threshold 1 selects nothing. This disproves the corresponding threshold proof for bounded set size.

### 5.5 Weighted MAX-SAT: LP rounding

For the preprocessed formula of Section 4.4, use  $x_j$  for a variable's truth value and  $y_i$  for the reward of clause  $C_i$ :

$$\begin{aligned} \max \quad & \sum_i w_i y_i \\ \text{subject to} \quad & y_i \leq \sum_{x_j \in C_i} x_j + \sum_{\neg x_j \in C_i} (1 - x_j) \quad \text{for every } i, \\ & x_j, y_i \in \{0, 1\}. \end{aligned}$$

A false clause forces  $y_i = 0$ ; a true clause permits  $y_i = 1$ . Nonnegative weights make the ILP optimal value exactly the maximum satisfied weight. Relax all variables to  $[0, 1]$  and write  $LP^* = \sum_i w_i y_i^* \geq \text{OPT}$ . Independently set  $x_j$  true with probability  $x_j^*$ .

#### One-clause bound: product, AM–GM, then concavity

For a clause of length  $\ell \geq 1$ , let  $a_1, \dots, a_\ell$  be its literal-success probabilities:  $x_j^*$  for a positive literal and  $1 - x_j^*$  for a negative one. Its variables are distinct, and  $\sum_h a_h \geq y_i^*$ . Hence

$$\begin{aligned} \Pr[C_i \text{ satisfied}] &= 1 - \prod_{h=1}^{\ell} (1 - a_h) \\ &\geq 1 - \left(1 - \frac{\sum_h a_h}{\ell}\right)^\ell \geq 1 - \left(1 - \frac{y_i^*}{\ell}\right)^\ell. \end{aligned}$$

The exponent is  $\ell$ , not  $1/\ell$ . Let  $g_\ell(y) = 1 - (1 - y/\ell)^\ell$ . For  $\ell = 1$  it is linear; for  $\ell \geq 2$ ,

$$g_\ell''(y) = -\frac{\ell-1}{\ell} (1 - y/\ell)^{\ell-2} \leq 0.$$

Since  $g_\ell(0) = 0$ , concavity on  $[0, 1]$  gives  $g_\ell(y) \geq yg_\ell(1)$ . Also  $(1 - 1/\ell)^\ell \leq e^{-1}$ . Thus, with  $b_\ell = 1 - (1 - 1/\ell)^\ell$ ,

$$\begin{aligned} \Pr[C_i \text{ satisfied}] &\geq b_\ell y_i^* \geq (1 - 1/e) y_i^*, \\ \mathbb{E}[\text{ALG}] &\geq (1 - 1/e) LP^* \geq (1 - 1/e) \text{OPT}. \end{aligned}$$

The LP can be solved in polynomial time; sampling and evaluating the result takes  $O(n + L)$  time. The conditional-product calculation in Section 4.3 derandomizes the algorithm.

#### Combine fair assignment and LP rounding: 3/4

For a clause of length  $\ell$ , fair assignment satisfies it with probability  $a_\ell = 1 - 2^{-\ell} \geq a_\ell y_i^*$ , since  $0 \leq y_i^* \leq 1$ . LP rounding gives probability at least  $b_\ell y_i^*$ . The useful comparison is clause by clause:

| $\ell$        | $a_\ell$   | $b_\ell$             | $a_\ell + b_\ell$ |
|---------------|------------|----------------------|-------------------|
| 1             | 1/2        | 1                    | 3/2               |
| 2             | 3/4        | 3/4                  | 3/2               |
| $\ell \geq 3$ | $\geq 7/8$ | $\geq 1 - 1/e > 5/8$ | $> 3/2$           |

Run both algorithms and return the better assignment. If their values are  $A, B$ , then pointwise  $\max\{A, B\} \geq (A + B)/2$ , so

$$\mathbb{E}[\max\{A, B\}] \geq \frac{1}{2} \sum_i w_i (a_{\ell_i} + b_{\ell_i}) y_i^* \geq \frac{3}{4} LP^* \geq \frac{3}{4} \text{OPT}.$$

The two outputs need not be independent. Averaging the global constants  $1/2$  and  $1 - 1/e$  would not prove  $3/4$ ; the complementary clause-length bounds are essential. Alternatively derandomize each algorithm first and take the better: the same  $3/4$  guarantee then holds deterministically.

### 5.6 MAX-DICUT: two $1/2$ rounding proofs

Let  $G = (V, A)$  be a directed graph with nonnegative arc weights and no self-loops. Put  $x_i = 1$  when vertex  $i$  is on the source side. For each arc, use a reward variable  $z_{ij}$ . The ILP and its relaxation are

$$\begin{aligned} \max \quad & \sum_{(i,j) \in A} w_{ij} z_{ij} \\ \text{subject to} \quad & z_{ij} \leq x_i, \quad z_{ij} \leq 1 - x_j, \\ & x_i, z_{ij} \in \{0, 1\} \quad (\text{replace by } [0, 1] \text{ for the LP}). \end{aligned}$$

The upper bounds allow a positive reward only when  $x_i = 1, x_j = 0$ . For any fixed integral cut, setting  $z_{ij} = 1$  on its outgoing arcs attains the cut weight, so the ILP is exact. If exact equality  $z_{ij} = x_i(1 - x_j)$  is required at every integral feasible point, add  $z_{ij} \geq x_i - x_j$ ; it is unnecessary for this nonnegative maximization objective. Below,  $x, z$  denote an optimal solution of the displayed relaxation and  $L = LP^* \geq \text{OPT}$ .

#### Shifted probabilities: a per-arc comparison

Independently select vertex  $i$  with probability  $p_i = 1/4 + x_i/2$ . For each arc,

$$\begin{aligned} \Pr[i \in S, j \notin S] &= (1/4 + x_i/2)(1/4 + (1 - x_j)/2) \\ &\geq (1/4 + z_{ij}/2)^2 \\ &= z_{ij}/2 + (2z_{ij} - 1)^2/16 \geq z_{ij}/2. \end{aligned}$$

Summing gives  $\mathbb{E}[\text{ALG}] \geq L/2 \geq \text{OPT}/2$ . This per-arc inequality holds for any feasible LP point, though optimality (or another appropriate lower bound on its value) is needed to compare with OPT.

#### Direct probabilities: a global comparison

Rounding instead with  $p_i = x_i$  also gives a  $1/2$  guarantee for an *optimal* solution of this LP. Let  $W = \sum_{(i,j) \in A} w_{ij}$ . If  $W = 0$ , any cut is optimal. Otherwise,

$$\Pr[i \in S, j \notin S] = x_i(1 - x_j) \geq z_{ij}^2.$$

Weighted Cauchy–Schwarz gives

$$L^2 = \left( \sum_{ij} \sqrt{w_{ij} z_{ij}} \sqrt{w_{ij}} \right)^2 \leq \left( \sum_{ij} w_{ij} z_{ij}^2 \right) W.$$

The feasible point  $x_i = 1/2, z_{ij} = 1/2$  has value  $W/2$ , so optimality implies  $L \geq W/2$ . Consequently

$$\mathbb{E}[\text{ALG}] \geq \sum_{ij} w_{ij} z_{ij}^2 \geq L^2/W \geq L/2 \geq \text{OPT}/2.$$

Direct rounding lacks the shifted scheme's per-arc bound  $z_{ij}/2$ , but its global analysis recovers the same factor. Both algorithms are polynomial time and can be derandomized using conditional expectation.

## Tuning an affine probability safely

Let  $p_i = a + bx_i$ , with  $a, b \geq 0$  and  $a + b \leq 1$ , so all probabilities and all factors below are nonnegative. Write  $c = 1 - a - b \geq 0$ . Then

$$p_i(1 - p_j) \geq (a + bz_{ij})(c + bz_{ij}) = ac + b(1 - b)z_{ij} + b^2z_{ij}^2.$$

For fixed  $b$ ,  $a + c = 1 - b$  and  $ac$  is largest when  $a = c = (1 - b)/2$ . For this symmetric choice,

$$\begin{aligned} (a + bz)^2 &= 4abz + (a - bz)^2 \geq 4abz, \\ \alpha &= 4ab = 2b(1 - b) \\ &= \frac{1}{2} - 2(b - \frac{1}{2})^2 \leq \frac{1}{2}. \end{aligned}$$

Maximize the retained fraction  $\alpha$ : choose  $b = 1/2, a = 1/4$ , obtaining  $1/2$ . This optimizes this particular square-based lower bound. It does not by itself prove that every other analysis of affine rounding is incapable of a stronger guarantee.

## 5.7 Randomized set cover: repeat until feasible

Use the set-cover LP in Section 5.4, without a frequency bound. Let  $n = |U| \geq 1$ ,  $m$  be the number of candidate sets, and  $r = \lceil \ln(2n) \rceil$ . (For  $U = \emptyset$ , return the empty cover.) Solve the LP once. A *trial* performs  $r$  independent rounds; in each round select each set  $S_j$  independently with probability  $x_j^*$ , then take the union of all selected sets. Check coverage. Discard a failed trial and start a fresh independent trial until a cover is found.

**Success probability.** For any element  $e$ , the probability it remains uncovered in one round is

$$\prod_{j:e \in S_j} (1 - x_j^*) \leq \exp\left(-\sum_{j:e \in S_j} x_j^*\right) \leq e^{-1}.$$

After  $r$  independent rounds its failure probability is at most  $e^{-r} \leq 1/(2n)$ . By the union bound, a trial succeeds with probability  $q \geq 1/2$ . Element-failure events need not be independent.

**Cost of a trial and cost of the returned trial.** A set is included in the union with probability  $1 - (1 - x_j^*)^r \leq rx_j^*$ . Thus its nonnegative total cost  $C$  satisfies  $\mathbb{E}[C] \leq rLP^*$ . However, successful trials can cost more than typical trials. If  $A$  is the success event, the returned trial has the distribution of one trial conditioned on  $A$ , and

$$\mathbb{E}[C_{\text{returned}}] = \mathbb{E}[C \mid A] = \frac{\mathbb{E}[C\mathbf{1}_A]}{q} \leq \frac{\mathbb{E}[C]}{q} \leq 2rLP^* \leq 2r \text{OPT}.$$

Fresh independent trials have a geometric stopping time  $T$  with  $\Pr[T > t] = (1 - q)^t$  and  $\mathbb{E}[T] = 1/q \leq 2$ . Hence the algorithm terminates almost surely, always returns a checked feasible cover, and takes expected polynomial time. One trial uses  $O(rm)$  random decisions and  $O(n + \sum_j |S_j|)$  time to check coverage after marking selected sets. The cost guarantee is in expectation; it is not a bound on every returned cover. The factor  $1/q$  cannot be dropped without an additional argument, because cost and success may be correlated.

## 6 Network Flow

### 6.1 Integer network flow and residual graph

Let  $G = (V, E)$  be a directed network with distinct source  $s$ , sink  $t$ , capacities  $c(u, v) \geq 0$  for  $(u, v) \in E$ ,  $n = |V|$ , and  $m = |E|$ . Write  $f(u, v)$  for flow on the original arc  $(u, v)$ . If parallel arcs occur, give each a distinct identity and apply this notation to that named arc. The original feasibility constraints are

$$0 \leq f(u, v) \leq c(u, v) \quad ((u, v) \in E),$$

$$f_{\text{in}}(v) = \sum_{(u,v) \in E} f(u, v) = \sum_{(v,z) \in E} f(v, z) = f_{\text{out}}(v) \quad (v \neq s, t).$$

The objective is the *net* source outflow  $\text{value}(f) = f_{\text{out}}(s) - f_{\text{in}}(s)$ . It equals net inflow at  $t$ . As in the original notes, when no arcs enter  $s$ ,  $\text{value}(f) = \sum_{(s,v) \in E} f(s, v)$ .

**Residual graph  $G_f$ .** For each original arc  $e = (u, v)$ , keep two *labelled* residual arcs:

| label and direction     | residual capacity   | augmentation by $\Delta$              |
|-------------------------|---------------------|---------------------------------------|
| $e^+ : u \rightarrow v$ | $c(u, v) - f(u, v)$ | $f(u, v) \leftarrow f(u, v) + \Delta$ |
| $e^- : v \rightarrow u$ | $f(u, v)$           | $f(u, v) \leftarrow f(u, v) - \Delta$ |

Only positive-capacity residual arcs are available. In particular, when  $(u, v)$  and  $(v, u)$  are both original arcs, an arc directed  $u \rightarrow v$  may mean increasing one original flow or decreasing the other. Its label, not membership of  $(u, v)$

in  $E$ , determines the update. Thus the old  $c_f(u, v)$  notation is unambiguous only when the residual arc's original arc and forward/reverse label are clear. For a simple residual  $s$ - $t$  path  $P$ , set  $\Delta = \min_{a \in P} c_f(a)$  and apply its labelled updates. Bounds  $0 \leq f(u, v) \leq c(u, v)$  remain valid; every internal path vertex has zero change in net outflow;  $\text{value}(f)$  increases by  $\Delta$ .

**Why reverse arcs are necessary.** Take unit-capacity arcs  $s \rightarrow a, s \rightarrow b, a \rightarrow b, a \rightarrow t, b \rightarrow t$ . After sending one unit along  $s \rightarrow a \rightarrow b \rightarrow t$ , no forward-only augmenting path remains, although the optimum is 2. The residual path  $s \rightarrow b \rightarrow a \rightarrow t$  uses the reverse of  $a \rightarrow b$ , cancels that unit, and produces the two direct unit routes  $s \rightarrow a \rightarrow t$  and  $s \rightarrow b \rightarrow t$ .

**Comparing two flows: use the residual network.** For feasible original flows  $f, g$ , set  $h(e^+) = \max\{g(u, v) - f(u, v), 0\}$  and  $h(e^-) = \max\{f(u, v) - g(u, v), 0\}$  for each original arc  $e = (u, v)$ . These values respect residual capacities; conservation follows by subtracting the two original conservation equations, and  $\text{value}(h) = \text{value}(g) - \text{value}(f)$ . Conversely, a residual flow  $h$  updates the original flow by  $f'(u, v) = f(u, v) + h(e^+) - h(e^-)$ , with  $0 \leq f'(u, v) \leq c(u, v)$  and  $\text{value}(f') = \text{value}(f) + \text{value}(h)$ . Thus residual flow models possible improvement. The ordinary vector  $g - f$  need not be a nonnegative flow in  $G$ : an arc may carry less under  $g$  than under  $f$ . Its negative component belongs on the corresponding reverse residual arc.

### 6.2 $s$ - $t$ cuts and max-flow/min-cut

An  $s$ - $t$  cut is a partition  $(A, \bar{A})$  where  $\bar{A} = V \setminus A$ ,  $s \in A$ , and  $t \in \bar{A}$ . More generally,  $\bar{X} = V \setminus X$  for any vertex set  $X$ . Its capacity counts only arcs directed *out of*  $A$ :

$$c(A, \bar{A}) = \sum_{\substack{(u,v) \in E \\ u \in A, v \in \bar{A}}} c(u, v).$$

Define  $f_{\text{out}}(A)$  and  $f_{\text{in}}(A)$  by summing flows on arcs leaving and entering  $A$ , respectively. For compact formulas below,  $\delta^+(A)$  and  $\delta^-(A)$  denote those sets of original arcs.

**Lemma 6.1** (Generalized conservation across a cut). For every feasible flow and every  $s$ - $t$  cut  $(A, \bar{A})$ ,  $f_{\text{out}}(A) - f_{\text{in}}(A) = \text{value}(f)$ .

*Proof.* Sum  $f_{\text{out}}(v) - f_{\text{in}}(v)$  over  $v \in A$ . Each arc with both endpoints in  $A$  appears once with each sign and cancels. The surviving terms are exactly  $f_{\text{out}}(A) - f_{\text{in}}(A)$ . Conservation makes each internal vertex's contribution zero;  $t \notin A$ , so only  $s$  contributes  $\text{value}(f)$ .  $\square$

Because  $f_{\text{in}}(A) \geq 0$  and each outgoing flow is at most its capacity, this proves the bound against *every* cut:

$$\boxed{\text{value}(f) = f_{\text{out}}(A) - f_{\text{in}}(A) \leq c(A, \bar{A}).} \quad (1)$$

An especially useful equivalent identity is

$$\boxed{c(A, \bar{A}) - \text{value}(f) = [c(A, \bar{A}) - f_{\text{out}}(A)] + f_{\text{in}}(A).} \quad (2)$$

The bracket is the sum of  $c(u, v) - f(u, v)$  over outgoing arcs; the final term is the sum of  $f(v, u)$  over incoming arcs. Thus every summand is nonnegative, and the right side is exactly the total residual capacity leaving  $A$ .

**Theorem 6.2** (Augmenting paths and optimality). For a feasible flow  $f$ , the following are equivalent:

1.  $f$  is maximum;
  2.  $G_f$  has no  $s$ - $t$  path;
  3.  $\text{value}(f) = c(A, \bar{A})$  for some  $s$ - $t$  cut  $A$ .
- Consequently maximum flow value equals minimum cut capacity.

*Proof.* A residual path permits a strictly positive augmentation, so a maximum flow has no such path. If none exists, define  $S = \{v : v \text{ is reachable from } s \text{ in } G_f\}$  and  $T = V \setminus S$ . Then  $s \in S, t \in T$ . For an original arc  $(u, v)$  from  $S$  to  $T$ ,  $c(u, v) - f(u, v) = 0$ ; otherwise its forward residual arc would make its head reachable. For an original arc  $(v, u)$  from  $T$  to  $S$ ,  $f(v, u) = 0$ ; otherwise its reverse residual arc would do the same. Equation (2), applied to  $(S, T)$ , gives  $c(S, T) = \text{value}(f)$ . Finally, equality certifies optimality of both  $f$  and  $S$  by (1). A maximum exists because the feasible set is nonempty, closed, and bounded; applying the equivalence to it proves the equality of the two optima, even for real capacities.  $\square$

**Proof trap:** define  $S$  by residual reachability. An arbitrary partition at termination need not be minimum; an outgoing residual arc contradicts the head's non-reachability, not necessarily non-reachability of  $t$  directly.

### 6.3 Ford–Fulkerson and integral flows

---

**Algorithm 6:** Ford–Fulkerson: arbitrary residual augmenting paths

---

**Input:** Network  $G$ , capacities  $c(u, v) \geq 0$ , source  $s$ , sink  $t$   
 $f(u, v) \leftarrow 0$  for every original arc  $(u, v) \in E$ ;  
**while** a residual  $s$ – $t$  path exists **do**  
    Find a simple  $s$ – $t$  path  $P$  in  $G_f$ ;  
     $\Delta \leftarrow \min_{a \in P} c_f(a)$ ;  
    Apply the labelled forward/reverse updates along  $P$  by  $\Delta$ ;  
**return**  $f$  and the source-reachable cut in  $G_f$ ;

---

If it terminates, the algorithm is correct by Theorem 6.2. With integer capacities, all flows, residual capacities, and bottlenecks stay integral; each augmentation increases value by at least 1. Thus there are at most  $F^* = \text{value}(f^*)$  augmentations and time  $O(m(1 + F^*))$  after discarding isolated vertices. This is *pseudo-polynomial*: a binary capacity  $C$  occupies  $O(\log C)$  bits, not  $C$  bits. Rational capacities also give termination after scaling by a common denominator, without a polynomial iteration bound from this argument. For irrational capacities, unfortunate path choices may cause infinitely many augmentations; arbitrary-path Ford–Fulkerson is not a capacity-independent algorithm.

**Integer bad example:  $2C$  augmentations instead of two.** Use arcs  $s \rightarrow a, s \rightarrow b, a \rightarrow t, b \rightarrow t$  of capacity  $C$  and  $a \rightarrow b$  of capacity 1. Alternate  $s \rightarrow a \rightarrow b \rightarrow t$  with  $s \rightarrow b \rightarrow a \rightarrow t$ , where the second path uses the reverse residual arc of  $a \rightarrow b$ . Each augmentation has bottleneck 1; every pair increases all four outer flows by 1 and returns the middle flow to 0. There are  $2C$  augmentations before reaching value  $2C$ , exponential in the bit length of  $C$ . Taking the two direct paths would require only two augmentations.

**Theorem 6.3** (Edgewise integrality). With integer capacities, there exists a maximum flow with  $f(u, v) \in \mathbb{Z}$  for every original arc  $(u, v)$ .

*Proof.* Start at zero and use Ford–Fulkerson. Every augmentation is integral, and termination follows from the finite integer upper bound on flow value.  $\square$

This is an existence theorem, not a statement that every maximum flow is integral. For example, unit arcs  $s \rightarrow a, s \rightarrow b, a \rightarrow c, b \rightarrow c, c \rightarrow t$  admit a maximum flow of value 1 that sends  $1/2$  along each branch. An integral maximum also exists. An integer *total value* alone is not enough to decode a matching or an indivisible assignment.

### 6.4 Capacity scaling: $O(m^2(1 + \log C))$

Assume integer capacities and  $C = \max_{(u,v) \in E} c(u, v)$  (return zero if  $E$  is empty or  $C = 0$ ). For a threshold  $k$ , retain only labelled residual arcs of capacity at least  $k$ ; call this graph  $G_f^{\geq k}$ .

---

**Algorithm 7:** Capacity scaling: augment only through residual capacities  $\geq k$

---

$f \leftarrow 0$ ;  $k \leftarrow 2^{\lceil \log_2 C \rceil}$ ;  
**while**  $k \geq 1$  **do**  
    **while**  $G_f^{\geq k}$  has an  $s$ – $t$  path **do**  
        Find an  $s$ – $t$  path  $P$  in  $G_f^{\geq k}$ ;  
         $\Delta \leftarrow \min_{a \in P} c_f(a)$ ;  
        Apply the labelled updates along  $P$  by  $\Delta$ ;  
     $k \leftarrow k/2$ ;  
**return**  $f$ ;

---

**Correctness.** Feasibility is preserved. At the end of the  $k = 1$  phase, integral residual capacities imply every positive residual arc has capacity at least 1. Hence there is no residual  $s$ – $t$  path, and  $f$  is maximum. **Threshold trap:** starting at  $k = C$  with real halving can miss 1. For a path with capacities 3, 1, thresholds 3, 1.5 find no path and then stop, incorrectly returning zero. Powers of two ensure a final phase at 1.

**At most  $2m$  augmentations per phase.** At the end of a  $k$  phase, let  $S$  be the vertices reachable in  $G_f^{\geq k}$ . Each residual arc leaving  $S$  has capacity less than  $k$ . Each original crossing arc contributes once to (2), so

$$F^* - \text{value}(f) \leq c(S, \bar{S}) - \text{value}(f) < mk \quad (m > 0).$$

At the start of the next phase, whose threshold is  $k/2$ , the gap is less than  $2m(k/2)$ . Equivalently, a phase with threshold  $k$  starts with gap less than  $2mk$ . This also holds for the first phase since  $F^* \leq \sum_{(s,v) \in E} c(s, v) \leq mC < 2mk$ . Every augmentation adds at least  $k$ , so a phase has fewer than  $2m$  augmentations. The phase need not itself reach the optimum. There are  $1 + \lceil \log_2 C \rceil$  phases and each path search/update costs  $O(m)$  in the usual graph-operation model, giving  $O(m^2(1 + \log C))$  time.

### 6.5 Shortest augmenting paths: $O(nm^2)$

Use BFS in the full positive-residual graph to find an  $s$ – $t$  path with the *fewest arcs*; augment by its bottleneck and repeat until BFS fails. This is the standard Edmonds–Karp shortest-augmenting-path rule. Some course materials use different names: remember the path rule and its bound. It works with arbitrary nonnegative real capacities in the exact-arithmetic model; the number of graph operations does not depend on their magnitudes.

---

**Algorithm 8:** Shortest augmenting paths (BFS; not arbitrary path choice)

---

$f \leftarrow 0$ ;  
**while** BFS in  $G_f$  reaches  $t$  from  $s$  **do**  
    Let  $P$  be the shortest residual path given by BFS parent pointers;  
     $\Delta \leftarrow \min_{a \in P} c_f(a)$ ;  
    Apply the labelled updates along  $P$  by  $\Delta$ ;  
**return**  $f$ ;

---

**Lemma 6.4** (Residual distances never decrease). Let  $d(v)$  be the BFS distance from  $s$  before an augmentation and  $d'(v)$  afterwards, with distance  $\infty$  for unreachable vertices. Then  $d'(v) \geq d(v)$  for every  $v$ .

*Proof.* If false, take a shortest new path to a vertex whose distance decreased, and its first vertex  $v$  with  $d'(v) < d(v)$ . Its predecessor  $u$  satisfies  $d'(u) \geq d(u)$  and  $d'(v) = d'(u) + 1$ . If  $u \rightarrow v$  was an old residual arc, then  $d(v) \leq d(u) + 1 \leq d'(u) + 1 = d'(v)$ , a contradiction. Otherwise it is a newly available reverse of an arc  $v \rightarrow u$  on the old shortest augmenting path. Every prefix of that path is shortest, so  $d(u) = d(v) + 1$ , and  $d'(v) = d'(u) + 1 \geq d(u) + 1 = d(v) + 2$ , again a contradiction.  $\square$

**Lemma 6.5** (A residual arc is critical only  $O(n)$  times). Call a labelled residual arc *critical* when an augmentation saturates it. Each of the  $2m$  possible residual labels is critical at most  $O(n)$  times.

*Proof.* Suppose  $a : u \rightarrow v$  is saturated on a shortest path when distances are  $d$ . Then  $d(v) = d(u) + 1$ . Before  $a$  can be used and saturated again, positive capacity on its particular label must be restored by an augmentation using its paired reverse label  $v \rightarrow u$ . At that intervening augmentation, with distances  $\hat{d}$ , shortest-path choice and monotonicity imply

$$\hat{d}(u) = \hat{d}(v) + 1 \geq d(v) + 1 = d(u) + 2.$$

At the next critical use of  $a$ , its tail distance is at least  $\hat{d}(u)$ . Thus the tail distance increases by at least 2 between critical uses. A finite shortest simple path has at most  $n - 1$  arcs, so only  $O(n)$  such uses are possible.  $\square$

Each augmentation saturates at least one label, so there are  $O(nm)$  augmentations. BFS and updating take  $O(n + m)$  time, or  $O(m)$  after irrelevant isolated vertices are omitted. Thus the standard total is  $O(nm^2)$ . Finite termination plus Theorem 6.2 proves correctness. This algorithm augments one path per BFS; a blocking-flow algorithm is a different variant.

### 6.6 Flow decomposition and reduction templates

**Decomposition.** A nonnegative feasible flow of value  $F \geq 0$  decomposes into weighted simple  $s$ – $t$  paths of total weight  $F$  and directed cycles. To see this, add a labelled artificial arc  $t \rightarrow s$  carrying  $F$ , making a circulation. Follow positive-flow arcs until a cycle appears and subtract its minimum flow. Conservation prevents getting stuck; each subtraction eliminates an arc, so repeating gives a cycle decomposition. Removing the artificial arc turns precisely its cycles into  $s$ – $t$  paths. Integer flows give integer weights, hence  $F$  unit paths after splitting weights. Cycles may be discarded without changing value or violating capacity constraints.

**Reduction checklist.** Specify vertices, arcs, capacities, source/sink, and target flow value. Prove (i) an original solution gives a flow and (ii) an *integral* flow meeting the target decodes into an original solution. State graph size and invoke a polynomial-time flow algorithm. A fractional maximum-flow witness need not decode; choose an integral maximum instead.

**Vertex capacities.** To impose throughput  $b_v$  on an internal vertex  $v$ , replace it by  $v_{\text{in}} \rightarrow v_{\text{out}}$  of capacity  $b_v$ . Redirect each original  $u \rightarrow v$  to  $u_{\text{out}} \rightarrow v_{\text{in}}$ , retaining its capacity. Incoming/outgoing notation refers to unsplit vertices where appropriate. Conservation forces all throughput across the splitting arc; conversely an original flow respecting  $b_v$  extends by putting that throughput on it. Leave terminals unsplit unless their own throughput is constrained.

**Multiple sources/sinks and exact supplies.** Add a super-source  $s_0$  with  $s_0 \rightarrow a_i$  of capacity equal to source  $i$ 's supply bound, and arcs  $b_j \rightarrow t_0$  for sink receipt bounds. If exact supplies  $a_i$  and demands  $b_j$  sum to  $B$ , target value  $B$  forces all these arcs to be saturated and gives the prescribed amounts. For unrestricted auxiliary arcs, use a finite upper bound on all possible flow (for example  $1 + \sum_{(u,v) \in E} c(u,v)$ ), rather than an unspecified infinity.

**Theorem 6.6** (Edge form of Menger). For distinct vertices  $s, t$ , the maximum number of pairwise edge-disjoint  $s$ - $t$  paths equals the minimum number of edges whose deletion destroys all  $s$ - $t$  paths. This holds for both directed and undirected graphs; directed paths must respect arc directions.

*Proof.* Give every original directed arc capacity 1. A family of  $q$  edge-disjoint  $s$ - $t$  paths gives a feasible value- $q$  flow. Conversely, decompose an integral maximum flow into unit paths and discard cycles. Capacity 1 prevents any original arc from belonging to two paths. Hence their maximum number equals the minimum number of outgoing arcs in an  $s$ - $t$  cut. Every such cut is an edge separator. Conversely, after deleting an edge separator, let  $S$  be the vertices reachable from  $s$ . Then  $t \notin S$ , and every original arc leaving  $S$  was deleted, so the cut is no larger than that separator. For undirected edges, insert both orientations of capacity 1. Cancel equal flow on opposite orientations first; this preserves conservation and value. Then decomposition uses each undirected edge at most once across all paths. A cut counts exactly one orientation of each crossing undirected edge.  $\square$

Unlike the internal-vertex separator version below, the edge version allows  $s$  and  $t$  to be adjacent: their direct edge can be deleted.

## 6.7 Bipartite matching: Hall's and Kőnig's theorems

Let  $G = (L \cup R, E)$  be bipartite. A *matching* is a set of edges with no shared endpoint; it *saturates* a vertex set if every vertex in that set is matched. A *perfect matching* saturates all vertices. Write  $\nu(G)$  for maximum matching size, and for  $A \subseteq L$  define  $N(A) = \{v \in R : uv \in E \text{ for some } u \in A\}$ .

**Flow reduction.** Make  $s \rightarrow u$  for  $u \in L$ ,  $u \rightarrow v$  for  $uv \in E$ , and  $v \rightarrow t$  for  $v \in R$ , all of capacity 1. A matching of size  $q$  gives  $q$  unit paths. Conversely, in an integral flow, edges  $uv$  carrying 1 form a matching: source/terminal capacity 1 and conservation allow at most one at each endpoint. Their number equals the flow value. Thus maximum matching size is the maximum flow value; saturating  $L$  means target  $|L|$ . A perfect matching also requires  $|L| = |R|$ .

### Augmenting paths: a certificate of optimality

An *M*-*augmenting path* is a simple path whose endpoints are unmatched by  $M$  and whose edges alternate outside and inside  $M$ , starting and ending outside  $M$ . It has odd length. Flipping membership of its edges gives the matching  $M \triangle E(P)$  of size  $|M| + 1$ .

**Theorem 6.7** (Berge's lemma). A matching  $M$  is maximum if and only if there is no  $M$ -augmenting path. This statement holds in general graphs as well as bipartite graphs.

*Proof.* An augmenting path improves  $M$ , so a maximum matching has none. Conversely, suppose another matching  $M'$  has  $|M'| > |M|$ . In the symmetric difference  $M \triangle M'$ , every vertex has degree at most 2 and the edges alternate between the two matchings. Components are alternating paths and even cycles. Cycles contribute equally many edges to each matching; the size inequality forces a path with one more  $M'$  edge than  $M$  edge. Its endpoints are unmatched in  $M$ , so it is augmenting.  $\square$

In the bipartite flow reduction, direct unmatched edges from  $L$  to  $R$  and matched edges from  $R$  to  $L$ . A search from all unmatched left vertices finds an augmenting path exactly when it reaches an unmatched right vertex. This is a residual  $s$ - $t$  path with the terminal arcs omitted. **Maximal is weaker than maximum:** a maximal matching only rules out augmenting paths of length 1, not longer ones.

### Hall's condition and deficiency

**Theorem 6.8** (Hall's marriage theorem). There is a matching saturating every vertex of  $L$  if and only if

$$|N(A)| \geq |A| \quad \text{for every } A \subseteq L.$$

In particular, when  $|L| = |R|$ , this is equivalent to a perfect matching.

*Proof.* Necessity: vertices in  $A$  need distinct partners in  $N(A)$ . For sufficiency, use the flow reduction but give every  $L \rightarrow R$  arc capacity  $B = |L| + 1$ , leaving terminal capacities 1. Integral flows still decode into matchings because each left and right vertex has capacity 1. The cut  $\{s\}$  has capacity  $|L| < B$ , so a minimum cut cannot cross a compatibility arc. If its source side contains

$A \subseteq L$  and  $D \subseteq R$ , this forces  $N(A) \subseteq D$ , and its capacity is  $|L| - |A| + |D|$ . For fixed  $A$ , choose  $D = N(A)$  to obtain

$$\nu(G) = \min_{A \subseteq L} (|L| - |A| + |N(A)|) = |L| - \max_{A \subseteq L} (|A| - |N(A)|). \quad (3)$$

Hall's condition makes every cut capacity at least  $|L|$ . Max-flow/min-cut and integrality therefore give a matching saturating  $L$ .  $\square$

**Deficiency formula and how to use it.** The quantity  $\delta = \max_{A \subseteq L} (|A| - |N(A)|) \geq 0$  is the *deficiency*; it is nonnegative because  $A = \emptyset$  is allowed. Equation (3) says that every maximum matching leaves exactly  $\delta$  left vertices unmatched. To disprove a full assignment, exhibit one deficient set  $A$ ; checking only single vertices, or only  $A = L$ , is insufficient. To prove a maximum value, give a matching and a deficient set attaining the same bound. A minimum cut in the capacity- $B$  network supplies a set attaining the maximum deficiency; do not enumerate all  $2^{|L|}$  subsets algorithmically.

**Corollary 6.9** (Regular bipartite graphs). Every finite  $d$ -regular bipartite graph with  $d \geq 1$  has a perfect matching.

*Proof.* Counting edges gives  $d|L| = d|R|$ . For every  $A \subseteq L$ , all  $d|A|$  incident edges end in  $N(A)$ , which can receive at most  $d|N(A)|$  edges. Thus  $|A| \leq |N(A)|$ , and Hall applies. Removing a perfect matching leaves a  $(d-1)$ -regular graph; repeating decomposes the edges into  $d$  perfect matchings.  $\square$

**Corollary 6.10** (Capacitated Hall condition). Suppose each  $u \in L$  needs one assignment and each  $v \in R$  may receive at most  $b_v \in \mathbb{Z}_{\geq 0}$  assignments along compatibility edges. All left vertices can be assigned if and only if

$$|A| \leq \sum_{v \in N(A)} b_v \quad \text{for every } A \subseteq L.$$

*Proof.* Replace the capacity of  $v \rightarrow t$  by  $b_v$ . The same minimum-cut argument gives capacity  $|L| - |A| + \sum_{v \in N(A)} b_v$ ; integrality converts a value- $|L|$  flow into individual assignments.  $\square$

This version has one capacity per right vertex. Additional group or period quotas need the extra network layers in Section 6.9.

### Kőnig's equality and constructing a minimum cover

A *vertex cover* meets every edge; write  $\tau(G)$  for its minimum size. In any graph  $\nu(G) \leq \tau(G)$  because disjoint matching edges need distinct covering vertices.

**Theorem 6.11** (Kőnig's theorem). For every bipartite graph,  $\nu(G) = \tau(G)$ : the maximum matching size equals the minimum vertex-cover size.

*Proof.* In the capacity- $B$  matching network from Hall's proof, a minimum cut has source side  $\{s\} \cup A \cup D$ , where  $N(A) \subseteq D$ . Then  $C = (L \setminus A) \cup D$  covers every edge: an edge whose left endpoint is in  $A$  must have its right endpoint in  $D$ . Moreover,  $|C| = |L| - |A| + |D|$ , exactly the cut capacity, which equals  $\nu(G)$ . Together with  $\nu(G) \leq \tau(G)$ , this proves equality.  $\square$

**Recover the actual cover from a maximum matching.** Start at all unmatched vertices in  $L$ . Traverse unmatched edges  $L \rightarrow R$  and matched edges  $R \rightarrow L$ ; let the reached sets be  $Z_L, Z_R$ . Return

$$C = (L \setminus Z_L) \cup Z_R.$$

No unmatched right vertex is reached, by Berge's lemma. If  $u \in Z_L$ , every neighbor  $v$  is in  $Z_R$ : an unmatched edge can be traversed directly; if  $uv$  is matched, the search could only have reached matched  $u$  through  $v$ . Therefore  $C$  covers every edge. The endpoints of any matching edge are either both reached or both unreached, so exactly one belongs to  $C$ . No unmatched vertex belongs to  $C$ . Hence  $|C| = |M|$ , certifying that both  $M$  and  $C$  are optimal. This extra search takes  $O(|V| + |E|)$  time.

**Independent-set corollary and pitfalls.** The complement of a vertex cover is an independent set, and conversely. Thus for bipartite graphs the maximum independent-set size is  $\alpha(G) = |V| - \tau(G) = |V| - \nu(G)$ , and  $V \setminus C$  constructs one. These are *unweighted cardinality* statements; weighted vertex cover uses the cut reduction below, not the cardinality  $\nu(G)$ . Kőnig's equality fails for general graphs: a triangle has  $\nu = 1$  and  $\tau = 2$ . Also, taking *both* endpoints of every matching edge gives the earlier 2-approximation, not necessarily a minimum cover.

**Weighted bipartite vertex cover is also exactly solvable.** For nonnegative vertex weights  $w_v$ , set  $c(s, u) = w_u$  for  $u \in L$ ,  $c(v, t) = w_v$  for  $v \in R$ , and give every compatibility arc  $u \rightarrow v$  capacity  $B = 1 + \sum_v w_v$ . A minimum cut cannot cross a capacity- $B$  arc. For its source side  $\{s\} \cup A \cup D$ , the cover  $(L \setminus A) \cup D$  has weight equal to the cut capacity. Conversely, every cover  $C$  gives this cut by choosing  $A = L \setminus C$  and  $D = R \cap C$ . Thus the minimum cut returns a minimum-weight cover; the weights need not be integral.

**Worked certificate: matching, Hall obstruction, and cover.** Let  $L = \{a, b, c, d\}$ ,  $R = \{1, 2, 3, 4\}$ , with

$$N(a) = N(b) = \{1\}, \quad N(c) = \{2, 3\}, \quad N(d) = \{3, 4\}.$$

Every vertex has a neighbor and  $N(L) = R$ , yet  $A = \{a, b\}$  has only one neighbor, so no perfect matching exists. The matching  $M = \{a1, c2, d3\}$  has size 3, and the deficient set  $A$  bounds the size by  $4 - (2 - 1) = 3$ , so  $M$  is maximum. From the unmatched left vertex  $b$ , the alternating search reaches  $Z_L = \{a, b\}$  and  $Z_R = \{1\}$ . It returns the minimum cover  $C = \{c, d, 1\}$  and maximum independent set  $V \setminus C = \{a, b, 2, 3, 4\}$ .

## 6.8 Vertex-disjoint paths and Menger's theorem

**Theorem 6.12** (Vertex form of Menger). For distinct nonadjacent vertices  $x, y$  in an undirected graph, the maximum number  $p$  of internally vertex-disjoint  $x$ - $y$  paths equals the minimum size  $q$  of a set  $Z \subseteq V \setminus \{x, y\}$  whose deletion separates  $x$  from  $y$ .

*Proof.* Let  $N$  be the original number of vertices and choose  $M = N + 1$ . Split every internal vertex into  $v_{\text{in}} \rightarrow v_{\text{out}}$  of capacity 1; keep  $x, y$  unsplit. Replace every undirected edge  $\{u, v\}$  by  $u_{\text{out}} \rightarrow v_{\text{in}}$  and  $v_{\text{out}} \rightarrow u_{\text{in}}$ , each of capacity  $M$ , interpreting unsplit endpoints as themselves. Source is  $x$ , sink is  $y$ . *Paths correspond to flow.* Internally disjoint paths give unit flows without exceeding any splitting capacity, so maximum flow  $F^* \geq p$ . Conversely, integral decomposition produces  $F^*$  unit paths. No two can use the same internal splitting arc, so they decode to internally vertex-disjoint paths. Thus  $p = F^*$ . *Separators correspond to cuts.* Given a separator  $Z$ , delete its splitting arcs and take the source-reachable set in the remaining network. The sink is unreachable. Every arc crossing out of this set in the original network must be a deleted splitting arc, so it defines a cut of capacity at most  $|Z|$ . Hence minimum cut capacity  $C^* \leq q$ . Because  $x, y$  are nonadjacent, deleting all  $N - 2$  other vertices separates them; therefore  $C^* \leq N - 2 < M$ . No minimum cut can contain an outgoing  $M$ -capacity arc. Its outgoing arcs are all unit splitting arcs; let  $Z$  be their original vertices. Every  $x$ - $y$  path must cross that cut and hence one of those splitting arcs, so  $Z$  is a separator with  $|Z| = C^*$ . Thus  $q \leq C^*$  as well. Max-flow/min-cut gives  $p = F^* = C^* = q$ .  $\square$

Nonadjacency is needed for this separator formulation: deleting internal vertices cannot destroy a direct edge  $xy$ . The same splitting idea gives the directed internally vertex-disjoint path variant when an internal separator exists.

## 6.9 Layered scheduling with two quota levels

There are workers  $i = 1, \dots, r$ , periods  $j = 1, \dots, k$ , and  $D$  tasks. Task  $d$  belongs to period  $p(d)$ . Worker  $i$  can perform tasks in  $A_i$ , may perform at most  $b_i$  tasks overall, where  $b_i$  is a nonnegative integer, and at most one per period. Every task needs exactly one worker. Construct the layers

$$s \xrightarrow{b_i} w_i \xrightarrow{1} w_{ij} \xrightarrow{1} d \xrightarrow{1} t, \quad d \in A_i, \quad p(d) = j.$$

There is one  $w_i$  per worker, one  $w_{ij}$  per worker-period, one vertex per task, and no other arcs. The target is  $D$ .

**Both directions.** Given a feasible schedule, send one unit along the indicated path for each task's assigned worker. Total and period quotas enforce the first two capacities; each task is used once, enforcing the last two. Conservation holds along each path, so the flow has value  $D$ . Conversely, an integral flow of value  $D$  saturates every task-to-sink arc. At each task, exactly one incoming assignment arc carries 1. Decode that worker as its assignee. Assignment arcs guarantee availability; capacity 1 on  $w_i \rightarrow w_{ij}$  enforces the period quota; capacity  $b_i$  on  $s \rightarrow w_i$  enforces the total quota. Thus a feasible schedule exists iff maximum flow has value  $D$ . A fractional assignment is not a schedule, but the integral maximum-flow theorem supplies the required witness.

**ILP formulation.** Use binary  $x_{id}$  only for available pairs  $d \in A_i$ :

$$\begin{aligned} & \text{minimize} && 0 \\ & \text{subject to} && \sum_{i: d \in A_i} x_{id} = 1 \quad \text{for every task } d, \\ & && \sum_{d \in A_i} x_{id} \leq b_i \quad \text{for every worker } i, \\ & && \sum_{\substack{d \in A_i: p(d)=j \\ x_{id} \in \{0,1\}}} x_{id} \leq 1 \quad \text{for every worker-period } (i, j), \end{aligned}$$

Unavailable variables are absent or fixed to zero. A nonnegative integer period quota  $b_{ij}$  replaces 1 in the second-layer capacity and corresponding ILP constraint. The network has  $O(rk + D)$  vertices and  $O(r + rk + D + \sum_i |A_i|)$  arcs, so one polynomial-time max-flow computation suffices. If an original formulation allows redundant coverage and only imposes upper quotas, discard excess assignments first to get exactly one worker per task.

## 6.10 Minimum-cut structure and tie-breaking

**Theorem 6.13** (Every minimum cut is saturated by every maximum flow). Fix any maximum flow  $f$ . For every minimum cut  $S$ , each original arc  $(u, v)$  leaving  $S$  satisfies  $f(u, v) = c(u, v)$ , and each original arc  $(v, u)$  entering  $S$  satisfies  $f(v, u) = 0$ .

*Proof.* Since  $c(S, \bar{S}) = \text{value}(f)$ , the left side of (2) is zero. Every term on its right is nonnegative, so each is zero.  $\square$

**No opposite crossings.** A positive-capacity arc cannot leave one minimum cut's source side and enter another's. Fix the *same* maximum flow for both cuts: the first requires  $f(u, v) = c(u, v) > 0$ , while the second requires  $f(u, v) = 0$ , a contradiction. The positive-capacity assumption is necessary for this argument.

**Theorem 6.14** (Residual characterization and closure of minimum cuts). Fix a maximum flow  $f$ . A set  $S$  with  $s \in S$ ,  $t \notin S$  is the source side of a minimum cut if and only if no positive-capacity residual arc leaves  $S$ . Consequently, the intersection and union of any two minimum-cut source sides are also minimum-cut source sides.

*Proof.* No residual arc leaves  $S$  exactly when every original outgoing arc is saturated and every original incoming arc has zero flow. Equation (2) makes this equivalent to  $c(S, \bar{S}) = \text{value}(f)$ . For intersection and union, use the same residual network: a vertex in  $S_1 \cap S_2$  cannot reach outside either set by one residual arc, and a vertex in  $S_1 \cup S_2$  stays in whichever set contains it. Both sets still contain  $s$  and exclude  $t$ , so the characterization applies.  $\square$

**Smallest/largest source sides and uniqueness.** Let  $R_s$  be the vertices reachable from  $s$  in  $G_f$ , and  $R_t$  the vertices that can reach  $t$  in  $G_f$  (search from  $t$  with all residual arcs reversed). Since  $f$  is maximum,  $R_s \cap R_t = \emptyset$ . Both  $R_s$  and  $V \setminus R_t$  have no outgoing residual arc, so they are minimum-cut source sides. Every minimum-cut source side  $S$  satisfies

$$R_s \subseteq S \subseteq V \setminus R_t :$$

residual closure forces it to contain everything reachable from  $s$ , and it cannot contain a vertex with a residual path to  $t$ . Thus these are the smallest and largest source sides by inclusion. The minimum-cut partition is unique if and only if  $R_s = V \setminus R_t$ , equivalently  $R_s \cup R_t = V$ . The two searches take  $O(n + m)$  time after maximum flow is found. Containment alone does not make an intermediate set a minimum cut; it must also have no outgoing residual arc.

**Minimum capacity first, then fewest outgoing arcs.** For integer capacities, define  $c'(u, v) = (m + 1)c(u, v) + 1$  and compute a minimum cut under  $c'$ . Its transformed capacity is

$$c'(S, \bar{S}) = (m + 1)c(S, \bar{S}) + |\delta^+(S)|.$$

Any increase of at least 1 in original cut capacity raises the first term by at least  $m + 1$ , which exceeds every possible difference in outgoing-arc count (at most  $m$ ). Therefore a minimum transformed cut first minimizes  $c(S, \bar{S})$  and, among ties, minimizes  $|\delta^+(S)|$ . The capacity encoding grows by only  $O(\log(m + 1))$  bits. Integrality is essential to the unit-gap argument; arbitrary real capacities need a separately justified gap bound.

**Lower bounds and vertex demands.** The variants  $\ell(u, v) \leq f(u, v) \leq c(u, v)$  and  $\text{in}_f(v) - \text{out}_f(v) = d(v)$  impose minimum arc flows and net supply/demand at vertices. Their full feasibility reduction is in Appendix G.

## A Targeted Practice Questions

These practice variations combine the supplied lectures, Tutorials 1–4, and question styles from the two sample finals. They are speculative revision questions, not information about the actual midterm. Marks are suggested practice weights. Attempt the questions before consulting the separate answer sketches.

### A.1 Questions

**Q1. Scheduling with two levels of quotas (10 marks).** There are workers  $i$ , disjoint periods  $j$ , and tasks, each belonging to one period. Worker  $i$  can perform tasks in  $A_i$ , may perform at most  $b_i$  tasks overall, and at most one task per period. Every task must be performed by exactly one worker. (a) Construct a flow network and specify the target flow value. (b) Prove both directions of the reduction. (c) Give an ILP. (d) Explain why a fractional feasible flow is not itself a schedule, and why this does not prevent a flow-based algorithm.

**Q2. Bounded-frequency weighted covering (8 marks).** Each element belongs to at most  $k$  candidate sets with nonnegative costs. Give an ILP and a deterministic  $k$ -approximation using its LP relaxation. Prove feasibility and cost separately. Does the same threshold proof work if instead every set contains at most  $k$  elements?

**Q3. Randomized cut, then remove the randomness (8 marks).** For weighted MAX- $k$ -CUT on a loopless undirected graph with nonnegative weights, design an algorithm obtaining at least  $(1 - 1/k)$  OPT in expectation. Convert it to a deterministic polynomial-time algorithm. State precisely how to compute conditional expectation after a partial labelling. Explain why independence of the edge indicators is unnecessary.

**Q4. Randomized set cover with a stopping rule (10 marks).** Solve the natural weighted-set-cover LP on a feasible instance with  $n \geq 1$  elements. One trial takes  $r = \lceil \ln(2n) \rceil$  independent rounds, selecting each set  $j$  independently with probability  $x_j^*$  in each round and taking their union. Discard unsuccessful trials and repeat independently until a cover is obtained. Prove almost-sure feasibility, expected polynomial running time, and expected returned cost at most  $2r$  OPT. Explain why “one trial has expected cost at most  $rLP^*$ , so the returned cover does too” is not justified.

**Q5. Minimum-cut structure and tie-breaking (8 marks).** (a) Fix any maximum flow  $f$ . Prove that for every minimum cut, outgoing arcs are saturated and incoming arcs carry zero flow. (b) Deduce that a positive-capacity arc cannot cross two minimum cuts in opposite directions. (c) For integer capacities, find a minimum cut with the fewest outgoing arcs.

**Q6. MAX-DICUT rounding and a changed probability (8 marks).** For a loopless directed graph with nonnegative arc weights, use the LP  $z_{ij} \leq x_i, z_{ij} \leq 1 - x_j$ , with variables in  $[0, 1]$ , maximizing  $\sum_{ij} w_{ij} z_{ij}$ . Prove that independently placing vertex  $i$  in the source side with probability  $p_i = 1/4 + x_i/2$  gives at least half the LP value in expectation. As an extension, determine whether direct rounding  $p_i = x_i$  also guarantees half the optimum when the LP solution is optimal.

**Q7. MAX-SAT clause lengths and combination (10 marks).** Write the weighted MAX-SAT ILP and LP. Prove the LP-rounding bound for a clause of length  $\ell$ , including the AM–GM and concavity steps. Then show why taking the better of LP rounding and fair random assignment obtains  $3\text{OPT}/4$  in expectation. State the preprocessing needed for the clause-product calculation.

**Q8. Metric TSP and the role of assumptions (8 marks).** Prove the double-tree factor 2 and Christofides factor  $3/2$ . Explain exactly where triangle inequality, even parity, and minimum-weight perfect matching are used. Give a four-vertex metric instance where a valid double-tree execution is nonoptimal. Explain why arbitrary nonmetric costs invalidate the shortcut argument.

**Q9. Knapsack scaling (8 marks).** For nonnegative integer item values, derive the value-indexed  $O(n^2V)$  DP, where  $V$  is the largest item value. Use it to obtain an FPTAS with  $O(n^3/\epsilon)$  arithmetic-operation complexity. Prove the guarantee and explain why overweight items must be removed before defining  $V$ .

**Q10. Vertex-disjoint routes (8 marks).** For distinct nonadjacent vertices  $x, y$  of an undirected graph, prove that the maximum number of internally vertex-disjoint  $x$ - $y$  paths equals the minimum size of an  $x$ - $y$  vertex separator. Give the flow construction and justify both correspondences. Why do capacity-1 splitting arcs suffice, and why are the other arcs assigned capacity larger than  $n$ ?

**Q11. Repair a flow algorithm (8 marks).** (a) A scaling algorithm starts at  $k = C$ , halves by real division, and stops below 1. Find an integer-capacity counterexample and repair the algorithm. (b) Prove that BFS augmentations give  $O(nm^2)$  time independent of capacity magnitudes. (c) Explain why arbitrary Ford–Fulkerson is only pseudo-polynomial for integer capacities and may fail to terminate for irrational capacities.

**Q12. An unfamiliar objective with a familiar proof (6 marks).** You are given  $n$  strings of length  $L$  over an alphabet of  $q$  symbols. Give a deterministic  $1/q$  retained-value approximation for their longest common subsequence in  $O(nL + q)$  time. Give a second proof using a sum of per-symbol upper bounds on OPT.

## A.2 Quick true/false drill

For each statement give a proof or a small counterexample.

**T1.** Complementing any 2-approximate vertex cover gives a constant-factor approximate maximum independent set.

**T2.** Integer capacities imply every maximum flow is integral on every arc.

**T3.** A feasible flow whose value equals the capacity of some cut is maximum.

**T4.** An integer-capacity flow algorithm taking  $O(mC)$  time is polynomial in the binary input length.

**T5.** A constant-factor approximation guarantee for MAX-SAT automatically gives one for minimizing unsatisfied clauses.

**T6.** Shortest residual distances never decrease under arbitrary Ford–Fulkerson path choices.

**T7.** A feasible solution of a minimization LP relaxation upper-bounds the integer optimum.

**T8.** Every graph on  $n \geq 3$  vertices has vertex-cover LP integrality gap at least  $2 - 2/n$ .

**T9.** Endpoints of a maximal matching always form a vertex cover; endpoints of a maximum matching always form a minimum cover.

**T10.** The indicators of two edges sharing a vertex must be independent in order to sum their expectations.

### A.3 Answer sketches

**Q1. Scheduling.** Use layers

$$s \longrightarrow i \longrightarrow (i, j) \longrightarrow d \longrightarrow t$$

with capacities  $b_i, 1, 1, 1$ , respectively. Include  $(i, j) \rightarrow d$  exactly when task  $d$  is in period  $j$  and  $d \in A_i$ . The target flow value is the number  $D$  of tasks. A schedule sends one unit on the path for each assignment. Conversely, an integral flow of value  $D$  saturates all task-to-sink arcs. Each task then has exactly one unit of incoming flow; decode that arc as its assigned worker. The worker and worker-period capacities enforce the two quotas. Fractional flow can split a task across workers, but integer capacities guarantee the existence of an integral maximum flow, which the standard augmenting-path algorithms can return.

For available pairs use binary  $a_{id}$  and the feasibility ILP

$$\begin{aligned} \sum_{i: d \in A_i} a_{id} &= 1 \quad (\text{each } d), \\ \sum_{d \in A_i} a_{id} &\leq b_i \quad (\text{each } i), \\ \sum_{\substack{d \in A_i \\ d \text{ in period } j}} a_{id} &\leq 1 \quad (\text{each } i, j). \end{aligned}$$

The network has polynomial size and requires one maximum-flow computation.

**Q2. Bounded-frequency covering.** Minimize  $\sum_j c_j x_j$  subject to  $\sum_{j: e \in S_j} x_j \geq 1$  for every element, and  $x_j \in \{0, 1\}$ . Relax to  $[0, 1]$  and select  $x_j^* \geq 1/k$ . A covering constraint has at most  $k$  terms, so at least one meets the threshold. With nonnegative costs,

$$\text{ALG} \leq \sum_j k c_j x_j^* = k LP^* \leq k \text{OPT}.$$

Bounded *set size* does not bound the number of terms in a covering constraint. For example, one element and two identical unit-cost singleton sets admit optimal LP solution  $(1/2, 1/2)$ . Set size is at most 1, but threshold 1 selects neither set.

**Q3. MAX- $k$ -CUT and conditional expectation.** Independent uniform vertex labels cut an edge with probability  $1 - 1/k$ . Linearity gives expectation  $(1 - 1/k) \sum_e w_e \geq (1 - 1/k) \text{OPT}$ ; edge indicators need not be independent. Under a partial labelling, an edge with both endpoints fixed contributes its weight or zero. An edge with at least one endpoint unfixed contributes  $(1 - 1/k)w_e$ . For the next vertex, only edges to already fixed neighbors have a label-dependent contribution. Choose the label maximizing the weight of differently labelled fixed neighbors. The current conditional expectation is the average of the  $k$  branch expectations, so this choice cannot decrease it. After all choices the objective equals its conditional expectation. A direct implementation takes  $O(k(n + m))$  time.

**Q4. Set cover with rejection sampling.** For any element  $e$ , one round misses it with probability

$$\prod_{j: e \in S_j} (1 - x_j^*) \leq \exp \left( - \sum_{j: e \in S_j} x_j^* \right) \leq e^{-1}.$$

After  $r$  rounds this is at most  $e^{-r} \leq 1/(2n)$ . A union bound implies trial success probability  $p \geq 1/2$ . For trial cost  $C$ ,

$$\mathbb{E}[C] = \sum_j c_j (1 - (1 - x_j^*)^r) \leq r \sum_j c_j x_j^* = r LP^*.$$

The accepted trial has the distribution of one trial conditioned on success  $A$ . Since  $C \geq 0$ ,

$$\mathbb{E}[C \mid A] = \frac{\mathbb{E}[C \mathbf{1}_A]}{p} \leq \frac{\mathbb{E}[C]}{p} \leq 2r LP^* \leq 2r \text{OPT}.$$

Fresh independent trials give a geometric stopping time of mean  $1/p \leq 2$ ; non-termination probability is  $\lim_{t \rightarrow \infty} (1 - p)^t = 0$ . Each trial is polynomial time and its output is checked for feasibility. The success event may be correlated with cost, so the unconditional trial expectation alone does not bound the returned cost.

**Q5. Cuts.** For a fixed maximum flow and any minimum-cut source side  $S$ ,

$$0 = c(\delta^+(S)) - |f| = \sum_{e \in \delta^+(S)} (c_e - f_e) + \sum_{e \in \delta^-(S)} f_e.$$

All summands are nonnegative, so each vanishes. An arc with  $c_e > 0$  crossing outwards forces  $f_e = c_e > 0$ ; the same arc crossing inwards for another minimum cut would force  $f_e = 0$ , a contradiction. For tie-breaking let  $m = |E|$  and set  $c'_e = (m + 1)c_e + 1$ . Then

$$c'(\delta^+(S)) = (m + 1)c(\delta^+(S)) + |\delta^+(S)|.$$

An integer capacity difference of at least 1 outweighs every cardinality difference, which is at most  $m$ . Minimize the new capacity with one max-flow/min-cut computation. Its encoding length grows by  $O(\log(m + 1))$ .

**Q6. MAX-DICUT.** By independence for the two distinct endpoints and the LP constraints,

$$\begin{aligned} \Pr[i \in U, j \notin U] &= (1/4 + x_i/2)(1/4 + (1 - x_j)/2) \\ &\geq (1/4 + z_{ij}/2)^2 \\ &= z_{ij}/2 + (2z_{ij} - 1)^2/16 \geq z_{ij}/2. \end{aligned}$$

Sum with the nonnegative weights. For direct rounding, put  $L = \sum w_{ij} z_{ij} = LP^*$  and  $W = \sum w_{ij}$ . If  $W = 0$  the claim is trivial. Otherwise  $x_i(1 - x_j) \geq z_{ij}^2$ , and Cauchy–Schwarz gives

$$\mathbb{E}[\text{ALG}] \geq \sum w_{ij} z_{ij}^2 \geq L^2/W.$$

The feasible point  $x_i = z_{ij} = 1/2$  implies  $L \geq W/2$  when the LP solution is optimal. Hence  $\mathbb{E}[\text{ALG}] \geq L/2 \geq \text{OPT}/2$ . The shifted rule has a local per-arc proof; the direct rule uses LP optimality for its global comparison.

**Q7. MAX-SAT.** Remove duplicate literals; treat tautological clauses as always satisfied. For each remaining nonempty clause  $C_i$ , use

$$\max \sum_i w_i y_i, \quad \sum_{j \in C_i^+} x_j + \sum_{j \in C_i^-} (1 - x_j) \geq y_i, \quad x_j, y_i \in \{0, 1\};$$

relax the domains to  $[0, 1]$ . Independent variable rounding gives literal success probabilities  $a_1, \dots, a_\ell$  in a clause with distinct variables, and  $\sum_h a_h \geq y_i^*$ . AM–GM gives

$$\Pr[C_i \text{ fails}] = \prod_h (1 - a_h) \leq \left( 1 - \frac{\sum_h a_h}{\ell} \right)^\ell \leq (1 - y_i^*/\ell)^\ell.$$

The function  $g(y) = 1 - (1 - y/\ell)^\ell$  is concave on  $[0, 1]$  with  $g(0) = 0$ . Thus  $g(y) \geq yg(1)$ , and the clause contributes at least  $b_\ell w_i y_i^*$  where  $b_\ell = 1 - (1 - 1/\ell)^\ell \geq 1 - 1/e$ . Fair assignment contributes at least  $a_\ell w_i y_i^*$ , with  $a_\ell = 1 - 2^{-\ell}$  because  $y_i^* \leq 1$ . For  $\ell = 1, 2$ ,  $a_\ell + b_\ell = 3/2$ ; for  $\ell \geq 3$ ,  $a_\ell \geq 7/8$  and  $b_\ell \geq 1 - 1/e > 5/8$ . Therefore, for outputs  $A, B$ ,

$$\mathbb{E}[\max(A, B)] \geq \frac{1}{2}(\mathbb{E}[A] + \mathbb{E}[B]) \geq \frac{3}{4} LP^* \geq \frac{3}{4} \text{OPT}.$$

The two outputs need not be independent. Averaging just the global  $1/2$  and  $1 - 1/e$  guarantees would not prove  $3/4$ .

**Q8. TSP.** Deleting a tour edge proves  $c(T) \leq \text{OPT}$  for an MST  $T$ . Doubling gives even degrees and cost  $2c(T)$ ; triangle inequality makes the final shortcut tour no more expensive. For Christofides, the odd-degree set has even cardinality. Shortcut an optimal tour to this set and split its alternating edges into two perfect matchings. The minimum matching  $M$  costs no more than their average, at most  $\text{OPT}/2$ . Thus the output costs at most  $c(T) + c(M) \leq 3 \text{OPT}/2$ .

For the requested example, use the unit-cost cycle  $A, B, D, C, A$  and diagonal costs 2. MST  $\{AB, AC, CD\}$  has Euler circuit  $A, B, A, C, D, C, A$  after doubling. First-visit shortcutting returns  $A, B, C, D, A$ , cost 6, while the original cycle has optimal cost 4. Without triangle inequality a shortcut edge can cost arbitrarily more than the detour it replaces.

**Q9. Knapsack.** Discard  $w_i > W$ , and handle no remaining items or  $V = 0$  separately. Let  $D[i, p]$  be the minimum weight of a subset of the first  $i$  items with value exactly  $p$ . Set  $D[0, 0] = 0$ , all other initial states to  $\infty$ , and

$$D[i, p] = \min\{D[i-1, p], w_i + D[i-1, p-v_i]\},$$

omitting the second term when  $p < v_i$ . Fill increasing  $i$ ; choose the largest  $p$  with  $D[n, p] \leq W$  and backtrack stored choices. Since  $p \leq nV$ , this takes  $O(n^2V)$  arithmetic operations.

For  $0 < \epsilon < 1$ , put  $K = \epsilon V/n$  and  $\hat{v}_i = \lfloor v_i/K \rfloor \leq n/\epsilon$ . Solve the DP with scaled values and original weights. It takes  $O(n^3/\epsilon)$  operations. For its returned set  $S$  and an original optimum  $S^*$ ,

$$\begin{aligned} v(S) &\geq K\hat{v}(S) \geq K\hat{v}(S^*) \\ &\geq \text{OPT} - nK = \text{OPT} - \epsilon V \geq (1 - \epsilon) \text{OPT}. \end{aligned}$$

Filtering ensures that the maximum-value item is itself feasible, so  $V \leq \text{OPT}$ . Keeping original weights preserves feasibility.

**Q10. Vertex form of Menger's theorem.** Split each internal vertex into  $v_{\text{in}} \rightarrow v_{\text{out}}$  of capacity 1, leaving  $x, y$  unsplit. Replace each undirected edge  $\{u, v\}$  by  $u_{\text{out}} \rightarrow v_{\text{in}}$  and  $v_{\text{out}} \rightarrow u_{\text{in}}$  of capacity  $M = n + 1$ , interpreting an endpoint  $x$  or  $y$  as the unsplit vertex. Internally disjoint paths induce a feasible integral flow. Conversely, decompose integral flow into unit  $x$ - $y$  paths and cycles and discard cycles; capacity-1 splitting arcs ensure internal vertex-disjointness.

A separator of size  $k$  deletes  $k$  splitting arcs and destroys all flow paths. Reachability after this deletion gives an original-network cut of capacity at most  $k$ . Because  $x, y$  are nonadjacent, all  $n - 2$  internal vertices form a separator, so a minimum cut has capacity at most  $n - 2 < M$ . Consequently it cuts no capacity- $M$  arc. Its outgoing splitting arcs identify a vertex separator whose size equals the cut capacity. Max-flow/min-cut now equates the two optima. Adjacency is excluded because a direct  $x$ - $y$  edge survives deletion of every internal vertex.

**Q11. Flow algorithms.** A two-arc path with capacities 3, 1 has maximum flow 1, but thresholds 3, 1.5 find no path before the flawed algorithm stops. Start instead at  $2^{\lfloor \log_2 C \rfloor}$  and halve through threshold 1; handle  $C = 0$  separately. Use only residual arcs of capacity at least the current threshold. For BFS augmentation, let  $d(v)$  be residual distance from  $s$  before an augmentation. Every surviving old arc satisfies  $d(v) \leq d(u) + 1$ . A new arc  $(v, u)$  reverses an arc of the chosen shortest path, so  $d(v) = d(u) + 1$ ; it too satisfies the inequality needed along any new path. Induction along a new residual path proves that no distance decreases. If  $(u, v)$  is saturated on a shortest path, then  $d(v) = d(u) + 1$ . Before it can be saturated in that direction again, an augmentation must use  $(v, u)$ ; at that later time

$$d_{\text{later}}(u) = d_{\text{later}}(v) + 1 \geq d_{\text{earlier}}(v) + 1 = d_{\text{earlier}}(u) + 2.$$

Thus each labelled residual arc can be critical only  $O(n)$  times. Each augmentation saturates one such arc, giving  $O(nm)$  augmentations and  $O(nm^2)$  time under the usual  $O(m)$  search bound.

With integer capacities, arbitrary Ford-Fulkerson increases value by at least 1, giving  $O(m|f^*|)$  time. The numeric value  $|f^*|$  may be exponential in its binary encoding length. With irrational capacities, there need be no positive lower bound on augmentation amounts; bad path choices can produce infinitely many positive augmentations. BFS avoids this dependence by bounding the number of augmentations combinatorially.

**Q12. Common subsequence.** For each symbol  $a$ , compute  $f_a = \min_i \text{count}_a(s_i)$ . Return  $a$  repeated  $f_a$  times for a symbol maximizing  $f_a$ ; this is always a common subsequence. Some symbol appears at least  $\text{OPT}/q$  times in an optimal common subsequence, and its  $f_a$  is at least that large. Alternatively,

$$\text{OPT} \leq \sum_a f_a \leq q \max_a f_a.$$

To achieve  $O(nL + q)$  even for variable  $q$ , initialize arrays once; while scanning each string, use timestamps to count only its occurring symbols. For each touched symbol update its minimum count and the number of strings in which it appears. Symbols appearing in fewer than  $n$  strings get  $f_a = 0$ . Each character and each touched symbol costs  $O(1)$ , followed by an  $O(q)$  final scan; no  $O(q)$  reset per string is needed.

## A.4 True/false answers

**T1. False.** Take  $q$  disjoint edges and one isolated vertex. Selecting all edge endpoints is a 2-approximate cover; its complement has size 1, while a maximum independent set has size  $q + 1$ .

**T2. False.** Use unit arcs  $s \rightarrow a, s \rightarrow b, a \rightarrow c, b \rightarrow c, c \rightarrow t$ . Sending 1/2 on each branch and 1 on  $c \rightarrow t$  is a fractional maximum flow. An integral maximum also exists.

**T3. True.** Weak duality gives  $|f| \leq |f^*| \leq c(S)$ ; equality of the outer values certifies maximum flow and minimum cut.

**T4. False.** Capacity  $C$  needs only  $O(\log(C + 1))$  bits. A bound polynomial in  $C$  may be exponential in input length.

**T5. False.** A satisfiable formula can have minimum unsatisfied cost 0. An assignment satisfying only half of its clauses may meet a MAX-SAT factor-2 guarantee while leaving positive cost, impossible for any finite multiplicative guarantee relative to 0.

**T6. False.** Use unit arcs along  $s \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow t$  and an additional arc  $s \rightarrow d$ . Initially  $d(s, c) = 3$ . Augment on the long path: the unused arc  $s \rightarrow d$  and new reverse arc  $d \rightarrow c$  give distance 2 to  $c$ .

**T7. False.** The optimal relaxation value is a *lower* bound. An arbitrary fractional feasible value need not bound the integral optimum in either direction. An integral feasible solution does give an upper bound.

**T8. False.**  $K_n$  witnesses a worst-case gap  $2 - 2/n$ : integral optimum  $n - 1$ , LP optimum  $n/2$ . This is not a statement about every graph; a single edge with isolated vertices has gap 1.

**T9. First true, second false.** An uncovered edge could be added to a matching, contradicting maximality. For one edge the maximum matching's endpoints give cover size 2, but minimum cover size 1.

**T10. False.** Linearity of expectation requires no independence.

## B Pseudo-Polynomial / Exponential DP

### B.1 0/1 Knapsack — $O(nW)$

$$DP[i, w] = \max \begin{cases} DP[i-1, w], & \text{skip } i, \\ DP[i-1, w-w_i] + v_i, & w_i \leq w \end{cases}$$

$$DP[0, w] = 0, \quad \text{OPT} = DP[n, W].$$

---

**Input:** Items  $(w_i, v_i)$ , capacity  $W$

$DP[0, \cdot] \leftarrow 0$

**for**  $i = 1, \dots, n$  **do**

**for**  $w = 0, \dots, W$  **do**

$DP[i, w] \leftarrow DP[i-1, w]$

**if**  $w_i \leq w$  **then**

$DP[i, w] \leftarrow \max(DP[i, w], DP[i-1, w-w_i] + v_i)$

**return**  $DP[n, W]$

---

### B.2 $d$ -Dimensional Knapsack — $O(n \prod_j W_j)$

Item  $i$  has value  $v_i$  and resource vector  $(w_{i1}, \dots, w_{id})$ .

$$DP[i, c_1, \dots, c_d] = \max \begin{cases} DP[i-1, c_1, \dots, c_d], \\ DP[i-1, c_1-w_{i1}, \dots, c_d-w_{id}] + v_i \end{cases}$$

where the second case is allowed iff

$$w_{ij} \leq c_j \quad \forall j \in [d].$$

---

**Input:** Items  $(v_i, w_{i1}, \dots, w_{id})$ , capacities  $(W_1, \dots, W_d)$

$DP[0, \cdot] \leftarrow 0$

**for**  $i = 1, \dots, n$  **do**

**foreach**  $(c_1, \dots, c_d) \in [0, W_1] \times \dots \times [0, W_d]$  **do**

$DP[i, \mathbf{c}] \leftarrow DP[i-1, \mathbf{c}]$

**if**  $\mathbf{w}_i \leq \mathbf{c}$  *componentwise* **then**

$DP[i, \mathbf{c}] \leftarrow \max\{DP[i, \mathbf{c}], DP[i-1, \mathbf{c}-\mathbf{w}_i] + v_i\}$

**return**  $DP[n, W_1, \dots, W_d]$

---

### B.3 Subset Sum — $O(nB)$

$$DP[i, s] = DP[i-1, s] \vee (s \geq a_i \wedge DP[i-1, s-a_i]).$$

$$DP[0, 0] = \text{True}, \quad DP[0, s > 0] = \text{False}.$$

---

**Input:**  $a_1, \dots, a_n$ , target  $B$

$DP[0, 0] \leftarrow \text{True}$

**for**  $i = 1, \dots, n$  **do**

**for**  $s = 0, \dots, B$  **do**

$DP[i, s] \leftarrow DP[i-1, s]$

**if**  $s \geq a_i$  **then**

$DP[i, s] \leftarrow DP[i, s] \vee DP[i-1, s-a_i]$

**return**  $DP[n, B]$

---

B.4 Partition —  $O(nS)$

Let

$$S = \sum_{i=1}^n a_i.$$
$$DP[i, s] = DP[i - 1, s] \vee (s \geq a_i \wedge DP[i - 1, s - a_i]).$$
$$\text{OPT} = \min_{\substack{0 \leq s \leq S \\ DP[n, s]}} |S - 2s|.$$

---

**Input:**  $a_1, \dots, a_n$   
 $S \leftarrow \sum_i a_i$   
compute Subset-Sum DP up to  $S$   
**return**  $\min_{DP[n, s]} |S - 2s|$

---

B.5 TSP: Held–Karp —  $O(n^2 2^n)$

Fix start vertex 0.

$$DP[S, v] = \text{minimum cost path } 0 \rightsquigarrow v \text{ visiting exactly } S,$$

where  $0, v \in S$ .

$$DP[\{0\}, 0] = 0.$$
$$DP[S, v] = \min_{u \in S \setminus \{v\}} \{DP[S \setminus \{v\}, u] + d(u, v)\}.$$
$$\text{OPT} = \min_{v \neq 0} \{DP[V, v] + d(v, 0)\}.$$

---

**Input:** Complete weighted graph  $G = (V, E)$   
 $DP[\{0\}, 0] \leftarrow 0$   
**foreach**  $S \subseteq V$  *with*  $0 \in S$  **do**  
    **foreach**  $v \in S \setminus \{0\}$  **do**  
         $DP[S, v] \leftarrow \min_{u \in S \setminus \{v\}} \{DP[S \setminus \{v\}, u] + d(u, v)\}$   
**return**  $\min_{v \neq 0} \{DP[V, v] + d(v, 0)\}$

---

B.6 Minimum Vertex Cover —  $O^*(2^n)$  Memoized DP

For  $S \subseteq V$ , let  $VC(S)$  be the minimum vertex cover size of  $G[S]$ .

$$VC(S) = \begin{cases} 0, & E[S] = \emptyset, \\ 1 + \min\{VC(S \setminus \{u\}), VC(S \setminus \{v\})\}, & (u, v) \in E[S]. \end{cases}$$

---

**Function**  $\text{VC}(S)$ :  
    **if**  $E[S] = \emptyset$  **then**  
        **return** 0  
    **if**  $\text{memo}[S]$  *exists* **then**  
        **return**  $\text{memo}[S]$   
    choose any  $(u, v) \in E[S]$   
     $\text{memo}[S] \leftarrow 1 + \min\{\text{VC}(S \setminus \{u\}), \text{VC}(S \setminus \{v\})\}$   
    **return**  $\text{memo}[S]$

---

B.7 Maximum Independent Set —  $O^*(2^n)$

For  $S \subseteq V$ :

$$MIS(S) = \begin{cases} 0, & S = \emptyset, \\ \max\{MIS(S \setminus \{v\}), 1 + MIS(S \setminus N[v])\}, & v \in S. \end{cases}$$

---

**Function**  $\text{MIS}(S)$ :  
    **if**  $S = \emptyset$  **then**  
        **return** 0  
    choose  $v \in S$   
    **return**  $\max\{\text{MIS}(S \setminus \{v\}), 1 + \text{MIS}(S \setminus N[v])\}$

---

$$\tau(G) + \alpha(G) = |V| \implies \text{MinVC}(G) = |V| - \text{MIS}(V).$$

B.8 Shortest Superstring —  $O(n^2 2^n)$

Let

$$ov(i, j) = \text{maximum suffix/prefix overlap of } s_i, s_j,$$
$$c(i, j) = |s_j| - ov(i, j).$$

State:

$$DP[S, j] = \text{minimum length of a superstring containing } S \text{ and ending at } j.$$

Base:

$$DP[\{j\}, j] = |s_j|.$$

Recurrence:

$$DP[S, j] = \min_{i \in S \setminus \{j\}} \{DP[S \setminus \{j\}, i] + c(i, j)\}.$$
$$\text{OPT} = \min_j DP[[n], j].$$

---

compute  $ov(i, j)$  and  $c(i, j)$  for all  $i, j$   
**for**  $j = 1, \dots, n$  **do**  
     $DP[\{j\}, j] \leftarrow |s_j|$   
**foreach**  $S \subseteq [n]$  **do**  
    **foreach**  $j \in S$  **do**  
         $DP[S, j] \leftarrow \min_{i \in S \setminus \{j\}} \{DP[S \setminus \{j\}, i] + c(i, j)\}$   
**return**  $\min_j DP[[n], j]$

---

B.9 Set Cover —  $O(m 2^n)$

Universe  $U = \{1, \dots, n\}$  and sets  $S_1, \dots, S_m$ .

$$DP[X] = \text{minimum number of sets covering } X.$$
$$DP[\emptyset] = 0.$$
$$DP[X] = 1 + \min_{i \in [m]} DP[X \setminus S_i].$$

Equivalently:

$$DP[X \cup S_i] = \min\{DP[X \cup S_i], DP[X] + 1\}.$$

---

$DP[\emptyset] \leftarrow 0$   
 $DP[X] \leftarrow \infty$  for  $X \neq \emptyset$   
**foreach**  $X \subseteq U$  **do**  
    **for**  $i = 1, \dots, m$  **do**  
         $DP[X \cup S_i] \leftarrow \min\{DP[X \cup S_i], DP[X] + 1\}$   
**return**  $DP[U]$

---

B.10 Weighted Set Cover —  $O(m 2^n)$

$$DP[\emptyset] = 0,$$
$$DP[X \cup S_i] = \min\{DP[X \cup S_i], DP[X] + w_i\}.$$

---

$DP[\emptyset] \leftarrow 0$   
 $DP[X] \leftarrow \infty$  for  $X \neq \emptyset$   
**foreach**  $X \subseteq U$  **do**  
    **for**  $i = 1, \dots, m$  **do**  
         $DP[X \cup S_i] \leftarrow \min\{DP[X \cup S_i], DP[X] + w_i\}$   
**return**  $DP[U]$

---

B.11 Minimum Graph Coloring —  $O^*(3^n)$  Simple Subset DP

Let  $\mathcal{I}(S)$  denote whether  $S$  is an independent set.

$$\chi(\emptyset) = 0.$$
$$\chi(S) = 1 + \min_{\substack{I \subseteq S \\ I \neq \emptyset \\ \mathcal{I}(I)}} \chi(S \setminus I).$$

---

$DP[\emptyset] \leftarrow 0$   
**foreach** *nonempty*  $S \subseteq V$  **do**  
     $DP[S] \leftarrow \infty$   
    **foreach** *nonempty independent*  $I \subseteq S$  **do**  
         $DP[S] \leftarrow \min\{DP[S], 1 + DP[S \setminus I]\}$   
**return**  $DP[V]$

---

## B.12 Bin Packing — $O(n2^n)$

Capacity  $B$ . State stores

$$DP[S] = (b, r)$$

where  $b$  is minimum number of bins and  $r$  is used capacity in the last bin, compared lexicographically.

$$DP[\emptyset] = (1, 0).$$

For  $i \in S$ , let

$$(b, r) = DP[S \setminus \{i\}].$$

$$C_i = \begin{cases} (b, r + w_i), & r + w_i \leq B, \\ (b + 1, w_i), & r + w_i > B. \end{cases}$$

$$DP[S] = \min_{i \in S} C_i.$$

---

```

DP[∅] ← (1, 0)
foreach nonempty  $S \subseteq [n]$  do
  DP[S] ← (∞, ∞)
  foreach  $i \in S$  do
     $(b, r) \leftarrow DP[S \setminus \{i\}]$ 
    if  $r + w_i \leq B$  then
      |  $cand \leftarrow (b, r + w_i)$ 
    else
      |  $cand \leftarrow (b + 1, w_i)$ 
    DP[S] ← minlex{DP[S], cand}
return DP[[n]].b

```

---

## B.13 Hamiltonian Path — $O(n^2 2^n)$

$$DP[S, v] = \begin{cases} \text{True}, & \exists \text{ path containing exactly } S \text{ and ending at } v, \\ \text{False}, & \text{otherwise.} \end{cases}$$

$$DP[\{v\}, v] = \text{True}.$$

$$DP[S, v] = \bigvee_{\substack{u \in S \setminus \{v\} \\ (u, v) \in E}} DP[S \setminus \{v\}, u].$$

---

```

for  $v \in V$  do
  DP[{v}, v] ← True
foreach  $S \subseteq V$  do
  foreach  $v \in S$  do
    DP[S, v] ←  $\bigvee_{\substack{u \in S \setminus \{v\} \\ (u, v) \in E}} DP[S \setminus \{v\}, u]$ 
return  $\bigvee_{v \in V} DP[V, v]$ 

```

---

## B.14 Steiner Tree — $O(3^k n + 2^k n^2)$

Let terminals be  $R$ ,  $|R| = k$ .

$DP[S, v]$  = minimum cost tree connecting terminals  $S$  and containing  $v$ .

Base:

$$DP[\{r\}, v] = d(r, v).$$

Merge:

$$DP[S, v] = \min_{\emptyset \neq A \subsetneq S} \{DP[A, v] + DP[S \setminus A, v]\}.$$

Shortest-path closure:

$$DP[S, v] = \min_{u \in V} \{DP[S, u] + d(u, v)\}.$$

$$\text{OPT} = \min_{v \in V} DP[R, v].$$

---

```

foreach  $r \in R$  do
  foreach  $v \in V$  do
    DP[{r}, v] ← d(r, v)
foreach  $S \subseteq R, |S| \geq 2$  do
  foreach  $v \in V$  do
    DP[S, v] ← min_{∅ ≠ A ⊂ S} {DP[A, v] + DP[S \ A, v]}
    foreach  $v \in V$  do
      DP[S, v] ← min_{u ∈ V} {DP[S, u] + d(u, v)}
return min_v DP[R, v]

```

---

## C Background and Prerequisites

### C.1 The optimization viewpoint

An optimization problem separates the description of legal solutions from the criterion used to rank them.

**Definition C.1** (Optimization problem). For an instance  $I$ , let  $F(I)$  be its feasible set and let  $c_I: F(I) \rightarrow \mathbb{R}$  be its objective. A minimization problem asks for

$$\text{OPT}(I) = \min_{x \in F(I)} c_I(x),$$

while a maximization problem replaces min by max. A complete answer may include both an optimal value and a witness  $x^*$  attaining that value.

The same problem often appears in three forms:

- **decision:** is there a feasible solution of value at most  $B$  (minimization), or at least  $B$  (maximization)?
- **search:** find any feasible solution, possibly satisfying a threshold;
- **optimization:** find the best feasible solution.

For an integer-valued objective known to lie in  $[L, U]$ , an optimization value can often be recovered using  $O(\log(U - L + 1))$  calls to the decision problem. This is a polynomial-time reduction when the endpoints have polynomial bit length; the bound and encoding must be stated rather than silently assumed.

**A standard model.** Many problems can be written as

$$\min\{c^\top x : x \in F\},$$

where  $x$  records choices,  $F$  records constraints, and  $c$  records costs. The domain of  $x$  matters:  $x \in \{0, 1\}^n$  is discrete, whereas  $x \in [0, 1]^n$  is a continuous relaxation. Enlarging  $F$  can make a problem easier to solve, but may produce a solution that is not feasible for the original problem.

### C.2 Asymptotics, input size, and cost models

For eventually non-negative functions,

$$f \in O(g) \iff \exists C > 0, \exists n_0, \forall n \geq n_0 : f(n) \leq Cg(n),$$

$$f \in \Omega(g) \iff \exists c > 0, \exists n_0, \forall n \geq n_0 : f(n) \geq cg(n),$$

$$f \in \Theta(g) \iff f \in O(g) \cap \Omega(g).$$

Useful limit tests are

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f \in o(g), \quad 0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty \Rightarrow f \in \Theta(g).$$

The robust versions use lim sup and lim inf when the ratio oscillates.

**Bit complexity.** Running time is measured against the length of the encoded input, not merely the numerical value of its parameters. An integer  $W$  requires  $\Theta(\log(W + 1))$  bits in binary. Consequently, a running time such as  $O(nW)$  is polynomial in the *value*  $W$  but can be exponential in its bit length. Such an algorithm is **pseudo-polynomial**. This distinction is central for knapsack-style dynamic programs and many weakly NP-hard optimization problems.

**Cost accounting.** For every claimed bound, identify:

1. the input-size parameters and their encodings;
  2. the number of iterations, states, or recursive calls;
  3. the cost of each operation or oracle call; and
  4. any hidden dependence on numerical precision.
- A recurrence such as  $T(n) = aT(n/b) + f(n)$  is appropriate for recursive decomposition. Iterative optimization algorithms are often clearer as

$$\text{preprocessing cost} + (\text{number of iterations})(\text{cost per iteration}).$$

### C.3 Proof obligations for algorithms

The loop-invariant template from CS3230 remains the default proof pattern: *initialization*, *maintenance*, and *termination*. For a recursive algorithm, replace the invariant by induction on the instance size. For optimization, separate the claims that are easy to accidentally conflate.

1. **Feasibility:** the returned solution lies in  $F(I)$ .
2. **Value guarantee:** its objective satisfies the promised bound.
3. **Optimality or approximation:** compare it with every feasible solution, a lower/upper bound, or an optimal witness.
4. **Termination and complexity:** the procedure finishes within the claimed resources under the stated computational model.

For a non-negative objective and an  $\alpha$ -approximation with  $\alpha \geq 1$ , the usual conventions are

$$\text{minimization: } \text{OPT}(I) \leq \text{ALG}(I) \leq \alpha \text{OPT}(I),$$

$$\text{maximization: } \text{OPT}(I)/\alpha \leq \text{ALG}(I) \leq \text{OPT}(I).$$

Feasibility gives one side of each inequality; the real proof is the other side. A common strategy is to compare the algorithm against a certificate bound  $L(I) \leq \text{OPT}(I)$  for minimization or  $U(I) \geq \text{OPT}(I)$  for maximization.

### Reusable proof patterns.

- **Exchange argument:** transform an optimum so that it agrees with the greedy choice without worsening its value.
- **Optimal substructure:** after fixing a decision, the remainder is an optimal solution to a well-defined subproblem.
- **Charging argument:** charge the cost of the computed solution to parts of an optimum, with bounded charge per part.
- **Relaxation bound:** compare an integral solution with the value of a relaxation whose feasible region contains all integral solutions.

## C.4 Algorithm-design tools retained from CS3230

**Dynamic programming.** A DP proof should state the subproblem, recurrence, base cases, evaluation order, and recovery of a witness. If  $D[s]$  denotes the best value for state  $s$ , the key equality is justified by partitioning all feasible solutions by their last decision:

$$D[s] = \underset{a \in A(s)}{\text{opt}} \{ \text{cost}(s, a) + D[\text{next}(s, a)] \}.$$

The running time is the number of reachable states times the transition cost. State coordinates that range up to a numeric value such as a capacity  $W$  are where pseudo-polynomial behaviour usually enters.

**Greedy algorithms.** A locally best choice is not a proof. Correctness normally requires an exchange argument, a stays-ahead invariant, or a structural property such as a cut property. When no such proof exists, the same greedy rule may still be useful as a heuristic or approximation algorithm, but its guarantee must be analysed separately.

**Graph subroutines.** Shortest paths, minimum spanning trees, maximum flow, matching, and connectivity recur as exact subroutines and as sources of relaxations. Always record their preconditions: for example, Dijkstra’s algorithm requires non-negative edge weights, whereas Bellman–Ford permits negative edges but must detect reachable negative cycles.

## C.5 Probability toolkit for randomized optimization

Let  $X_1, \dots, X_n$  be random variables and let  $X = \sum_i X_i$ . The most reusable facts from CS3230 are

$$\mathbb{E}[X] = \sum_i \mathbb{E}[X_i] \quad (\text{no independence required}),$$

$$\Pr \left[ \bigcup_i A_i \right] \leq \sum_i \Pr[A_i] \quad (\text{union bound}),$$

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t} \quad \text{for } X \geq 0 \text{ (Markov).}$$

Indicator variables  $X_i = \mathbf{1}[A_i]$  turn counting questions into expectations because  $\mathbb{E}[X_i] = \Pr[A_i]$ . For independent Bernoulli variables with  $\mu = \mathbb{E}[X]$  and  $0 < \delta < 1$ , useful Chernoff forms are

$$\Pr[X \leq (1 - \delta)\mu] \leq e^{-\delta^2 \mu / 2}, \quad \Pr[X \geq (1 + \delta)\mu] \leq e^{-\delta^2 \mu / (2 + \delta)}.$$

These concentration inequalities require stronger assumptions than linearity of expectation; independence must be checked rather than silently assumed. If one independent run fails with probability at most  $q < 1$ , then  $t$  runs all fail with probability at most  $q^t$ . This is the amplification principle seen in Freivalds’ algorithm. In optimization it also supports randomized rounding: solve a fractional relaxation, interpret fractional coordinates as selection probabilities, and prove objective and feasibility bounds using expectation plus concentration.

## C.6 Logic and constraint modelling from CS4269

A propositional formula has *syntax*; a valuation gives it *semantics*. For a valuation  $v$ ,

- $v \models \phi$  means that  $v$  satisfies  $\phi$ ;
- $\phi$  is satisfiable if some  $v$  satisfies it;
- $\phi$  is valid if every  $v$  satisfies it;
- $\Gamma \models \phi$  means every model of  $\Gamma$  is a model of  $\phi$ ; and
- $\phi \equiv \psi$  means the two formulas have the same models.

The distinction between satisfiability and validity matters: a feasible solution corresponds to *one* satisfying assignment, not to a formula that is true under all assignments.

**Constraint-encoding recipe.** The graph-colouring, Hamiltonian-path, clique, exact-cover, and scheduling encodings in CS4269 share one pattern:

1. introduce a variable for each atomic choice;
2. write separate constraints for existence, uniqueness, compatibility, and global requirements;
3. conjoin the constraints into one formula;
4. prove a two-way correspondence between models and feasible solutions;
5. prove that the encoding has polynomial size and is computable in polynomial time.

For variables  $p_1, \dots, p_k$ , “exactly one” is

$$\left( \bigvee_{i=1}^k p_i \right) \wedge \left( \bigwedge_{1 \leq i < j \leq k} (\neg p_i \vee \neg p_j) \right).$$

The first part gives existence; the pairwise clauses give uniqueness.

## C.7 From Boolean constraints to binary optimization

Set  $x_i = 1$  when proposition  $p_i$  is true and  $x_i = 0$  otherwise. Common logical constraints then become linear inequalities:

| Boolean constraint                     | 0–1 linear form                                                            |
|----------------------------------------|----------------------------------------------------------------------------|
| $p_1 \vee \dots \vee p_k$              | $x_1 + \dots + x_k \geq 1$                                                 |
| $p \rightarrow q$                      | $x_p \leq x_q$                                                             |
| exactly one of $p_1, \dots, p_k$       | $x_1 + \dots + x_k = 1$                                                    |
| $z \leftrightarrow (p \wedge q)$       | $z \leq x_p, z \leq x_q,$<br>$z \geq x_p + x_q - 1$                        |
| clause $\ell_1 \vee \dots \vee \ell_k$ | $\sum_{\ell_i = p_j} x_j$<br>$+ \sum_{\ell_i = \neg p_j} (1 - x_j) \geq 1$ |

**Example: minimum vertex cover.** For a graph  $G = (V, E)$ , let  $x_v$  indicate whether vertex  $v$  is selected:

$$\begin{aligned} \min \quad & \sum_{v \in V} x_v \\ \text{subject to} \quad & x_u + x_v \geq 1 \quad \forall \{u, v\} \in E, \\ & x_v \in \{0, 1\}. \end{aligned}$$

Each edge constraint is the clause  $p_u \vee p_v$ . Replacing  $x_v \in \{0, 1\}$  by  $0 \leq x_v \leq 1$  gives a linear relaxation. Its feasible region is larger, so for this minimization problem its optimum is a lower bound on the integral optimum.

## C.8 Complexity classes and reductions

**Definition C.2** (Core classes). **P** is the class of decision problems solvable in polynomial time. **NP** is the class whose YES instances have polynomial-size certificates verifiable in polynomial time. A problem  $B$  is NP-hard if every problem in **NP** polynomial-time reduces to  $B$ ; it is NP-complete if, in addition,  $B \in \text{NP}$ .

A polynomial-time many-one reduction  $A \leq_p B$  is a polynomial-time computable map  $f$  such that

$$x \in A \iff f(x) \in B.$$

The direction has an operational meaning: an algorithm for  $B$  would yield an algorithm for  $A$ . To prove that  $B$  is hard, reduce a known hard problem  $t$  to  $B$ , not the other way around.

Every reduction proof has three obligations:

1. **construction:** define  $f(x)$  completely;
2. **correctness:** prove both directions of  $x \in A \iff f(x) \in B$ ; and
3. **complexity:** prove that  $f$  runs in polynomial time and its output has polynomial encoded size.

The SAT encodings from CS4269 are reductions when they satisfy exactly these obligations. Cook–Levin supplies the central starting point: SAT is NP-complete.

**Decision versus optimization hardness.** NP-completeness formally applies to decision problems. To discuss an optimization problem, define its threshold version. For example, the decision version of minimum vertex cover asks whether a cover of size at most  $k$  exists. If an exact optimizer ran in polynomial time, evaluating whether its returned value is at most  $k$  would solve this decision problem in polynomial time. Thus NP-hardness of the threshold problem implies NP-hardness of exact optimization.

## C.9 A checklist for the rest of CS4234

For each new problem or algorithm, record the following before doing algebra:

1. What is the instance, and what is its encoded length?
2. What are the variables, feasible solutions, and objective direction?
3. Is the task decision, search, or optimization?
4. What invariant, exchange, induction, charging, or relaxation argument proves the guarantee?
5. Is the guarantee exact, approximate, expected, or with high probability?
6. Does the runtime depend polynomially on the bit length, or only on a numerical value?
7. For a reduction, is the direction correct, is the equivalence proved both ways, and is the construction polynomial in encoded size?

## D Extended Probability and Mathematical Reference

### D.1 Basic Probability Identities

$$\Pr(A \cup B) \leq \Pr(A) + \Pr(B) \quad (\text{Union bound})$$

Inclusion–Exclusion (2 terms):

$$\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B).$$

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}, \quad A \perp B \iff \Pr(A \cap B) = \Pr(A) \Pr(B)$$

Law of Total Probability, for a partition  $\{E_i\}$ :

$$\Pr(A) = \sum_i \Pr(A \mid E_i) \Pr(E_i).$$

$$\text{Bayes: } \Pr(H \mid D) = \frac{\Pr(D \mid H) \Pr(H)}{\Pr(D)}$$

### D.2 Indicator Trick & Linearity

For indicator  $I_E = \mathbf{1}\{E\}$ :  $\mathbb{E}[I_E] = \Pr(E)$ . For a sum  $X = \sum_{i=1}^n I_i$  (*counting something*):

$$\mathbb{E}[X] = \sum_{i=1}^n \Pr(I_i = 1) \quad (\text{no independence needed})$$

If  $I_i$  are pairwise independent:

$$\text{Var}(X) = \sum_{i=1}^n \text{Var}(I_i) = \sum_i p_i(1 - p_i)$$

### D.3 Expectation/Variance Identities

$$\mathbb{E}[aX + b] = a \mathbb{E}[X] + b \quad (\text{Linearity})$$

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]] \quad (\text{Law of total expectation})$$

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

$$\text{Var}(X) = \mathbb{E}[\text{Var}(X \mid Y)] + \text{Var}(\mathbb{E}[X \mid Y]) \quad (\text{Law of total variance})$$

$$\text{Tail-sum (nonneg. integer } X): \mathbb{E}[X] = \sum_{t \geq 1} \Pr[X \geq t]$$

### D.4 Inequalities (Concentration)

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a} \quad (\text{Markov})$$

$$\Pr(|X - \mu| \geq t) \leq \frac{\text{Var}(X)}{t^2} \quad (\text{Chebyshev})$$

**Chernoff/Hoeffding for sums of bounded/0–1 r.v.’s.** Let  $X = \sum_{i=1}^n X_i$ ,  $X_i \in [0, 1]$  independent,  $\mu = \mathbb{E}[X]$ .

$$\Pr[X \geq (1 + \delta)\mu] \leq \exp\left(-\frac{\mu \delta^2}{2 + \delta}\right) \quad (\delta > 0)$$

$$\Pr[X \leq (1 - \delta)\mu] \leq \exp\left(-\frac{\mu \delta^2}{2}\right) \quad (0 < \delta < 1)$$

$$\Pr(|X - \mu| \geq t) \leq 2 \exp\left(-\frac{t^2}{n}\right) \quad (\text{Hoeffding, if } X_i \in [0, 1])$$

*Handy simplification* for  $0 < \delta \leq 1$ :  $\Pr(|X - \mu| \geq \delta\mu) \leq 2 \exp(-\mu\delta^2/3)$ .

## D.5 Common Inequalities for Mathematical Proofs

Many mathematical arguments rely on a small set of fundamental inequalities. Below is a unified chain collecting the most frequently used algebraic, geometric, convexity, and analytic inequalities. Each tool is stated in a form that is easy to substitute directly into proofs.

### Master Chain of Useful Inequalities.

#### (1) Basic Algebraic Inequalities

$$a \leq b \Rightarrow a + c \leq b + c, \quad a \leq b, \ c \geq 0 \Rightarrow ac \leq bc.$$

$$(a - b)^2 \geq 0 \Rightarrow 2ab \leq a^2 + b^2 \Rightarrow |ab| \leq \frac{a^2 + b^2}{2}.$$

#### (2) AM–GM–HM Hierarchy

For all  $a, b > 0$ ,

$$\frac{a + b}{2} \geq \sqrt{ab} \geq \frac{2}{\frac{1}{a} + \frac{1}{b}}.$$

#### (3) Cauchy–Schwarz Inequality

$$\left(\sum_{i=1}^n a_i b_i\right)^2 \leq \left(\sum_{i=1}^n a_i^2\right) \left(\sum_{i=1}^n b_i^2\right).$$

#### (4) Triangle Inequality

$$|x + y| \leq |x| + |y|, \quad \|u + v\| \leq \|u\| + \|v\|.$$

#### (5) Jensen’s Inequality (Convex Functions)

If  $f$  is convex and  $\lambda_i \geq 0$ ,  $\sum \lambda_i = 1$ ,

$$f\left(\sum_{i=1}^n \lambda_i x_i\right) \leq \sum_{i=1}^n \lambda_i f(x_i).$$

#### (6) Bernoulli’s Inequality

For  $x \geq -1$ ,

$$(1 + x)^r \geq 1 + rx \quad (r \geq 1 \text{ or } r \leq 0).$$

#### (7) Weighted AM–GM

If  $a_i > 0$ ,  $\alpha_i \geq 0$ ,  $\sum \alpha_i = 1$ ,

$$a_1^{\alpha_1} a_2^{\alpha_2} \cdots a_n^{\alpha_n} \leq \alpha_1 a_1 + \alpha_2 a_2 + \cdots + \alpha_n a_n.$$

#### (8) Young’s Inequality

For  $p, q > 1$ ,  $\frac{1}{p} + \frac{1}{q} = 1$ ,

$$ab \leq \frac{a^p}{p} + \frac{b^q}{q}.$$

#### (9) Hölder’s Inequality

$$\|fg\|_1 \leq \|f\|_p \|g\|_q, \quad p, q > 1, \quad \frac{1}{p} + \frac{1}{q} = 1.$$

#### (10) Logarithmic Inequalities

$$\ln(1 + x) \leq x, \quad 1 + x \leq e^x, \quad \ln x \leq x - 1.$$

#### (11) Exponential Bounds

$$1 + x \leq e^x \leq 1 + x + \frac{x^2}{2} e^{|x|}.$$

#### (12) Chebyshev’s Sum Inequality

For similarly sorted sequences,

$$\frac{1}{n} \sum_{i=1}^n a_i b_i \geq \left(\frac{1}{n} \sum_{i=1}^n a_i\right) \left(\frac{1}{n} \sum_{i=1}^n b_i\right).$$

#### (13) Rearrangement Inequality

For sorted  $a_1 \leq \cdots \leq a_n$ ,

$$\sum_{i=1}^n a_i b_i \leq \sum_{i=1}^n a_i b_{\sigma(i)} \leq \sum_{i=1}^n a_i b_i^\downarrow.$$

(14) Markov and Chebyshev Bounds

Pr[X ≥ a] ≤ E[X] / a,      Pr(|X − μ| ≥ t) ≤ Var(X) / t^2.

(15) Jensen + Logarithm = AM-GM

ln((a1 + ... + an) / n) ≥ 1/n ∑ ln ai.

- Notes.
- Inequalities (1)–(4) handle almost all algebraic manipulations in proofs.
  - (5)–(8) come from convexity and are widely used in optimization and analysis.
  - (9)–(13) generalize inner-product arguments and are essential in analysis.
  - (14)–(15) appear frequently in algorithm analysis and probability arguments.

D.6 Common Taylor Series Expansions

In this subsection we list the most frequently used Taylor (and Maclaurin) series expansions. Unless stated otherwise, all expansions are Maclaurin series (expansion at x = 0) and hold for sufficiently small |x|.

(1) Exponential Function

e^x = 1 + x + x^2/2! + x^3/3! + ... = ∑\_{n=0}^∞ x^n/n!

Useful bounds:

1 + x ≤ e^x ≤ 1 + x + x^2/2 e^{|x|}.

(2) Natural Logarithm

For |x| < 1,

ln(1 + x) = x − x^2/2 + x^3/3 − x^4/4 + ... = ∑\_{n=1}^∞ (−1)^{n+1} x^n/n.

Special asymptotic form:

ln(1 + x) = x − x^2/2 + O(x^3).

(3) Geometric Series

For |x| < 1,

1/(1 − x) = 1 + x + x^2 + x^3 + ... .

(4) Binomial Series

For any real α, and |x| < 1,

(1 + x)^α = ∑\_{n=0}^∞ (α choose n) x^n  
= 1 + αx + α(α − 1)/2 x^2  
+ α(α − 1)(α − 2)/6 x^3 + ... .

(5) Sine Function

sin x = x − x^3/3! + x^5/5! − ... = ∑\_{n=0}^∞ (−1)^n x^{2n+1}/(2n + 1)!

(6) Cosine Function

cos x = 1 − x^2/2! + x^4/4! − ... = ∑\_{n=0}^∞ (−1)^n x^{2n}/(2n)!

(7) Tangent (Derivative at 0 and Series)

Though more rarely needed, the Maclaurin series is:

tan x = x + x^3/3 + 2x^5/15 + 17x^7/315 + ... .

Radius of convergence: π/2.

(8) Inverse Tangent

For |x| ≤ 1,

arctan x = x − x^3/3 + x^5/5 − ... = ∑\_{n=0}^∞ (−1)^n x^{2n+1}/(2n + 1).

(9) Hyperbolic Sine and Cosine

sinh x = x + x^3/3! + x^5/5! + ... = ∑\_{n=0}^∞ x^{2n+1}/(2n + 1)!

cosh x = 1 + x^2/2! + x^4/4! + ... = ∑\_{n=0}^∞ x^{2n}/(2n)!

(10) Square Root Expansion

For |x| < 1,

√(1 + x) = 1 + x/2 − x^2/8 + x^3/16 − ... .

(11) Reciprocal

For |x| < 1,

1/(1 + x) = 1 − x + x^2 − x^3 + ... .

(12) Useful Asymptotic Forms

e^x = 1 + x + O(x^2),      ln(1 + x) = x − x^2/2 + O(x^3),  
sin x = x + O(x^3),      cos x = 1 − x^2/2 + O(x^4).

D.7 Common Distributions (means/vars)

Bernoulli(p) : E[X] = p, Var(X) = p(1 − p)  
Binomial(n, p) : E[X] = np, Var(X) = np(1 − p)

Geometric(p): trials to first success; memoryless.

E[X] = 1/p,      Var(X) = 1/p^2.

Poisson(λ):

E[X] = λ,      Var(X) = λ.

Binomial ≈ Poisson when n → ∞, p → 0, np = λ.

Exponential(λ) : E[X] = 1/λ, Var(X) = 1/λ^2, memoryless

Normal N(μ, σ^2) (CLT for sums):

Pr((X − μ)/σ ∈ [a, b]) ≈ Φ(b) − Φ(a).

D.8 Asymptotic Approximations

H\_n = ∑\_{k=1}^n 1/k = ln n + γ + O(1/n)  
n! ~ √(2πn) (n/e)^n (Stirling),  
(n choose k) ≈ n^n / (k^k (n − k)^{n−k} √(2πk(n − k))).

D.9 Classic “Balls-into-Bins” Facts

Throw m balls into n bins uniformly and independently.

E[occupied bins] = n (1 − (1 − 1/n)^m) ≈ n (1 − e^{−m/n})

Birthday paradox:

Pr(no collisions with m balls) ≈ exp(−m(m − 1)/2n).

For m = n, the maximum load is Θ(log n / log log n) w.h.p.

D.10 Coupon Collector

Collect n distinct coupons, each trial draws one uniformly at random.

E[T] = n H\_n ≈ n ln n + γn ; Pr[T > n(ln n + c)] ≤ e^{−c}

D.11 Conditioning & Decomposition

E[X] = \sum\_i E[X | E\_i] Pr(E\_i),

Var(X) = E[Var(X | Y)] + Var(E[X | Y]).

Wald’s identity (independent X\_i with E[X\_i] = μ, and stopping time N independent of X\_i):

E[\sum\_{i=1}^N X\_i] = E[N] \cdot \mu

D.12 Jensen & Convexity

For convex g: E[g(X)] ≥ g(E[X]). Frequently used with g(x) = e^{tx} (mgf) in Chernoff-style proofs.

D.13 “With High Probability” (w.h.p.)

A statement holds w.h.p. if Pr[event] ≥ 1 − n^{−c} for some constant c > 0. Typical pattern: apply a union bound over n events after a Chernoff/Hoeffding bound on each.

(i) **Linearity** never needs independence; **variance additivity** does. (ii) State the range/independence assumptions before using Chernoff/Hoeffding. (iii) For hashing/“pairwise independence,” Chernoff may fail; use Chebyshev or limited-independence bounds.

E Discrete Mathematics Reference

Focus: primes, modular arithmetic, and base conversion/manipulation. Short statements you can drop into proofs or code comments.

E.1 Divisibility, GCD, and Bézout

- **Bézout’s identity.** For a, b ∈ Z, ∃ x, y ∈ Z with ax + by = gcd(a, b). Use: Extended Euclid; modular inverses when gcd(a, m) = 1.
- **LCM–GCD relation.** lcm(a, b) · gcd(a, b) = |ab|.
- **Euclidean algorithm.** gcd(a, b) = gcd(b, a mod b), terminates in O(log min{a, b}) steps.

E.2 Modular Arithmetic & Inverses

- **Mod inverse existence.** a^{−1} (mod m) exists ⇔ gcd(a, m) = 1; compute via Extended Euclid.
- **Fermat’s little theorem (FLT).** If p is prime and p ∤ a, then a^{p−1} ≡ 1 (mod p); hence a^{−1} ≡ a^{p−2} (mod p).
- **Euler’s theorem.** If gcd(a, m) = 1, then a^{φ(m)} ≡ 1 (mod m). φ is multiplicative and φ(p^k) = p^k − p^{k−1}.
- **Exponent reduction.** For gcd(a, m) = 1, a^b mod m can be computed with b mod φ(m) (careful with b = 0 and lifting when gcd(a, m) ≠ 1).
- **Wilson’s theorem.** p prime ⇔ (p − 1)! ≡ −1 (mod p).

E.3 Chinese Remainder Theorem (CRT)

- **Pairwise coprime moduli.** For x ≡ a\_i (mod m\_i), i = 1..k, with pairwise coprime m\_i and M = ∏ m\_i, the unique solution modulo M is

x ≡ \sum\_{i=1}^k a\_i M\_i \overline{M\_i} \pmod{M},

M\_i = M/m\_i, \quad M\_i \overline{M\_i} \equiv 1 \pmod{m\_i}.

- **General (two congruences).** x ≡ a (mod m), x ≡ b (mod n) has a solution ⇔ a ≡ b (mod g) where g = gcd(m, n). When solvable, solutions form a class modulo lcm(m, n); one solution is

x \equiv a + m \cdot \left( \frac{b - a}{g} \cdot \overline{m/g} \right) \pmod{\text{lcm}(m, n)}.

E.4 Prime Factorization Facts

- **Multiplicativity.** If gcd(m, n) = 1, then τ(mn) = τ(m)τ(n) and σ(mn) = σ(m)σ(n), where

\tau(n) = \prod\_i (e\_i + 1), \quad \sigma(n) = \prod\_i \frac{p\_i^{e\_i+1} - 1}{p\_i - 1} \quad \text{for } n = \prod\_i p\_i^{e\_i}.

- **Legendre’s formula.** v\_p(n!) = \sum\_{k=1}^\infty \left\lfloor \frac{n}{p^k} \right\rfloor (exponent of prime p in n!).

- **Lifting The Exponent (LTE), common case.** For odd prime p | (a − b) and n ∈ N,

v\_p(a^n - b^n) = v\_p(a - b) + v\_p(n).

Variants handle +, p = 2, and when p ∤ ab; useful in gcd/valuation problems.

E.5 Primitive Roots & Orders

- **Existence.** A primitive root modulo m exists iff m ∈ {1, 2, 4, p^k, 2p^k} for odd prime p.
- **Order divides totient.** If gcd(a, m) = 1, the multiplicative order ord\_m(a) divides φ(m); a^k ≡ 1 (mod m) ⇔ ord\_m(a) | k.

E.6 Combinatorics Mod Prime

- **Lucas’s theorem.** For prime p and n = \sum n\_i p^i, k = \sum k\_i p^i in base p,

\binom{n}{k} \equiv \prod\_i \binom{n\_i}{k\_i} \pmod{p}.

- **Kummer’s theorem.** v\_p(\binom{n}{k}) equals the number of carries when adding k and n − k in base p.

E.7 Base Representation & Digit Properties

- **Horner evaluation.** If N = (d\_ℓ d\_{ℓ−1} ... d\_0)\_b, then

N = (((d\_ℓ b + d\_{ℓ−1})b + d\_{ℓ−2}) ... )b + d\_0.

Compute N mod m via r := 0, then r = (r · b + d\_i) mod m.

- **Divisibility rules (any base b ≥ 2).**

N \equiv \sum\_i d\_i \pmod{b-1},

N \equiv \sum\_i (-1)^i d\_i \pmod{b+1} \quad (\text{LSB } i = 0).

Use: Quick checks for (b − 1) or (b + 1) divisors; rolling hash invariants.

- **Digit DP invariant.** Counting numbers by digit constraints in base b typically uses state (pos, tight, carry/sum mod M, ...) with transitions O(b) per digit.
- **Base change.** Value is preserved: (d)\_b = \sum\_i d\_i b^i = \sum\_j e\_j c^j = (e)\_c. For large integers in arrays, use chunk base B = 10^k to keep operations in 64-bit.

E.8 Sieve & Multiplicative Precomputation

- **Eratosthenes.** Marks primes ≤ n in O(n log log n). Linear sieve computes primes and SPF in O(n).
- **Multiplicative DP.** For multiplicative f, precompute f(p^e) on the fly with SPF; f(nm) = f(n)f(m) if gcd(n, m) = 1.

E.9 Miscellaneous Handy Facts

- **CRT-friendly mod split.** For M = 10^k, choose M = M\_1 M\_2 with M\_1, M\_2 coprime (e.g. 2^a 5^b separated) and work mod M\_1 and M\_2 then recombine via CRT.
- **Pigeonhole for cycles.** Sequences defined by x\_{i+1} ≡ f(x\_i) (mod m) enter a cycle by step ≤ m + 1 (useful for detecting periods).
- **Binomial mod composite.** Use prime-power decomposition m = \prod p\_i^{e\_i}, compute \binom{n}{k} mod p\_i^{e\_i} with factorials excluding p\_i and v\_{p\_i} via Legendre, then CRT.

E.10 Planar Graph: Definition: Kuratowski’s theorem

A finite graph is planar if and only if it does not contain a subgraph that is a subdivision of the complete graph K\_5 or the complete bipartite graph K\_{3,3}. If a graph G = (V, E) is planar, the following hold:

- |E| ≤ 3|V| − 6
- f ≤ 2|V| − 4
- (∄ cycle of size 3) → |E| ≤ 2|V| − 4

Euler’s formula

If a graph is connected, planar, and finite:

v - e + f = 2

Subdivision

A subdivision of a graph is obtained by inserting a vertex on an edge 0 or more times. (a graph is a subdivision of itself)

Wagner’s theorem

A finite graph is planar if and only if it does not have K\_5 or K\_{3,3} as a minor.

## Minor

A minor of a graph is obtained by contracting an edge into a vertex 0 or more times.

Both being a minor or and being a subdivision of partial orders on graphs up to isomorphism, i.e. only quasi-ordered (Robertson-Seymour theorem)

## Topological minor

Subdivision appear as subgraph

## Homeomorphic graph

Containing common subdivision

## F Boolean-to-ILP Encoding Reference

### F.1 Binary encodings of logical constraints

1. Express logical constraints using FOL formula
2. Convert FOL formula to grounded propositional logic formula
3. Binary encoding

| Boolean Expression                                     | Linear Constraint                        |
|--------------------------------------------------------|------------------------------------------|
| $\bigvee_{i \in I} A_i$                                | $\sum_{i \in I} A_i \geq 1$              |
| $\bigwedge_{i \in I} A_i$                              | $\sum_{i \in I} A_i =  I $               |
| $\neg \left( \bigvee_{i \in I} A_i \right)$            | $\sum_{i \in I} A_i = 0$                 |
| $\neg \left( \bigwedge_{i \in I} A_i \right)$          | $\sum_{i \in I} A_i \leq  I  - 1$        |
| $\left( \bigvee_{i \in I} A_i \right) \rightarrow B$   | $ I B - \sum_{i \in I} A_i \geq 0$       |
| $\left( \bigwedge_{i \in I} A_i \right) \rightarrow B$ | $B - \sum_{i \in I} A_i \geq -( I  - 1)$ |
| $A \rightarrow \left( \bigvee_{i \in I} B_i \right)$   | $\sum_{i \in I} B_i - A \geq 0$          |
| $A \rightarrow \left( \bigwedge_{i \in I} B_i \right)$ | $\sum_{i \in I} B_i -  I A \geq 0$       |

Table 1: Generalized Boolean expressions using binary variables.

All variables in the table are binary. These encode constraints on assignments; an LP relaxation need not preserve logical equivalence.

## G Flow Extensions: Lower Bounds and Demands

This construction extends the basic flow reductions. It is useful reference material; the supplied lecture only briefly mentions these extensions. Suppose each arc must satisfy  $\ell_e \leq f_e \leq c_e$ , and vertices require *net inflow*  $\text{in}_f(v) - \text{out}_f(v) = d(v)$ . Here  $d(v) > 0$  means demand and  $d(v) < 0$  means supply. Check  $0 \leq \ell_e \leq c_e$  and  $\sum_v d(v) = 0$  first. Set  $f_e = \ell_e + g_e$ , so  $0 \leq g_e \leq c_e - \ell_e$ , and define

$$d'(v) = d(v) - \sum_{e \in \delta^-(v)} \ell_e + \sum_{e \in \delta^+(v)} \ell_e.$$

Then  $g$  must satisfy  $\text{in}_g(v) - \text{out}_g(v) = d'(v)$ . Add a super-source  $\sigma$  and super-sink  $\tau$ . For  $d'(v) < 0$ , add  $\sigma \rightarrow v$  of capacity  $-d'(v)$ ; for  $d'(v) > 0$ , add  $v \rightarrow \tau$  of capacity  $d'(v)$ . Let  $B = \sum_{d'(v) > 0} d'(v) = \sum_{d'(v) < 0} -d'(v)$ . There is a feasible original flow iff the auxiliary maximum flow has value  $B$ , saturating all super-source and super-sink arcs.

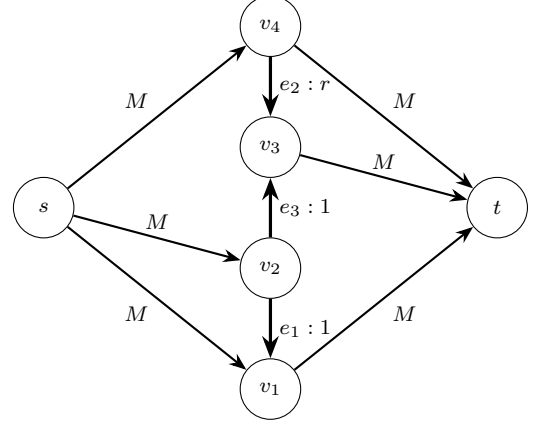
**Why this works.** A valid  $g$  extends to such a flow by placing  $-d'(v)$  or  $d'(v)$  on its auxiliary arc, restoring conservation. Conversely, saturating those arcs and applying conservation at  $v$  gives net original inflow  $d'(v)$ . Deleting auxiliary arcs and adding  $\ell$  recovers a feasible  $f$ . Integral data give an integral witness. For lower-bounded  $s$ - $t$  flow with unconstrained nonnegative value, add an auxiliary  $t \rightarrow s$  arc with a sufficiently large capacity and solve circulation feasibility; its flow records the  $s$ - $t$  value. To maximize it, remove the artificial arc and the super-source/sink arcs, then augment from  $s$  to  $t$  in the lower-bound residual network. Its forward capacity is  $c_e - f_e$  and its backward

capacity is  $f_e - \ell_e$ , so cancellation never violates the lower bound. Continue until no augmenting path remains. Feasibility alone does not certify maximum value.

## H Additional Algorithm Examples

### H.1 An infinite Ford–Fulkerson augmentation sequence

Ford–Fulkerson can fail to terminate with irrational capacities and adversarial path choices. In this construction let  $r = (\sqrt{5} - 1)/2$ , so  $r^2 = 1 - r$ , and choose  $M \geq 6$ . The detailed infinite sequence is an extension of the core termination discussion.



**Pathological augmenting-path sequence.** First choose the initial augmenting path

$$P_0 = (s, v_2, v_3, t),$$

which sends one unit of flow and saturates  $e_3$ . Define the following residual augmenting paths:

$$P_1 = (s, v_4, v_3, v_2, v_1, t),$$

$$P_2 = (s, v_2, v_3, v_4, t),$$

$$P_3 = (s, v_1, v_2, v_3, t).$$

Notice that some edges in these paths are *reverse residual edges* rather than original edges of  $G$ . In particular,

$$P_1 : v_3 \rightarrow v_2 \text{ is the reverse residual edge of } e_3,$$

$$P_2 : v_3 \rightarrow v_4 \text{ is the reverse residual edge of } e_2,$$

$$P_3 : v_1 \rightarrow v_2 \text{ is the reverse residual edge of } e_1.$$

After  $P_0$ , repeatedly choose

$$P_1, P_2, P_1, P_3$$

indefinitely. Thus the complete sequence is

$$P_0, (P_1, P_2, P_1, P_3), (P_1, P_2, P_1, P_3), (P_1, P_2, P_1, P_3), \dots$$

If the residual capacities of  $(e_1, e_2, e_3)$  are initially

$$(a_n, a_{n+1}, 0),$$

then one four-path cycle gives

$$\begin{aligned} (a_n, a_{n+1}, 0) &\xrightarrow{P_1} (a_{n+2}, 0, a_{n+1}) \\ &\xrightarrow{P_2} (a_{n+2}, a_{n+1}, 0) \\ &\xrightarrow{P_1} (0, a_{n+3}, a_{n+2}) \\ &\xrightarrow{P_3} (a_{n+2}, a_{n+3}, 0). \end{aligned}$$

Here

$$a_{n+2} = a_n - a_{n+1}, \quad a_0 = 1, \quad a_1 = r.$$

Since

$$r^2 = 1 - r,$$

we have

$$a_n = r^n > 0$$

for every finite  $n$ . Therefore every cycle leaves another positive-capacity augmenting path, so Ford–Fulkerson never terminates.

**Why the outer arcs never become the bottleneck.** After the initial augmentation of size 1, the augmentation amounts are  $r, r, r^2, r^2, r^3, r^3, \dots$ . Their total is  $1 + 2r/(1 - r) < 5$ . Thus every capacity- $M$  outer arc retains forward residual capacity greater than 1 when  $M \geq 6$ , while every prescribed bottleneck is at most 1. This validates the entire infinite sequence, not just the recurrence on the three special arcs.

## H.2 Random-endpoint vertex cover: an optional expectation proof

For an unweighted graph, repeatedly choose any uncovered edge and select one of its two endpoints uniformly at random, then remove all edges incident to the selected vertex. Return the selected vertices. Each iteration removes at least its chosen edge, and every removed edge is covered, so the output is feasible.

Fix a minimum cover  $C^*$ . There are at most  $n$  iterations, since selected vertices are distinct. For each  $i \in \{1, \dots, n\}$ , let  $A_i$  indicate that iteration  $i$  occurs, and let  $Y_i$  indicate that it selects a vertex of  $C^*$ ; after termination set both to zero. Conditional on the history of any active iteration, the chosen edge has at least one endpoint in  $C^*$ , so

$$\mathbb{E}[Y_i \mid \text{history}] \geq \frac{1}{2} A_i.$$

Taking expectations and summing gives

$$\frac{1}{2} \mathbb{E}[|C|] = \frac{1}{2} \sum_i \mathbb{E}[A_i] \leq \mathbb{E} \left[ \sum_i Y_i \right] \leq |C^*|.$$

Thus  $\mathbb{E}[|C|] \leq 2 \text{OPT}$ . This does not claim a factor-2 bound for every run. Maintaining active edges and removing a vertex's incident edges gives polynomial time. The simpler maximal-matching algorithm in the main notes is deterministic and has the same approximation factor.